Job Role 👍 ⇒ Job

Web Designer / Developer ⇒ UI / UX
Front End Developer
Backend Developer
JS Developer
React js Developer
Mysql Admin Trainee
Html & xml Developers
Full Stack Developer ⇒ Software Developer

JD 👍 ⇒ Statements

Data ⇒ Raw Facts ⇒ 0 & 1 ⇒ Collection of inFormations ⇒ collection details

DB ⇒ Data Base ⇒ Storage ⇒ Only stored the data's

Data Type ⇒ structural data (Primitive Data) & unStructural Data (non primitive or Reference Data) ⇒ Semi structural

Primitive Data Type	Non Primitive Data Type
Number ⇒ [0,1,2,3,4…….n] ⇒ n(n+1)/2 Integer ⇒ int ,float,double,long integer,short integer,long **String** ⇒ [a,b,c,d,e……..z] Char⇒ a,b,c charset⇒ add,sub,sum ,characters⇒ addition **Boolean ⇒ [True,False] ⇒ [1 & 0]** **Undefined ⇒ [?] ⇒ may be getting a value ⇒ a=10 , b =** **Null ⇒ [{ }]** **Bigint ⇒ bigInt ⇒ [(2n*3)]**	**Object ⇒ ={ }** Function ⇒ (){ } Array ⇒ =[ ]

10000000000000000000000000000000 = bigint(10000) 7502132895 ⇒ bigInt(75021) Symbol ⇒ [@,#,$,%,&, ,,.(,)]	

OOP'S ⇒ c & c++

⇒ Object Oriented Programming language (System)

⇒ Class ⇒ Blue Print ⇒ Group of Objects ⇒ car ,Bike,student,Biriyani

⇒ Object ⇒ instance of Class ⇒ it is stand alone entity ⇒ Real world things ⇒ property & methods ⇒ alto800, pulsar, full stack ,Egg biriyani

⇒ Abstraction ⇒ Hide ⇒ need time only explore the functions or operation

⇒ Encapsulation ⇒ Binding ⇒ Join oe merge ⇒ oBject & methods

⇒ Inheritance ⇒ Parent Child concepts ⇒ Derivation or Construction ⇒ Separation from parent class to child class

Class (child)className extends parentClassName { }

⇒ Hierarchy Inheritance ⇒ parent ⇒ 2 child
⇒ Multilevel Inheritance ⇒ parent ⇒ child (parent) ⇒ child (parent)
⇒Single Level Inheritance ⇒ parent ⇒ child

⇒ Hierarchy Inheritance ⇒ parent ⇒ 2 child ⇒ mobile

Class mobile extends ElectronicItem{ }

Class android extends Mobile{ }

Class Ios extends Mobile{ }

Class Mac extends Mobile{ }

⇒ Multilevel Inheritance ⇒ parent ⇒ child (parent) ⇒ child (parent)

```
Class android extends Mobile{ }

Class Xomi extends Android { }

Class Redmi  extends Xomi { }

Class Note9pro  extends Redmi { }
```

⇒Single Level Inheritance ⇒ parent ⇒ child

```
Class Redmi8a extends Redmi { }
```

⇒ Polymorphism ⇒ One Interface and many implementation ⇒

Languages 👍

Coding Things ⇒ Scripting Languages & Core Languages

Scripting Languages	Core Languages
ECMA script ⇒ Es ,Javascript ⇒ Js , Reactjs ,angular Js ,vue js,html, css,..etc…	c&c++,php,java,python,nodejs,.Net ⇒ #c, Ruby, etc..
Syntex ,format,rule we can break or overcome	Syntex ,format,rule we can’t break or overcome
Javascript ⇒ logic ,operations ⇒ only focus to front end	Java ⇒ core lang ⇒ only works with logic and problematic things in coding (backend)
Browser only understand the scripting language	Can’t Browsar

Pattern or Archi structure 🙂 ⇒ 10

⇒ MVC ⇒ Full stack

M ⇒ Model	**V ⇒ View**	**C ⇒ Controller**
DB ⇒ Database	**FE ⇒ Front End**	**BE ⇒ Backend**
Schema	**UI/ux**	**UX/ui**
DBMS/RDBMS	**Client/userside**	**Server/developmentside**

Logical Operators ⇒ AND ,OR,NOT ⇒ &&, || , !
Comparison Operator ⇒ == or === a =10 , a=200, b="200" ⇒ a==b a===b
Ternary Operator ⇒ ?

Statements ⇒ Controller or condition statements(4) and Looping statements (5)

Condition Statements	Looping Statements
if() ⇒ success If else() ⇒if ⇒ true(max) else ⇒ false (max) Else if() ⇒ nested if switch() ⇒ case	for(){} for(of){} for(in){} While (){} Do while(){}

Operation

z=a+b; ⇒ Operation ⇒ Addition / Operator ⇒ + = / Operands ⇒ key words ⇒ z,a,b

Access specifier ⇒ Public <void> / Private / Productive #

map(), sort(),filter(),find(),forEach()

DSA ⇒ Data Structure Algorithm

Data (Raw facts or 0&1) ⇒ Meaning less

Data Type ⇒ 2 +1 ⇒ Give some definition

Structure Data unStructure Data Semi Structure Data
Primitive Data Non Primitive (Reference)
7 3

Object Function Array

Array List File

Linear Data type Non linear Data type

Array linked list Stack Queue Tree Graph

Hash table singly,doubly,circley linked list FIFO LIFO Set Map

Algorithm

Algorithm ⇒ Both understanding ⇒
Procedure ⇒ Human understanding
pseudocode ⇒ focus to machine

Full Stack 😂

Combination of Front end backend and Data bases project making process.

Full stack is process of Development and Deployment of Web Related Project
Project ⇒ Application, Website, software

Project ⇒ step by step
Development ⇒ Built an application
Deployment ⇒ Out put ⇒ user intr\erface

Full stack ⇒ pattern

MVC
Model ⇒ View ⇒ Controller

Model	View	Controller
Database ⇒ schema	Front end ⇒ UI/ux	Backend ⇒ ui/UX
DB	Client / User side	Server / server side
Oracle,sql,mysql,postsql,no sql	html,css,js,bootatrap,tailwind,react,angular,vue,jquery	C,c++,php,java,python,node,ruby,core java,.net
Mysql / No sql	Html,Css,Javascript,Bootstrap,Reactjs	Java ⇒ spring Python ⇒ django .net ⇒ c#, asp servlet Nodejs ⇒ MERN

Front End 👍

Html,Css,Javascript,Bootstrap,Reactjs

Html Css JS ⇒ Browsar understand only the scripting languages ⇒ xml

IDE's ⇒ Env ⇒ Tool , Platform

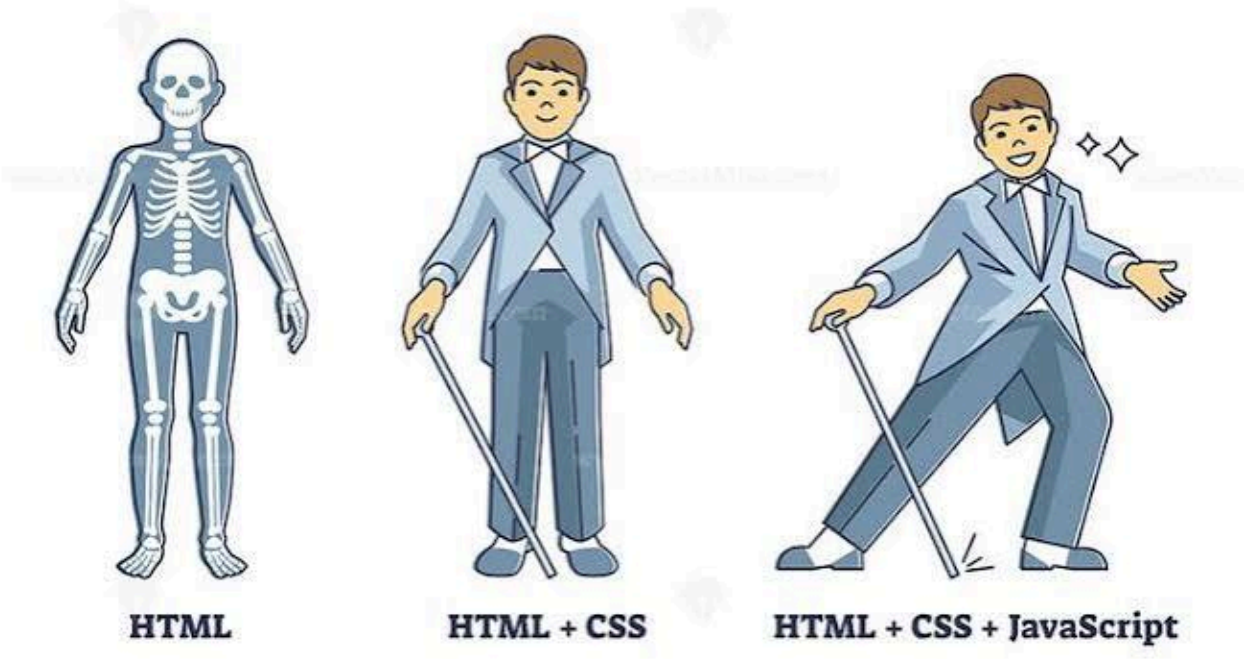
Html Notes

Html ⇒

Hyper TExt Markup Language

To be Give a structure of Web Pages

Html Tags or Elements to help the creating a structure of web Pages



Part of Modules and Section in web pages

Elements 😀 ⇒ Tags
- **Block Level Elements**
- **Inline Level Elements**

Tags 👍 ⇒ Elements
- **Self Closing tags**
- **Non Self Closing tags**

Html ⇒ Tags ⇒ < > < / > ⇒ Angular Brackets
Tags ⇒ < > < / > ⇒ Angular Brackets

<> ⇒ Open Tags
</> ⇒ Closing tags

Html ⇒ 200 above ⇒ 100 ⇒ 50 tags ⇒ https://www.w3schools.com/tags/default.asp

Fundamental tags or Basic **<html>, <head>,<body>,<meta>,<link>,<title>**	**Basic Tag or important tags** **<h1> to <h6>, <p>,<img>,<a>, ,**
Formatting tags **<hr>, ,<b>,,,<i>,<sup>, <sub>, etc..**	**Function tags or Main tags important tags** **<form>,<table>, , ,<dl>,<input>, <label>,<select>,<tr>,<td>,<th>, **
Multimedia tags https://pixabay.com/ **<img>,<auido>,<video>,<iframe>,<canvas>, <marquee>**	**Semantic tags or html 5 tags** **<audio>,<video>,<iframe>,<canvas>, <marquee>,<section>,<article>,<aside>, <header>,<footer>,<nav>, etc..**

https://www.mapembed.org/?gad_source=1&gad_campaignid=22280139497&gbraid=0AAAAA98a5lNp9ItHluXLxubkbB6wjlApz&gclid=CjwKCAjwzNTUBhAjEiwA7zcvWvFqTY37hu6Gy339HXGprRy7uBA4SSvc9EcWW_RmL5KvjpPyAuw6txoCn3kQAvD_BwE

Elements 😀 ⇒ Tags
All tags or Each an dEvery tags are called Elements

Block Level Elements
Inline Level Elements

Block Level Elements	Inline Level Elements
Takes entire Width of web pages	Doesn't take entire width of webpage
Start New Lines	Doesn't start new lines
Ex: h1,h2,p,div	Ex : img,a,span

Tags 👍 ⇒ Elements

Open and Close angular brack keyword are called Tags

<> </> ⇒ <h1>,<p> ,<img>

Self Closing tags

Non Self Closing tags

Self Closing tags	Non Self Closing tags
It's have closing tags	It's doesn't have closing tags
Ex: <img>	<h1></h1>,<p>

Attributes 👍

Addition or extra information about the Elements.

In Opening tags contains attributes.

Common Attributes: style,id,class,name,type

Ex: <img src="" alt=""> <h1>

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="shortcut icon" href="car.png" type="image/x-icon">
    <title>BMW Car's</title>
</head>

<body>
    <div>
        <h1>BAsic Tag <span style="color: blue;">s</span> </h1>
        <!-- <h1>Hello world</h1>
```

```
        <h2>hello world</h2>
        <h3>hello world</h3>
        <h4>hello world</h4>
        <h5>hello world</h5>
        <h6>hello world</h6>

        <p>hello world</p>
        <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.
Suscipit quos modi cumque illum est debitis
            exercitationem veritatis impedit quasi consequatur, ex quam
totam magnam facere itaque similique corrupti!
            Excepturi, perspiciatis.</p>

        <p>It is a long established fact that a reader will be distracted
by the readable content of a page when looking
            at
            its layout. The point of using Lorem Ipsum is that it has a
more-or-less normal distribution of letters, as
            opposed to using 'Content here, content here', making it look
like readable English. Many desktop publishing
            packages and web page editors now use Lorem Ipsum as their
default model text, and a search for 'lorem
            ipsum'
            will uncover many web sites still in their infancy. Various
versions have evolved over the years, sometimes
            by
            accident, sometimes on purpose (injected humour and the
like).</p>

        <p>Lorem ipsum dolor sit amet consectetur adipisicing elit.
Possimus, odit. Architecto mollitia accusantium
            quam!
            Laudantium accusantium eius, velit voluptatibus doloremque
minima maiores eaque dicta dignissimos fugit
            sapiente, aliquam laborum recusandae.</p> -->

        <!-- <img
src="https://a.storyblok.com/f/197805/7a2484c876/how-to-create-a-car-newbl
ogcover.png" alt=""> -->
        <!-- <img src="car.png" alt="car pink " width="300px">
```

```
        <a href="https://www.bmw-kunexclusive-chennai.in/">bmw</a>
        <a href="https://www.bmw-kunexclusive-chennai.in/"
target="_blank">bmw new tab</a>

        <a href="https://www.bmw-kunexclusive-chennai.in/"
target="_blank"> <img src="car.png" alt="car pink "
                width="300px"> </a> -->
    </div>

    <div>
        <!-- comment lines  -->
    </div>

    <img>
    <h1>Tags concept '<span style="color: blue;">s</span></h1>
    <!--
    <h1>hello world</h1>  non  self closing tags
    <p>hello wolrd</p>

    <img src="car.png" width="100px"> self closing tags
    <br>
    <hr>
    <input type="text">
    -->
    </div>

    <div>
        <h1>Element concept '<span style="color: blue;">s</span></h1>

        <!-- <h1>Ganapathi Raja</h1>

        <h1>Ganapathi <p style="color: red; font-size: 12px;">Raja</p>
        </h1>

        <h1>Ganapathi <span style="color: red; font-size: 12px;"> Raja
</span> </h1>

        <h1>hello <i style="color: blueviolet;">world</i></h1> -->
    </div>
```

```
    <div>
        <h1>Basic Styling concept '<span style="color:
blue;">s</span></h1>

        <p>heLlo wold</p>


        <p style="
            color: red;
            background-color: bisque;

            font-size: 46px;
            font-weight: bold;
            font-style: italic;
            font-family: 'Gill Sans', 'Gill Sans MT', Calibri, 'Trebuchet
MS', sans-serif;

            text-align: center;
            text-decoration: underline;
            text-transform: uppercase;
            text-shadow: 5px 0px 0px rgb(0, 0, 0);

            border: 2px solid blueviolet;
            padding: 50px;
            margin: 100px;
        ">heLlo wold</p>
    </div>
</body>

</html>
```

Function tags or Main tags important tags

<form>,<table>,<ol>,<ul>,<dl>,<input>,
<label>,<select>,<tr>,<td>,<th>,<li>

Form Tags

```
<div>
        <h2>Form Tag <span style="color: blue; font-style: italic;">
Concepts </span></h2>

        <!-- <form action=""></form>
        <form action="" method="get"></form>
        <form action="" method="post"></form>
        <form></form> -->

        <form style="border: 2px solid whitesmoke; padding: 50px;
border-radius: 10px; width: 600px;">
            <h1>Students Registartion Form </h1>
            <label for="sname"> Students Name </label> <br>
            <input type="text" name="snameaswini" id="studnetsname"
placeholder="Enter your Full name" required>

            <br><br>

            <label for="smobile">Students Mobile</label><br>
            <input type="tel" name="smobileloveeee" id="studentsmobile"
placeholder="Enter your mobile number" required>

            <br><br>

            <label for="sgmail">Students Gmail</label><br>
            <input type="email" name="semial" id="studentsemail"
placeholder="Enter your Email" required pattern="">
```

```
            <br><br>
            <label for="sname"> Students Father name </label> <br>
            <input type="text" name="sname" id="studnetsname"
placeholder="Enter your Full name" required>

            <br><br>
            <label for="sfathername"> Students Fathers Name </label> <br>
            <input type="text" name="sfathername" id="studnetsfathername"
placeholder="Enter your Full Fathers name">

            <br><br>
            <label for="sname"> Students DOB </label> <br>
            <input type="date" name="sdob" id="studnetsdob">

            <br><br>
            <label for="sname"> Students DOB </label> <br>
            <input type="datetime" name="sdobss" id="studnetsdob">

            <br><br>
            <label for="sname"> Students DOB </label> <br>
            <input type="datetime-local" name="sdobssss" id="studnetsdob"
disabled>

            <br><br>
            <label for="sname"> Students Reg Number </label> <br>
            <input type="number" name="sreg" id="studnetsdob" readonly>

            <br><br>
            <label for="sgender"> Students Gender </label> <br>
            <input type="radio" name="sgender" id="sothers"><label
for="sgender">Others</label>
            <input type="radio" name="sgender" id="sMale"><label
for="sMale">Male</label>
            <input type="radio" name="sgender" id="sFemale"><label
for="sFemale">Female</label>

            <br><br>
            <label for="sgender"> Students Gender </label> <br>
            <select name="gender" id="sgender">
                <option value="Choose">choose a gender</option>
```

```
                <option value="female">Female</option>
                <option value="male">Male</option>
                <option value="others">Others</option>
            </select>

            <br><br>

            <label for="ssubject">Students Subjects</label><br>
            <input type="checkbox" name="ssubject" id="sfullstack"><label
for="sfullstck">Full stck</label>
            <input type="checkbox" name="ssubject" id="sfrontend"><label
for="sfullstck">FRont End </label>
            <input type="checkbox" name="ssubject" id="sbackend"><label
for="sfullstck">Backend</label>
            <input type="checkbox" name="ssubject" id="sdb"><label
for="sfullstck">Db</label>
            <input type="checkbox" name="ssubject" id="spython"><label
for="sfullstck">python</label>
            <input type="checkbox" name="ssubject" id="sjava"><label
for="sfullstck">java</label>

            <br><br>
            <label for="ssubject">Students photo</label>
            <!-- <input type="image" src="" alt=""> -->
            <input type="file" name="simage" id="sphoto">

            <br><br>

            <label for="ssubject">Students password</label> <br>
            <input type="password" name="spass" id="spassword"
placeholder="enter your password">

            <br><br>
            <label for="ssubject">Students Address</label> <br>
            <textarea name="saddress" id="sadd" placeholder="Enter your
address"></textarea>

            <br><br>

            <input type="button" value="Register">
```

```
            <input type="submit" value="Submit">
            <button>Confirm your Registration</button>
            <a href="#"
                style="color: black; text-decoration: none;border: 1px
solid; padding: 5px; border-radius: 5px;">Send
                Data</a>
        </form>
    </div>
```

```
 <div>
        <h2>Table Tag <span style="color: blue; font-style: italic;">
Concepts </span></h2>
        <!-- <table>
            <thead>
                <td></td>
            </thead>
            <tbody>
                <tr>
                    <td></td>
                </tr>
            </tbody>
        </table>
        <table>
            <tr>
                <th></th>
            </tr>
            <tr>
                <td></td>
            </tr>
        </table> -->

        <style>
            table {
                border-collapse: collapse;
                width: 600px;
            }

            table tr,
            th,
```

```
        td {
            border: 1px solid;
            padding: 10px;
        }

        /* table tr:nth-child(1) {
            background-color: azure;
            color: blue;
        }

        tr:nth-child(3) {
            background-color: blanchedalmond;
        }


        table tr:nth-child(odd) {
            background-color: aqua;
        }

        table tr:nth-child(even) {
            background-color: blueviolet;
        } */
    </style>
    <!-- <table border="1px" bgcolor="grey" cellpadding="10px"> -->
    <table>
        <tr>
            <th>sid</th>
            <th>sName</th>
            <th>sMobile</th>
            <th>sEmail</th>
            <th>sSubject</th>
        </tr>
        <tr>
            <td>001</td>
            <td>Ganapathi</td>
            <td>7502132895</td>
            <td>gnapathi@gmail.com</td>
            <td>Full stack</td>
        </tr>
        <tr>
```

```
            <td>002</td>
            <td>Raja</td>
            <td>7502132896</td>
            <td>raja@gmail.com</td>
            <td>Front ENd</td>
        </tr>
        <tr>
            <td>003</td>
            <td>Ravi</td>
            <td>7502132885</td>
            <td>ravi@gmail.com</td>
            <td>python</td>
        </tr>
        <tr>
            <td>004</td>
            <td>Pavi</td>
            <td>7532132895</td>
            <td>pavi@gmail.com</td>
            <td>Full stack</td>
        </tr>
        <tr>
            <td>005</td>
            <td>aswini</td>
            <td>9502132895</td>
            <td>aswini@gmail.com</td>
            <td>back stack</td>
        </tr>
        <tr>
            <td>006</td>
            <td>yuva</td>
            <td>7602132895</td>
            <td>yuva@gmail.com</td>
            <td>Full stack</td>
        </tr>
    </table>

    <!-- <table>
        <tr>
            <th>sid</th>
            <th>sName</th>
```

```
            <th>sMobile</th>
            <th>sEmail</th>
            <th>sSubject</th>
        </tr>
        <tr>
            <td>001</td>
            <td>Ganapathi</td>
            <td colspan="2">gnapathi@gmail.com</td>
            <td>Full stack</td>
        </tr>
        <tr>
            <td>002</td>
            <td>Raja</td>
            <td>7502132896</td>
            <td>raja@gmail.com</td>
            <td>Front ENd</td>
        </tr>
        <tr>
            <td>003</td>
            <td>Ravi</td>
            <td>7502132885</td>
            <td>ravi@gmail.com</td>
            <td>python</td>
        </tr>
        <tr>
            <td>004</td>
            <td>Pavi</td>
            <td>7532132895</td>
            <td>pavi@gmail.com</td>
            <td>Full stack</td>
        </tr>
        <tr>
            <td>005</td>
            <td>aswini</td>
            <td>9502132895</td>
            <td>aswini@gmail.com</td>
            <td>back stack</td>
        </tr>
        <tr>
            <td>006</td>
```

```
            <td>yuva</td>
            <td>7602132895</td>
            <td>yuva@gmail.com</td>
            <td rowspan="2">Full stack</td>
        </tr>
        <tr>
            <td>007</td>
            <td>rani</td>
            <td>6602132895</td>
            <td>Rani@gmail.com</td>
        </tr>
    </table> -->

    <!-- <table>
        <tr>
            <th>sid</th>
            <th>sName</th>
            <th>sMobile</th>
            <th>sEmail</th>
            <th>sSubject</th>
        </tr>
        <tr>
            <td>001</td>
            <td>Ganapathi</td>
            <td>7502132895</td>
            <td>gnapathi@gmail.com</td>
            <td>Full stack</td>
        </tr>
        <tr>
            <td>002</td>
            <td>Raja</td>
            <td>7502132896</td>
            <td>raja@gmail.com</td>
            <td>Front ENd</td>
        </tr>
        <tr>
            <td>003</td>
            <td>Ravi</td>
            <td>7502132885</td>
            <td>ravi@gmail.com</td>
```

```
            <td>python</td>
        </tr>
        <tr>
            <td>004</td>
            <td>Pavi</td>
            <td>7532132895</td>
            <td>pavi@gmail.com</td>
            <td>Full stack</td>
        </tr>
        <tr>
            <td>005</td>
            <td>aswini</td>
            <td>9502132895</td>
            <td>aswini@gmail.com</td>
            <td>back stack</td>
        </tr>
        <tr>
            <td>006</td>
            <td>yuva</td>
            <td>7602132895</td>
            <td>yuva@gmail.com</td>
            <td>Full stack</td>
        </tr>
        <tr>
            <td>001</td>
            <td>Ganapathi</td>
            <td>7502132895</td>
            <td>gnapathi@gmail.com</td>
            <td>Full stack</td>
        </tr>
        <tr style="background-color: aquamarine;">
            <td>002</td>
            <td>Raja</td>
            <td>7502132896</td>
            <td>raja@gmail.com</td>
            <td>Front ENd</td>
        </tr>
        <tr>
            <td>003</td>
            <td>Ravi</td>
```

```
                <td>7502132885</td>
                <td>ravi@gmail.com</td>
                <td>python</td>
            </tr>
            <tr>
                <td>004</td>
                <td>Pavi</td>
                <td>7532132895</td>
                <td>pavi@gmail.com</td>
                <td>Full stack</td>
            </tr>
            <tr>
                <td>005</td>
                <td>aswini</td>
                <td>9502132895</td>
                <td>aswini@gmail.com</td>
                <td>back stack</td>
            </tr>
            <tr>
                <td>006</td>
                <td>yuva</td>
                <td>7602132895</td>
                <td>yuva@gmail.com</td>
                <td>Full stack</td>
            </tr>
        </table> -->
    </div>
```

```
<div style="border: 2px solid whitesmoke; padding: 50px; border-radius:
10px; width: 600px;">
        <h2>List Tags <span style="color: blue; font-style: italic;">
Concepts </span></h2>
        <div>
            <h3>Order List</h3>
            <ol>
                <li>Ganapathi</li>
                <li>Yuva</li>
                <li>Pavi</li>
                <li>Aswini</li>
```

```
        <li>Ravi</li>
        <li>Raja</li>
    </ol>

    <ol type="1">
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
    </ol>

    <ol type="A">
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
        <li>Ganapathi</li>
        <li>Yuva</li>
```

```
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
    </ol>

    <ol type="a">
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
    </ol>

    <ol type="I">
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
```

```
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
    </ol>

    <ol type="i">
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
    </ol>


    <h3>unOrder List</h3>
    <ul>
        <li>Ganapathi</li>
        <li>Yuva</li>
        <li>Pavi</li>
        <li>Aswini</li>
        <li>Ravi</li>
        <li>Raja</li>
    </ul>
```

```
        <ul style="list-style-type: disc;">
            <li>Ganapathi</li>
            <li>Yuva</li>
            <li>Pavi</li>
            <li>Aswini</li>
            <li>Ravi</li>
            <li>Raja</li>
        </ul>
        <ul style="list-style-type: circle;">
            <li>Ganapathi</li>
            <li>Yuva</li>
            <li>Pavi</li>
            <li>Aswini</li>
            <li>Ravi</li>
            <li>Raja</li>
        </ul>
        <ul style="list-style-type: square;">
            <li>Ganapathi</li>
            <li>Yuva</li>
            <li>Pavi</li>
            <li>Aswini</li>
            <li>Ravi</li>
            <li>Raja</li>
        </ul>

        <ul style="list-style-type: none;">
            <li><span>😇</span> Ganapathi</li>
            <li> <span>😇</span> Yuva</li>
            <li> <span>😇</span> Pavi</li>
            <li> <span>😇</span> Aswini</li>
            <li> <span>😇</span> Ravi</li>
            <li> <span>😇</span> Raja</li>
        </ul>

        <h3>defintion or describtion list</h3>
        <dl>
            <dt>WHO</dt>
            <dd>World Health Org</dd>

            <dt>HTML</dt>
```

```
            <dd>Hyper Text Markup Language</dd>

            <dt>Css</dt>
            <dd>Cascading Style Sheet</dd>

            <dt>JS</dt>
            <dd>Java script</dd>
        </dl>

        <div>JS</div>
        <p style="padding-left: 15px;">Java script</p>
    </div>

    <div>
        <h3>Nested List</h3>
        <ol>
            <li>Ganapathi
                <ol>
                    <li>Yuva
                        <ul>
                            <li>Aswini</li>
                            <li>Ravi</li>
                            <li>Raja</li>
                        </ul>
                    </li>
                    <li>Pavi</li>
                    <li>Aswini</li>
                    <li>Ravi</li>
                    <li>Raja</li>
                </ol>
            </li>
            <li>Yuva</li>
            <li>Pavi</li>
            <li>Aswini</li>
            <li>Ravi</li>
            <li>Raja
                <ul>
                    <li>Yuva</li>
                    <li>Pavi
                        <ul>
```

```
                            <li>Aswini</li>
                        </ul>
                    </li>
                </ul>
            </li>
        </ol>
    </div>
    <div>
        <style>
            ul {
                /* display: flex; */
            }

            ul li {
                /* float: left; */
                /* float: right; */
                display: inline-block;
                list-style-type: none;
            }
        </style>
        <h3>Horizontal List</h3>
        <ul>
            <li>About us</li>
            <li>Technologies</li>
            <li>Career</li>
            <li>New & Media</li>
            <li>Event</li>
            <li><button>Get In Touch</button></li>
        </ul>
    </div>
</div>
```

```
<div style="border: 2px solid whitesmoke; padding: 50px; border-radius:
10px; width: 600px;">
        <h2> Multimedia tags <span style="color: blue; font-style:
italic;"> Concepts </span></h2>
        <img src="carred.jpg" alt="red Car" width="600px">
        <!-- <audio >
```

```
            <source src="audio.mp3">
        </audio> -->
        <audio src="audio.mp3" controls loop></audio>
        <!-- <audio src="audio.mp3" controls autoplay loop muted></audio>
-->

        <video src="car.mp4" controls autoplay loop muted></video>

        <iframe src="https://www.flipkart.com/" frameborder="0"></iframe>

        <div class="embed-map-fixed">
            <div class="embed-map-container"><iframe
class="embed-map-frame" frameborder="0" scrolling="no"
                    marginheight="0" marginwidth="0"

src="https://maps.google.com/maps?width=1200&height=400&hl=en&q=Chennai&t=
&z=14&ie=UTF8&iwloc=B&output=embed"></iframe><a
                    href="https://classicjoy.games"

style="font-size:2px!important;color:gray!important;position:absolute;bott
om:0;left:0;z-index:1;max-height:1px;overflow:hidden">Emulator
                    Games</a></div>
            <style>
                .embed-map-fixed {
                    position: relative;
                    text-align: right;
                    width: 1200px;
                    height: 400px;
                }

                .embed-map-container {
                    overflow: hidden;
                    background: none !important;
                    width: 1200px;
                    height: 400px;
                }

                .embed-map-frame {
                    width: 1200px !important;
                    height: 400px !important;
```

```
                }
            </style>
        </div>

        <iframe width="560" height="315"
src="https://www.youtube.com/embed/_sxHin3QjKs?si=5dZdCwhDGjaKFWPp"
            title="YouTube video player" frameborder="0"
            allow="accelerometer; autoplay; clipboard-write;
encrypted-media; gyroscope; picture-in-picture; web-share"
            referrerpolicy="strict-origin-when-cross-origin"
allowfullscreen></iframe>


        <marquee behavior="left" direction="">
            <span>Ganapathi</span>
            <span>PAvi</span>
            <span>Yuva</span>
            <span>larece</span>
        </marquee>



    </div>
```

```
<div style="border: 2px solid whitesmoke; padding: 50px; border-radius:
10px; width: 600px;">
        <h2>Html 5 or Sematic Tag <span style="color: blue; font-style:
italic;"> Concepts </span></h2>

        <header></header>
        <nav></nav>
        <footer></footer>
        <section></section>
        <article></article>
        <aside></aside>
        <marquee behavior="" direction=""></marquee>
        <canvas></canvas>
        <audio src=""></audio>
        <video src=""></video>
        <iframe src="" frameborder="0"></iframe>
```

```
        <!-- drag and drop  -->
    </div>
```

```
   <div style="border: 2px solid whitesmoke; padding: 50px; border-radius:
10px;">
        <h2>File Path <span style="color: blue; font-style: italic;">
Concepts </span></h2>

        <img src="yellowcar.jpg" alt="car" width="400px">
        <img src="./carred.jpg" alt="car" width="400px">

        <img src="assets/greencar.jpg" alt="car" width="400px">
        <img src="levon/imgs/redpolicecar.jpg" alt="car" width="400px">


        <img src="../Html/logo.png" alt="car" width="400px">

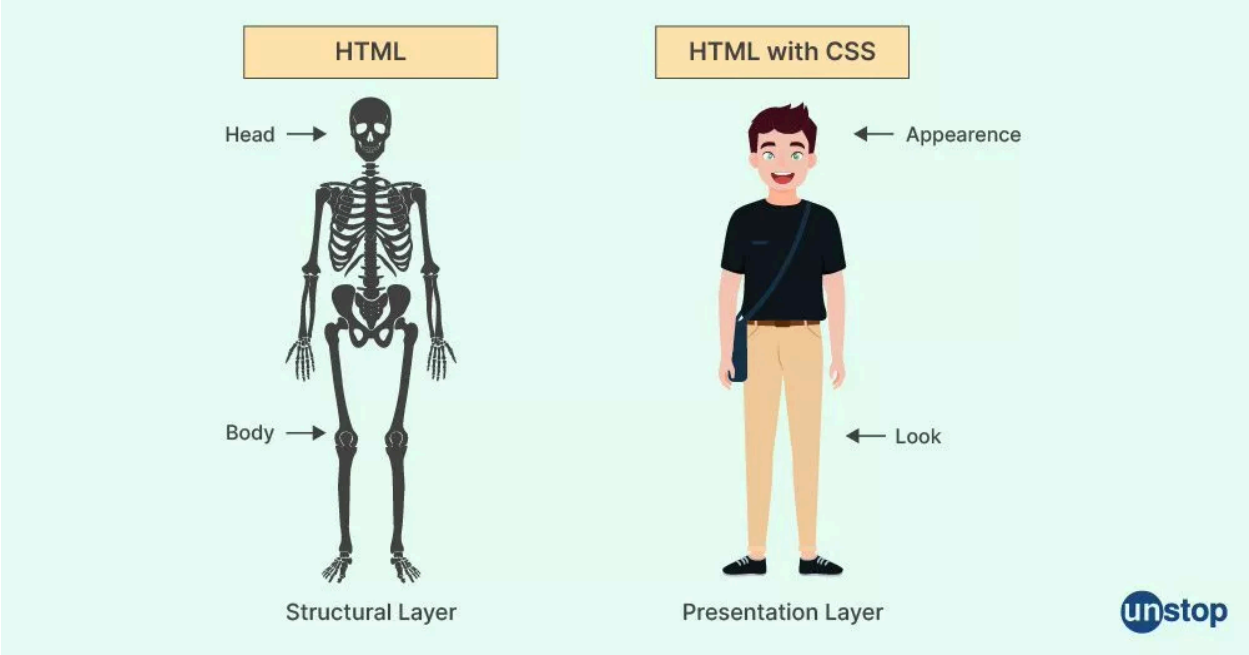



    </div>
```

Css Notes
Css ⇒ Cascading Style Sheet
⇒ Sass

⇒ .css ⇒ .scss

Why Css 🙂



Importing the css with Html 👍 (3)

Inline Styling Method ⇒ html line styling ⇒ style attributes

```
<!-- Inline Styling -->
    <div style="
        color: red;
        text-transform: uppercase;
        font-weight: bold;
        font-size: 28px;
    ">
        HelLo woRLD
    </div>
```

Internal Styling Method ⇒ Head part of html file ⇒ style tags

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <link rel="shortcut icon" href="css.png" type="image/x-icon">
    <title>Css Class 😇😇😎</title>
    <!-- internal Styling -->
    <style>
        div {
            color: red;
            text-transform: uppercase;
            font-weight: bold;
            font-size: 28px;
        }
    </style>
</head>
```

External Styling Method ⇒ separate css file (.css)

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>About css</title>
    <link rel="stylesheet" href="style.css">
</head>
```

Css Properties 👍

Selector Properties (5)

Name selector
Id selector
Class selector
Group selector
Universal selector

```
<style>
        /* Name Or Elements Selector  */
        h1 {
            color: rgb(247, 0, 255);
            font-size: 36px;
            background-color: aqua;
        }

        div {
            color: blue;
        }
```

```
        span {
            color: blueviolet;
        }

        /* # id selector  */
        #heading {
            color: pink;
            font-size: 45px;
            background-color: blueviolet;
        }

        #heading1 {
            font-style: italic;
        }

        /* . class selector  */
        .heading {
            color: rgb(255, 0, 43);
            font-size: 45px;
            background-color: rgb(38, 2, 72);
        }

        .heading1 {
            font-style: italic;
        }

        /* , group selector  */
        h1,
        h2,
        div,
        span,
        #heading,
        .heading {
            text-decoration: underline;
            font-family: 'Franklin Gothic Medium', 'Arial Narrow', Arial,
sans-serif;
        }

        /* * universal */
        * {
```

```
            color: red;
            font-size: 100px;

            padding: 0%;
            margin: 0%;
            box-sizing: border-box;
            font-family: 'Trebuchet MS', 'Lucida Sans Unicode', 'Lucida
Grande', 'Lucida Sans', Arial, sans-serif;
        }
    </style>
```

Typography Properties (12 to 18)

```
<h1 style="
        color: red;
        background-color: bisque;

        text-align: end;
        text-align: right;
        text-align: left;
        text-align: start;
        text-align: justify;
        text-align: center;
        text-transform: uppercase;
        text-transform: lowercase;
        text-transform: capitalize;
        text-decoration: underline;
        text-decoration: overline;
        text-decoration: line-through;
        text-decoration: underline;
        text-shadow: 3px 1px 0px black;
        text-wrap: inherit;

        font-size: 56px;
        font-weight: lighter;
        font-weight: 600;
        font-style: oblique;
        font-style: italic;
        font-family: 'Courier New', Courier, monospace;

        writing-mode: horizontal-tb;
```

```
        writing-mode: sideways-lr;
        writing-mode: sideways-rl;
        writing-mode: vertical-lr;
        writing-mode: horizontal-tb;

        line-height: 100px;
        letter-spacing: 10px;

        ">hello world</h1>
```

Box Properties (4)

```
<h1 style="
        color: red;
        background-color: bisque;

        border: 2px solid blueviolet;
        padding: 50px;
        margin: 100px;

        border-color: brown;
        border-width: 5px;
        border-style: dotted;

        border: 10px solid blue;
        border: 10px dotted blue;
        border: 10px dashed blue;
        border: 10px double blue;

        padding-left: 100px;
        padding-top: 100px;
        padding-right: 100px;
        padding-bottom: 100px;

        padding: 200px 100px 100px 300px;

        padding: 200px 300px;

        padding: 100px 300px 200px;

        margin: 200px;
```

```
        margin: 0px 0px 0px 0px;

        padding: 100px;
        margin: 50px;

        /* box-shadow: 10px 10px 0px 0px rgba(255, 0, 0, 0.272); */

        /* border-radius: 50px; */

        /* border-top-right-radius: 100px;
        border-bottom-left-radius: 100px; */

        border-left: 100px solid red;
        border-top: 100px solid blue;
        border-right: 100px solid red;
        border-bottom: 100px solid blue;
        ">hello world</h1>
```

Dimensional Properties (2 + 4)

Width ⇒ Breath	**Height ⇒ length**
Horizontal ⇒ Row ⇒ left to right	**Vertical ⇒ Column ⇒ top to bottom**
Width ⇒ min-width , max-width **Min ⇒ starting point Max ⇒ ending pointing**	**Height ⇒ min-height, max-height**
%,px ,vh,auto,length,inherit,initial,none	**%,px ,vh,auto,length,inherit,initial,none**

Units Properties (4)

Units ⇒ %,px,em.rem,

```
<div>
        <h1>Units properties</h1>

        <p style="
            /* font-size: 300%; */
            /* font-size: 150px; */
            /* font-size: 10em; */
            /* font-size: 10rem; */
```

```
            /* font-size: 10ex; */
            /* font-size: 1rex; */
            font-size: 10vh
        ">hello world</p>
    </div>
```

https://www.w3schools.com/css/css_units.asp

Background Properties (5)

Color,image,vedio

```
 <div style="
    background-color: blue;
    background-image: url(car.avif);
    background-repeat: no-repeat;
    /* background-size: cover; */
    /* background-size: 100%; */
    background-size: 100% 100%;
    /* background-position: center; */
    /* background-position: top; */
    background-position: bottom;
    background-attachment: fixed;
    height: 100vh;">
        <h1>BAckground Property</h1>
```

Display Properties (3)

```
<div>
        <h1>Display Properties (3)</h1>

        <h1>hello <div style="color: blue; display: inline;">world</div>
        </h1>

        <!-- <h1>hello world</h1> -->

        <div>hiii <b style="color: blueviolet; font-size: 45px; display:
block;">Friends😎😎</b></div>

        <div>
            <p>menus</p>
```

```
            <style>
                ul li {
                    /* display: inline-block; */
                    /* display: inline-flex; */
                    float: left;
                    list-style-type: none;
                    padding: 5px;
                }
            </style>
            <ul>
                <li>home</li>
                <li>about</li>
                <li>service</li>
            </ul>
        </div>
    </div>
```

Flex Properties (1 + 6) ⇒ 1d

```
<style>
        #mainflexbox {
            background-color: antiquewhite;
            border: 2px solid red;
            padding: 10px;
            margin: 10px;
            display: flex;
            /* justify-content: center; */
            justify-content: space-around;
            justify-content: space-between;
            justify-content: space-evenly;
            align-items: center;
            text-align: center;
            flex-direction: row-reverse;
            /* flex-direction: column;
            flex-direction: column-reverse; */
            flex-wrap: wrap;
        }

        .flexitem {
            background-color: aquamarine;
```

```
            width: 200px;
            height: 200px;
            border: 1px solid green;
            padding: 10px;
            margin: 10px;
        }

        .bg1 {
            background-color: blue;
        }

        .bg2 {
            width: 100px;
            height: 150px;
        }

        #mainflexbox div:nth-child(3) {
            background-color: brown;
            width: 150px;
            height: 50px;
            flex-grow: 2;
        }

        #mainflexbox div:nth-child(4) {
            background-color: green;
            width: 150px;
            height: 150px;
            flex-grow: 2;
        }

        #mainflexbox div:nth-child(5) {
            background-color: goldenrod;
            width: 100px;
            height: 100px;
            border-radius: 50%;
        }
    </style>
<div>
        <h1>Flex Properties (1 + 6)</h1>
```

```
        <div id="mainflexbox">
            <div class="flexitem bg1">1</div>
            <div class="flexitem bg2">2</div>
            <div class="flexitem">3</div>
            <div class="flexitem">4</div>
            <div class="flexitem">5</div>
            <!-- <div class="flexitem bg1">1</div>
            <div class="flexitem bg2">2</div>
            <div class="flexitem">3</div>
            <div class="flexitem">4</div>
            <div class="flexitem">5</div>
            <div class="flexitem bg1">1</div>
            <div class="flexitem bg2">2</div>
            <div class="flexitem">3</div>
            <div class="flexitem">4</div>
            <div class="flexitem">5</div> -->
        </div>
    </div>
```

Grid Properties (1 + 4) ⇒ 2d

```
        .gridcontainer {
            background-color: antiquewhite;
            border: 2px solid red;
            padding: 10px;
            margin: 10px;
            display: grid;
            /* grid-template-columns: auto auto auto auto auto auto auto
auto auto auto auto auto; */
            grid-template-columns: 400px 300px 200px;
            grid-template-rows: 200px 300px 100px;
            /* row-gap: 100px;
            column-gap: 100px; */
            gap: 20px;
        }

        .griditem {
            background-color: blueviolet;
            color: white;
            padding: 20px;
```

```
            margin: 10px;
            border: 2px solid red;
            text-align: center;
            /* justify-content: center; */
        }
```

```
<div>
        <h1>Grid Properties (1 + 4) ⇒ 2d</h1>
        <div class="gridcontainer">
            <div class="griditem">1</div>
            <div class="griditem">2</div>
            <div class="griditem">3</div>
            <div class="griditem">4</div>
            <div class="griditem">5</div>
            <div class="griditem">6</div>
            <div class="griditem">7</div>
            <div class="griditem">8</div>
            <div class="griditem">9</div>
            <div class="griditem">10</div>
            <div class="griditem">11</div>
            <div class="griditem">12</div>
            <div class="griditem">12</div>
        </div>
    </div>
```

Position Properties (5) ⇒ 3D

Static (Default)
FIxed (Locked)
Relative (Parent Position)
Absolute (Child position)
Sticky (Flexi or aDJUSIBLE)

```
<div>
        <h1>Position Properties (5) ⇒ 3D</h1>
        <div class="container">
            <div class="items">1</div>
            <div class="items">2</div>
            <div class="items">3</div>
            <div class="items">4


            </div>
```

```
            <div class="items">5</div>
        </div>
    </div>
.container {
            background-color: antiquewhite;
            border: 2px solid red;
            padding: 10px;
            margin: 10px;
            display: flex;
            justify-content: space-around;
            align-items: center;
            position: relative;
        }

        .items {
            background-color: black;
            color: white;
            border: 2px solid red;
            padding: 20px;
            margin: 20px;
            text-align: center;
        }

        .container div:nth-child(1) {
            width: 100px;
            height: 100px;
            position: static;
            left: 0;
            top: 100px;
        }

        .container div:nth-child(2) {
            background-color: rgb(195, 113, 14);
            width: 200px;
            height: 100px;
        }

        .container div:nth-child(3) {
            width: 100px;
            height: 200px;
```

```
            background-color: blue;
        }

        .container div:nth-child(4) {
            width: 105px;
            height: 100px;
            background-color: blueviolet;
            position: absolute;
            top: -100px;
            left: 0;
        }

        .container div:nth-child(5) {
            width: 50px;
            height: 50px;
            border-radius: 50%;
            background-color: greenyellow;
            position: fixed;
            /* top: 0; */
            right: 0;
            bottom: 0;
        }
    </style>
```

Z-index Properties (1) 3D(POSITION)

```
        .container {
            background-color: antiquewhite;
            border: 2px solid red;
            padding: 10px;
            margin: 10px;
            display: flex;
            justify-content: space-around;
            align-items: center;
            position: relative;
        }

        .items {
            background-color: black;
```

```
        color: white;
        border: 2px solid red;
        padding: 20px;
        margin: 20px;
        text-align: center;
    }

    .container div:nth-child(1) {
        width: 100px;
        height: 100px;
        position: static;
        left: 0;
        top: 100px;

    }

    .container div:nth-child(2) {
        background-color: rgb(195, 113, 14);
        width: 200px;
        height: 100px;
    }

    .container div:nth-child(3) {
        width: 100px;
        height: 200px;
        background-color: blue;
    }

    .container div:nth-child(4) {
        width: 105px;
        height: 100px;
        background-color: blueviolet;
        position: absolute;
        top: 100px;
        left: 0;
        z-index: 1;
    }

    .container div:nth-child(5) {
        width: 50px;
```

```
            height: 50px;
            border-radius: 50%;
            background-color: greenyellow;
            position: fixed;
            /* top: 0; */
            left: 0;
            bottom: 0;
            /* z-index: -1; */
            z-index: 2;
            /* z-index: 0.9999; */
        }
    </style>
```

Responsive Design Properties

Break Point Properties

Mobile	**320px to 480px**
Tablet	**481px to 767px**
Laptops	**768px to 1024px**
Desktop	**1024px to 1200px**
Extra large	**1201 px above**

Media Query Properties

@media () and () {}

Flex Properties

Grid Properties

```
        @media (min-width: 320px) and (max-width: 480px) {
            body {
                background-color: rgb(244, 206, 206);
            }

            h1 {
                font-size: 23px;
            }

            .flexitem {
                width: 100%;
            }
```

```
        .gridcontainer {
            display: block;
        }

        .container {
            display: block;
        }

        .items {
            width: inherit !important;
        }

        .container div:nth-child(4) {
            top: -40px;
        }

        .container div:nth-child(5) {
            width: 25px !important;
            height: 25px !important;
            left: 220px !important;
        }
    }

    @media (min-width: 481px) and (max-width: 767px) {
        body {
            background-color: rgb(233, 58, 58);
        }

        h1 {
            font-size: 23px;
        }

        .flexitem {
            width: 100%;
        }

        .gridcontainer {
            display: block;
        }
```

```
        .container {
            display: block;
        }

        .items {
            width: inherit !important;
        }

        .container div:nth-child(4) {
            top: -40px;
        }

        .container div:nth-child(5) {
            width: 25px !important;
            height: 25px !important;
            left: 630px !important;
        }
    }
```

Canvas Properties

```
<div>
    <h1>canvas tag and drawing concepts of css</h1>
    <div style="
        border: 1px solid red;
        padding: 20px;
        margin: 20px;
        width: 100px;
        height: 100px;
    ">square </div>

    <div style="
        border: 1px solid red;
        padding: 20px;
        margin: 20px;
        width: 200px;
        height: 100px;
    ">R.angle </div>
```

```
        <div style="
            border: 1px solid red;
            padding: 20px;
            margin: 20px;
            width: 100px;
            height: 100px;
            border-radius: 50%;
        ">circle </div>

        <div style="
            border: 1px solid red;
            padding: 20px;
            margin: 20px;
            width: 200px;
            height: 100px;
            border-radius: 50%;
        ">circle </div>

        <div style="
            border: 1px solid red;
            padding: 20px;
            margin: 20px;
            width: 100px;
            height: 200px;
            border-radius: 50%;
        ">circle </div>

    </div>
```

Color Properties

https://www.w3schools.com/cssref/css_colors.php
https://htmlcolorcodes.com/
https://cssgradient.io/

60 : 30 : 10
Basic Animation Properties

```
.container div:nth-child(5):hover {
            background-color: blueviolet;
```

```
            /* transition: all .6s ease; */
            /* transition: all .6s ease-in ; */
            /* transition: all .6s ease-out; */
            transition: all .6s ease-in-out;
            /* transform: scale(1.5); */
        }
```

Js Notes

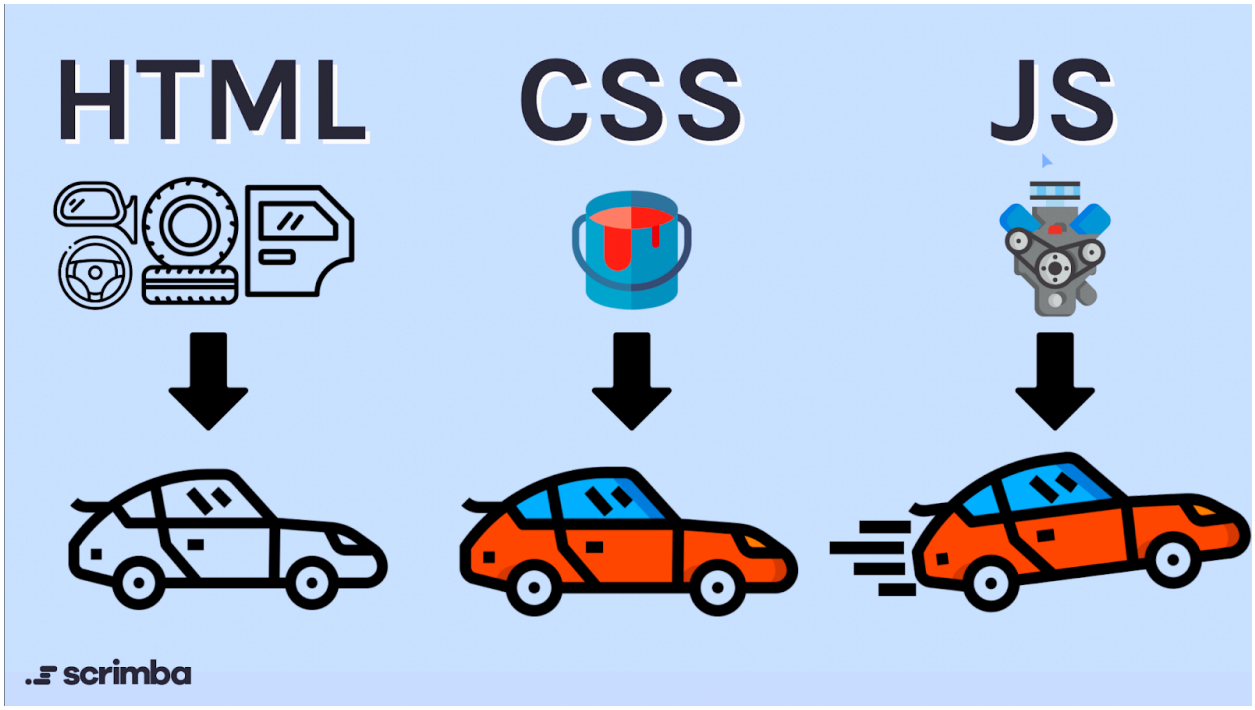
Java Script ⇒ .js ⇒ .ts

It is Programming scripting language

It give interactive ness or Functionalities of Web Pages

Java script is a high level programming scripting language and it's give more interactiveness of the web pages and web application.

We create or develop a static web application using only html,css when will you add your scripting concept of js that time only project will transforming to Dynamic web application



Importing methods 😎

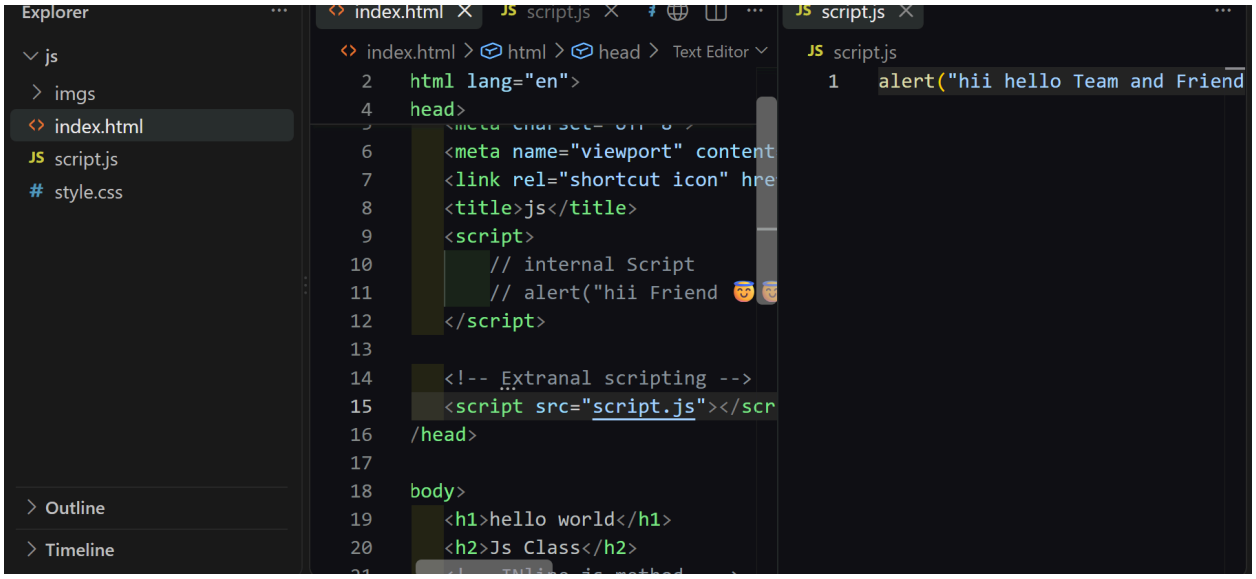
Inline Scripting method

```
<body>
    <h1>hello world</h1>
    <h2>Js Class</h2>
    <!-- INline js method  -->
    <script>
        alert("hii team")
    </script>
</body>
```

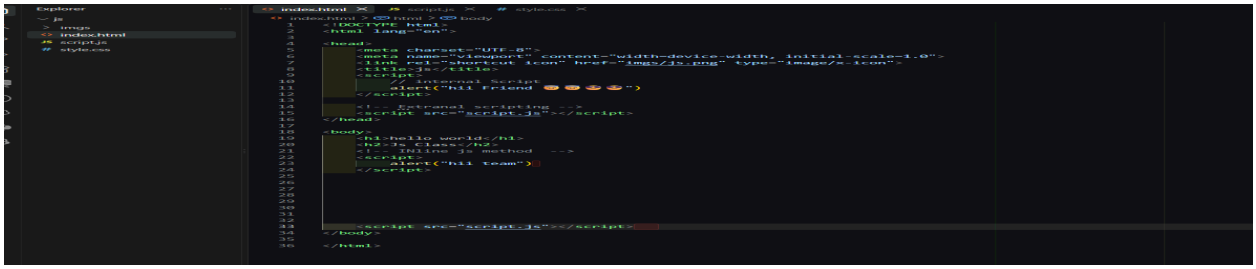
Internal Scripting method

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="shortcut icon" href="imgs/js.png" type="image/x-icon">
    <title>js</title>
    <script>
        // internal Script
        alert("hii Friend 😇😇😎😎")
    </script>
</head>
```

External Scripting method



LExical Scripting method



Javascript Vs Java 👍

Java	Javascript
Core Language	Scripting language
Combination of OOPS & DSA	Optional ⇒ OOPS & DSA
Structure & Syntax or Rules we wont avoid or overcome	Structure & Syntax or Rules we will or can avoid or overcome
Backend Logics	Frontend Logic or validation
Java ⇒ oops dsa ⇒ core concepts	ECMA script ⇒ similar to java ⇒ Js

	Js ⇒ ECMA script 5(ES 5) , ECMA script 6 (ES 6) ⇒ es6 (js)

JS Concepts 👍

ES 5 Topics

Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information

Data Type ⇒ 2 +1

Primitive Data 7	**Non primitive Data 3**
Number **String** **Boolean** **Bigint** **Undefined** **Null** **Symboles**	**Array ⇒ =[], ⇒ = [‘ ‘,””,1,true]** **Object ⇒ ={}, ⇒ ={ key : value }** **Function**

```
<script>
        //  Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
        // Data Type  ⇒ 2 +1
        // Stractured Data type  ==> Primitive Data type (7)
        // Number  ==> [0,1,2,3,4.......n] ==> without using any
symblole  "",''
        let num1 = 100;
        const Total = 2000;
        let myNum = 10.80;
        console.log(num1);
        console.log(Total);
        console.log(myNum);
        // String ==> [a,b,c,d......z] ==> with using any symbole like "
",''
        let myName = " ganapathi";
        let myName1 = 'raja';
        console.log(myName);
        console.log(myName1);
        // Boolean ==> [true, false ==> 0,1]
        // let isValid = true;
        let isValid = false;
        console.log(isValid);
```

```
        // bigINt [ (2n*3) ==> more the 5 digits ]
        let myMobile = 75202132895
        let myValue = 100000000000000000000000
        BigInt(myMobile);
        BigInt(myValue);
        console.log(BigInt(myValue))
        // Undefined ==> [ ? ] ==> may be getting a values
        var a = 1;
        console.log(a);
        // Null ==> [ { } ]  ==> never getting a values
        var a = null;
        // symbole [ @,!,#,$,%];

        // UnStractured Data type  ==> NON - Primitive Data type (3)
    </script>
```

```
<script>

        // UnStractured Data type  ==> NON - Primitive Data type (3)
        // Array  ==> arr or [ '',"",10]
        // Hytrogenius element an dstore continuos=use loaction
        // it has two important key words
        //  index ==>start with 0==>  addresss ==> position ==> refernce
key,
        //  element ==> values ==> datas ==>
        var myArr = [100, 200, 300, "ganapathi", "sunder", "hari", true,
false, 100.200, "%^$", 7502132895, "pc", "hello", '1'];
        console.log(myArr);
        console.log(myArr[0]); // index finding
        console.log(myArr[12]);
        // console.log(myArr[13]);
        // console.log(myArr[8]);
        // console.log(myArr[8, 9, 10]);

        // Object  ==> Stand alone entity  ==> obj or  = {} or ={key :
value}

        var myCar = {
```

```
            name: "spressoo",
            color: "grey",
            price: 500000
        };

        var myCar1 = {
            name: "spressoo",
            color: "grey",
            price: 500000,
            quality: function functionality() {
                console.log(myCar1.name);
                console.log(this.color); // 3rd calling object value ==>
this lkeyword to cll inside object within the function
            }
        };
        console.log(myCar);
        console.log(myCar.name); //1st method ==>  object litral method
        console.log(myCar.color);
        console.log(`${myCar.price}`); //2nd method ==>  object
destructing method

        console.log(myCar1.quality())
        //  console.log(${ myCar.price }); // object destructing method

        // Function  ==> fnction ==> funName () { return } ==>
=()=>{return}
    </script>
```

```
 <script>
        //  Array  ==> = [ ]

        let myDatas = [1, 2, 3, 4, 5, "a", "b", "c", true, false, null,
7502132896,];
        console.log(myDatas);
        console.log(myDatas[0]);
        console.log(myDatas[1]);
        console.log(myDatas[5]);
```

```
        console.log(myDatas.indexOf(true));

        // Object ==> ={ key : value  }

        const car = {
            name: "bmw",
            color: "navy",
            price: 1200000,
            model: "m5"
        }

        console.log(car)
        console.log(car.model)
        console.log(car.name) // . 1st method litral

        console.log(`${car.price}`)
        console.log(`${car.color}`) // `${}` 2nd method object
desctructing

        let waterBottle = {
            name: "jio",
            qty: "5l",
            price: 20,
            capacity: function size() {
                let smallSize = "250ml";
                let sample = this.name
                console.log(smallSize)
                console.log(waterBottle.qty)
                console.log(this.name) // this 3rd method (.this) method
                console.log(this.price) // this measn value assigner
            }
        };
        console.log(waterBottle)
        console.log(waterBottle.capacity())
        // console.log(this.AnimationTimeline) // not adviceable

        // Functions  ==> (){} or =()=>{} ==> or function
        // Reusable code (organized) collection to perform single loci
operations
```

```
        // function funName(){ return } or const funNAme =()=>{return};

        // console.log(100 + 200);



        // function ==> 2 classification
        // with Parameter function
        // parameter ==> arguments ==> referce datas  ==> props ==>
params
        function sumOftwoNumbers(a, b) { // a,b parameter ==> value
carrier
            let c = a + b;
            return c;
        }

        const sumOfNumbers = (value1, value2) => {
            return valuesAdd = value1 + value2;
        }

        console.log("nasika 100,200:  ", sumOftwoNumbers(100, 200));
        console.log("pavidra 1000,2000:  ", sumOftwoNumbers(1000,
2000));
        console.log("surender 1000,2000:  ", sumOftwoNumbers(10, 20));
        console.log("surender 1000,2000:  ", sumOftwoNumbers(10, 1));

        // without  Parameter function

        function Nasika() {
            let a = 100;
            var b = 300;
            const c = a + b;
            return c;
            console.log(c)
        };
        Nasika();
        console.log(Nasika());

        function Pavidra() {
            let x = 200, y = 300;
```

```
            const z = x + y;
            console.log(z);
        };
        Pavidra()

        const surender = () => {
            const a = 1000; const b = 400;
            return c = a + b;
        }
        console.log(surender())


        // functions 4 types
        // Function expression method
        // function FunName () { return }
        function Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        };
        Pavidra()
        // Arrow method ==> HOF ==> Call back ==> recurssive function
==> cohansive function ==>
        // variabledeg funcName =()=>{ return }
        const surender = () => {
            const a = 1000; const b = 400;
            return c = a + b;
        }
        console.log(surender())

        // IIFE
        const myMainfun = (
            function Pavidra() {
                let x = 200, y = 300;
                const z = x + y;
                console.log(z);
            }
        )();
        // Generative Method
```

```
            function* Pavidra() {
            let x = 200, y = 300;
            const z = x + y;
            console.log(z);
        }

    </script>
```

Variable ⇒ Containers (temp storage) ⇒ 3 ⇒ var ,let, const
Variable Scoping & Hoisting ⇒ { } ⇒ 3

```
<script>

        // Data ⇒ Raw Facts ⇒ 0,1 ⇒ collection information
        // Collection of Inforamtaions or detalis
        // Raw facts ==> 0,1 ==> true  false
        // Data type ==> 2
        // Stractured Data type  ==> Primitive Data type (7)
        // UnStractured Data type  ==> NON - Primitive Data type (3)
        // Variable  ⇒ Containers (temp storage ) ⇒ 3 ⇒ var ,let,
const
        // var let const
        // To store temporarly in data in Opertaion basis

        // a = "ganapathi";

        // var a = "ganapathi";
        // var a = "sundar";

        // let a = "Keerthi";
        // let a = 1000;

        // const pi = 3.14
        // const Opi = 22 / 7

        // let pi = 20;

        // const a = 100;
```

```
        // const a = "ganapathi";

        // variable  varName = values

        var obj1 = "hii"
        let obj2 = 'hello'
        const obj3 = 100;

        // Variable  ⇒ {  } == > Scoping => 3
        // global scope  ==> ES5 scope  ==> var let const
        // local  scope ==>  Block level scope ==> functional scope ==>
let const
        // lexical  scope  ==> let const


        var trainer = "ganapathi";
        function batch14() {  // Globle scope
            console.log("hii globel", trainer);
            let trainee = "pavidra";
            console.log(trainee);

            function team1() { // local scope
                console.log(trainer); // lexivcaln
                console.log(trainee);
                const trainee1 = "Mani"
                console.log(trainee1)
            }
            team1()
            console.log(trainee1)

        }

        console.log(trainer);
        batch14();
        // console.log(trainee);


        function sla() {
            var trainer = "ganapathi";
```

```
            function batch14() {
                console.log(trainer);
                let trainee = "pavidra";
                function team1() {
                    console.log(trainer);
                    console.log(trainee);
                    const trainee1 = "Mani"
                    console.log(trainee1)
                }
            }
        }

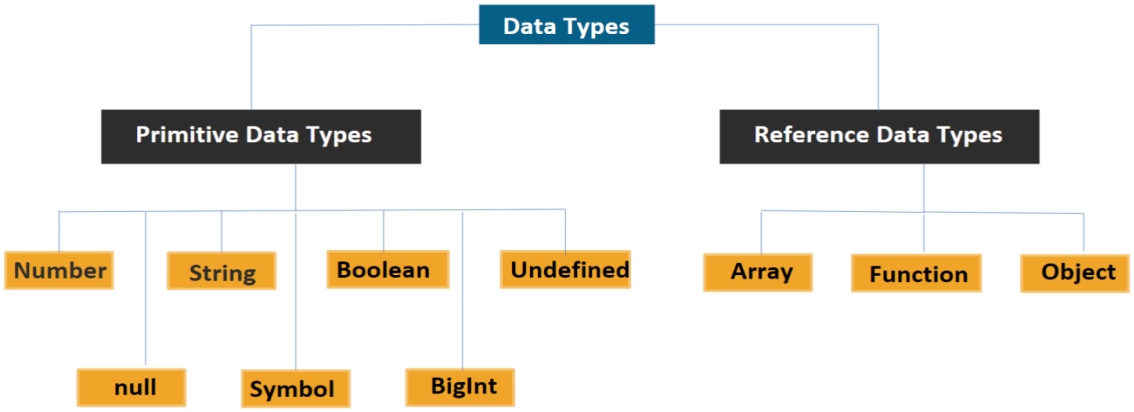
    </script>
```

Var	Let	Const
Es5 version variable	Es6 version variable	Es6 version variable
Void scoping (public)	Block / functional scoping	Block / functional scoping
Global scoping variable	Local scoping variable	Local scoping variable
Read / write operation can perform	Read / write operation can perform	Read operation can perform
Changeable / Mutable	Changeable / Mutable	Non Changeable / imMutable
Scoping means ⇒ { }		
Variable Hoisting ⇒ var Web hoisting ⇒ Local project will migrate globalize project		

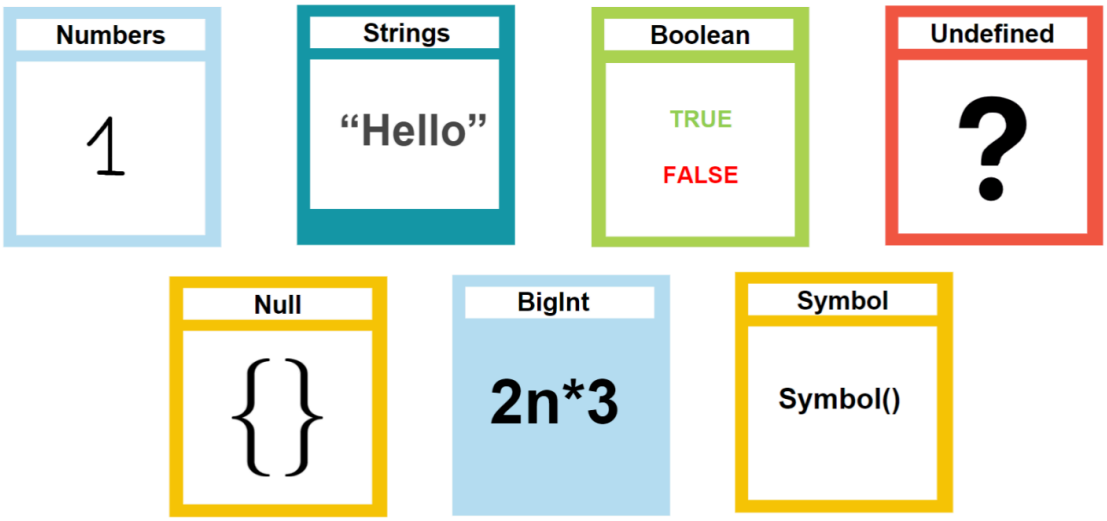
```
// Variable Hoisting
        // const a = "Ganpathi";
        // let a = "Ganpathi";
        // var a = "Ganpathi";
        console.log(a);
        var a = "Ganpathi Raja";
        // let a = "Ganpathi Raja";
        // const a = "Ganpathi Raja";
```

[redacted]

Datatypes in JavaScript



Primitive Data Types



Reference Data Types

Objects: An object is a standalone entity. It consists of properties and methods.

```
Syntax: let obj1 = { }
```

Arrays: An Array is a collection of heterogenous elements which are stored at contiguous memory locations.

```
Syntax: let arr[ ] ;
let arr = new Array( )
```

Functions: A function is a block of organized, reusable code that is used to perform a single, related actions.

```
Syntax: function
name() {
alert("This is a
function");
}
```

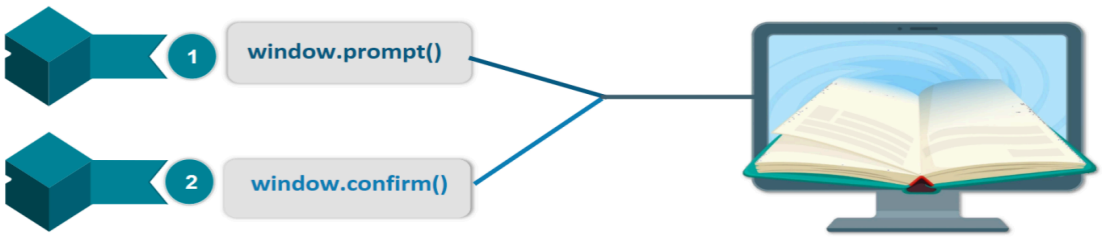
Understanding JavaScript Output Statements

- In JavaScript, there are different ways to display the data.
- Some of the commonly used ways to display output are:

Statement	Description
innerHTML	It writes a data into a certain HTML element
document.write()	It displays a data in the HTML output
window.alert()	It displays the data in the alert box
console.log()	It displays the data in the browser console

Understanding JavaScript Input Statements

- It is important to take input from the user to display the data as per their need.
- Apart from forms, JavaScript provides two statements to accept input from the user as mentioned below:



Js I/O Methods

Input method ⇒ 2

Output Method ⇒ 4

```
<h2>Output methods</h2>

    <button onclick="outputMethod()">click me see my output</button>

    <h2 id="outputmethod">hii output methods</h2>

    <button onclick="outInnerhtmlmethod()">click me InnerHtml</button>



    <div>
        <h2 id="input1"></h2>
        <h2 id="input2"></h2>
        <button onclick="inputMethod1()">click Give input 1</button>
        <button onclick="inputMethod2()">click Give input 2</button>
    </div>
    <script>
        // Inputs 2 /
        // window.prompt();
        // window.prompt("Enter your thougt today ...!")
        function inputMethod1() {
            const userMsg = window.prompt("Enter your thougt today ...!");
            let myThought = document.getElementById("input1");
            myThought.innerHTML = userMsg;
        };
        // window.confirm();
        function inputMethod2() {
            if (window.confirm("Are you sure submit your form..👍👍!")) {
                let myinput2method = "successfull submitted your
form...👍👍👍"
                let myThought = document.getElementById("input2");
                myThought.innerHTML = myinput2method;
            }
            else {
                let myMessages = "Your Application not submitted ..sorry
....❌❌🤦‍♂️🤦‍♂️";
                let myThought = document.getElementById("input2");
                myThought.innerHTML = myMessages;
```

```
        }

    };



    //  Output methods 4
    // console.log(); ==> 4
    const myName = "Ganapathi Raja ";
    console.log(myName);
    console.info(myName);
    console.warn(myName);
    console.error(myName);


    // window.alert() ==> 2
    // window.alert(myName);
    // alert(myName);


    // document.write()
    document.write(myName);
    function outputMethod() {
        let messages = "hii team and Friends...😍😎😎😇😂";
        // document.write(messages);
        document.writeln(messages);
    };
    // innerHTML
    const outInnerhtmlmethod = () => {
        let messages = "hii team and Friends...😍😎😎😇😂";
        let myPlacehtml = document.getElementById("outputmethod");
        myPlacehtml.innerHTML = messages;
    }
</script>
```

Operators ⇒ 4 + 2

Z = x+y ⇒ let z = a+b;

Operands ⇒ x,y,z

Operators ⇒ +.=

OPeration ⇒ addition
Variable ⇒ let ⇒ temp

Operators	Symbols or Operators
Arithmetic Operators (7)	+,-,*,/,%,++,--
Assignment Operators (8)	=,=+,=-=,*,=/,=%,
Comparison Operators	==,===,!=,<,=<,>,>=
Logical Operators (3)	AND &&,OR ||,NOT !
Ternary ⇒ ?	? ⇒ if statements
typeOf ⇒ Operators	typeOf()

```
<script>



        // Arithmetic Operators (7)
        // +,-,*,/,%,++,--
        let a = 10; b = 30;
        const key = 1;
        // const key = 2;
        // const key = 3;
        // const key = 4;
        // const key = 5;
        // const key = 6;
        // const key = 7;
        // const key = 17;
        switch (key) {
            case 1:
                var c = a + b;
                console.log(c);
                break;

            case 2:
                var c = a - b;
                console.log(c);
                break;
```

```
        case 3:
            var c = a * b;
            console.log(c);
            break;
        case 4:
            var c = a / b;
            console.log(c);
            break;
        case 5:
            var c = a % b;
            console.log(c);
            break;
        case 6:
            a++
            var c = a + b;
            console.log(c);
            break;
        case 7:
            a--;
            var c = a + b;
            console.log(c);
            break;
        default:
            console.log("You enter invalid Operation")
            break;
    }


    // let x = 200; y = 400; z = x + y;
    // console.log(z);

    // console.log(10 + 20);

    // Assignment Operators (8)
    // =,=+,=-=,*,=/,=%,

    let x = 200;
    let y = 400;
    // x++;
```

```
x = x + 40;
y = y - 70;
y = y * 2
const z = x + y;
console.log(z);

// Comparison Operators
// ==,===,!=,<,=<,>,>=

let x = 200;
let y = 400;

if (x > y) {
    console.log("X is bigger value", x)
}
else {
    console.log("Y is bigger value", y)
}

let value1 = 100;
const value2 = 100;
const value2 = 1000;
const value2 = "1000";
const value2 = true;
= , == , ===
if (value1 = value2) {  // assigneing the values
if (value1 == value2) { // check the values are same or not
if (value1 === value2) { // vaues and their data type
if (!(value1 === value2)) { // vaues and their data type
    console.log("values same")
}
else {
    console.log("values not same")
}

// Logical Operators(3)
// AND &&, OR ||, NOT!

let value1 = 1000; value2 = 2000; value3 = 400;
```

```
if ((value1 > value2) && (value1 > value3)) {
    console.log("Value 1 is bigger", value1)
}
else if ((value2 > value1) && (value2 > value3)) {
    console.log("Value 2 is bigger", value2)
}
else {
    console.log("Value 3 is bigger", value3)
}

if ((value1 > value2) || (value1 > value3)) {
    console.log("Value 1 is bigger", value1)
}
else if ((value2 > value1) || (value2 > value3)) {
    console.log("Value 2 is bigger", value2)
}
else {
    console.log("Value 3 is bigger", value3)
}



//typeOf operator
let ganapathi = 100;
let myname = "ganapathi";
let isvalidate = true;
let isvalidated = false;
let myvalue =

    function add() {
        return
    }

const hi = () => {
    return
}

let myarry = {
```

```
            hi: 100,
        };

        let myObject = [1, 2, 4, 5];

        console.log(typeof (ganapathi));
        console.log(typeof (myname));
        console.log(typeof (isvalidate));
        console.log(typeof (isvalidated));
        console.log(typeof (myvalue));
        console.log(typeof (funame));
        console.log(typeof (add));
        console.log(typeof (hi));
        console.log(typeof (myarry));

        console.log(typeof (myObject));
        console.log(Array.isArray(myObject));

        //ternary operator ==> ?

        let ganapthi = 12;
        const voting = 18;
        // if (ganapthi >= voting) {
        //     console.log("eligible to vote");
        // }
        // else {
        //     console.log("not eligible to vote ");
        // }

        console.log(ganapthi >= voting ? "eligible to vote" : "not
eligible to vote");

    </script>
```

Statements ⇒ 2

Conditional statements (4)
Looping Statements (5)

https://pythontutor.com/visualize.html

Conditional statements (4)	Looping Statements (5)

if(){ Statements };
 	
if(){ Statements } else{ Statements};
 	
if(){ Statements }
else if(){ Statements }
else{ Statements};
 	
switch(){
Case: “case”
Break
…
 	
Default:
}
	for(){ Statements };
	
	for(of) { Statements };
	
	for(in) { Statements };
	
	While () { Statements };
	
	do{ Statements } while();

```
<script>
        //Conditional statements (4)
        // if(){ Statements };  ==> true (max) ==> success
        // let todaClass = true;
        let todaClass = false;
        if (todaClass) {
            alert("Yes, Definiy classes are there!.....")
        }



        let votingEligiblity = 18;
        // const ganapathi = 17;
        const ganapathi = 26;
        if (ganapathi > votingEligiblity) {
```

```
            alert("Eligible to voting ....!😍😍😍😍");
        }

        // if(){ Statements } ==> max (true, success) else{ Statements};
==> max (false, failure)
        if (todaClass) {
            alert("Yes, Definiy classes are there!.....")
        }
        else {
            alert("No,  classes today!.....")
        }



        if (ganapathi > votingEligiblity) {
            alert("Eligible to voting ....!😍😍😍😍");
        }
        else {
            alert("Not Eligible to voting ....!❌❌🤦‍♂️🤦‍♂️");
        }

        // if(){ Statements }
        // else if(){ Statements }
        // else{ Statements};  ==> nested if statements ==> ladder

        let a = 1000; b = 2000; c = 3000; d=4000;
        // if (a > b) {
        //     alert("A is bigger value..!")
        // }
        // else if (a > c) {
        //     alert("A is bigger value..!")
        // }
        // else if (b > a) {
        //     alert("B is bigger value..!")
        // }
        // else if (b > c) {
        //     alert("B is bigger value..!");
        // }
        // else {
        //     alert("C is bigger value..!")
```

```
// }

if ((a > b) && (a > c)) {
    alert("A is bigger value..!")
}
else if ((b > a) && (b > c)) {
    alert("B is bigger value..!")
}
else {
    alert("C is bigger value..!")
}

// switch(){
// Case: “case”
// Break
// …

// Default:
// }


// let userName = "superAdmin";
// let adminPass = "aa@123";

// if (userName == "superAdminmm") {
//     console.log("login completed...")
//     if (adminPass == "aa@123") {
//         console.log("login completed...");
//         console.log("Valid PAssword...");
//     }
//     else {
//         console.log("in Valid PAssword...");
//     }
// }
// else {
//     console.log("Can't login");
// }

// switch(){
```

```
// Case: “case”
// Break
// …

// Default:
// }

// arithmetic Operation

let a = 100; b = 200;
// let key = "add";
// let key = "sub";
let key = "divhjjhj";
switch (key) {
    case "add":
        let add = a + b;
        console.log("addition: ", add);
        break;

    case "sub":
        let sub = a - b;
        console.log("addition: ", sub);
        break;

    case "multi":
        let multi = a * b;
        console.log("addition: ", multi);
        break;

    case "div":
        let div = a / b;
        console.log("addition: ", div);
        break;
    case "modulo":
        let modulo = a % b;
        console.log("addition: ", modulo);
        break;
    case "Inc":
        a++;
```

```
                let Inc = a + b;
                console.log("addition: ", Inc);
                break;
            case "Dec":
                b--;
                let Dec = a - b;
                console.log("addition: ", Dec);
                break;

            default:
                console.log("You enter a wrong operation")
                break;

            // finally:
        }



    </script>
```

```
<script>
        // Looping Statements (5)
        //init
        //condition checking
        //inc/ dec

        // for(){ Statements };
        // for (let i = 0; i <= 5; i++) {
        //     console.log(i);
        // }
        // let i = 0;
        for (let i = 0; i <= 5; i++) {
        //     console.log(i);
        // }

        // let i = 0 ==> 0<=5 ==> console.log(i); ==> 0++ = 1
        // let i = 1 ==> 1<=5 ==> console.log(i); ==> 1++ = 2

        // let i = 4 ==> 4<=5 ==> console.log(i); ==> 4++ = 5
```

```
        // let i = 5 ==> 5<=5 ==> console.log(i); ==> 5++ =6
        // let i = 6 ==> 6<=5 ==> x


        // for(of) { Statements };

        let myArr = [10000, 2, 3, 4, 5, 6, 7];
        console.log(myArr[0]);
        console.log(myArr[1]);
        console.log(myArr[3]);



        for (index of myArr) {
            console.log(index)
        }

        // for(in) { Statements };

        let myObj = {
            name: "ganapathi",
            mobile: 7502132895,
            gmail: "vgr@gmail.com"
        };
        console.log(myObj);
        console.log(myObj.name);
        console.log(myObj.mobile);
        console.log(myObj.gmail);

        for (keys in myObj) {
            console.log(`my object value ${keys} and values
${myObj[keys]}`)
        }


        // While () { Statements };
        let i = 1;
        while (i < 6) {
            console.log("while loop", i);
```

```
            i++;
        }

        // let i = 1
        // 1<=6;
        // console.log("while loop", i);  ==> 1 ++

        // do{ Statements } while();

        let i = 0;
        do {
            console.log("while loop", i);
            i++;
        } while (i <= 5);


    </script>
```

INbuilt Object Function

Math object (5)

Date Object (5 to 8)

```
<script>
        // INbuilt Object Function
        // Math object (5)
        let num1 = 10.47;
        let num2 = 10.50;
        let num3 = 10.40;
        let num4 = 10.56;
        let num5 = 16;
        let num6 = 2;
        console.log(Math.abs(num1))
        console.log(Math.round(num1))
        console.log(Math.round(num4))
        console.log(Math.ceil(num3))
        console.log(Math.floor(num4))
        console.log(Math.sqrt(num5))
        console.log(Math.sqrt(num6))
        console.log(Math.pow(2, 4))
        console.log(Math.log(10))
```

```
        console.log(Math.max())
        console.log(Math.min())
        // console.log(num1);
        // console.log(num2);
        // console.log(num3);
        // console.log(num4);
        // console.log(num5);
        // console.log(typeof (num1));
        // console.log(typeof (num2));
        // console.log(typeof (num3));
        // console.log(typeof (num4));
        // console.log(typeof (num5));

        // Date Object (5 to 8)

        let myDate = new Date();
        console.log(myDate);
        console.log(myDate.getDate());
        console.log(myDate.getDay());
        console.log(myDate.getMonth());
        console.log(myDate.getFullYear());
        console.log(myDate.getHours());
        console.log(myDate.getMinutes());
        console.log(myDate.getMilliseconds());
        console.log(myDate.getSeconds());
        console.log(myDate.getTime())
        console.log(myDate.toString())
        console.log(myDate.toDateString())

    </script>
```

Operation

String OPerations (5 to 8)

Array Operation (28 ⇒ 15)

```
<script>
        // Operation
        // String OPerations (5 to 8) ==> " ", ' '
        let myName = 'GanapathirajaV';
        let fName = "Ganapathi";
        let mName = "Raja";
```

```
        let lName = "V"
        let fullname = "ganapathirajav";
        let str = "                          gnaapathirajav
";
        let str1 = "ganapathirajav";
        let str2 = 'GANAPATHIARAJAV'
        let str3 = "my Name is Ganpathi i'm mobile app developer";

        console.log(myName);
        console.log(fName);
        console.log(mName);
        console.log(lName);
        console.log(fullname);
        console.log(str);
        console.log(str1);
        console.log(str2);
        console.log(str3);


        console.log(myName.length); // counting the characters

        console.log(fName + " " + mName + lName);
        console.log(fName.concat(mName)); // merge or join
        console.log(fName.concat(mName, lName));
        console.log(lName.concat(fName, mName))

        console.log(fullname.slice(1));
        console.log(fullname.slice(2));
        console.log(fullname.slice(3));
        console.log(fullname.slice(12));

        console.log(fullname.slice(0, 9));
        console.log(fullname.slice(9, 13));

        console.log(fullname.substring(1));
        console.log(fullname.substring(0, 9));

        console.log(fullname.substr(9, 13));

        console.log(str1);
```

```
        console.log(str2);
        console.log(str1.toUpperCase());
        console.log(str2.toLowerCase());

        console.log(str);
        console.log(str.length);
        console.log(str.trim());
        console.log(str.trimEnd());
        console.log(str.trimRight());
        console.log(str.trimStart());
        console.log(str.trimLeft());
        let myfullname = "ganapathi          raja      v";
        console.log(myfullname);
        console.log(myfullname.length);
        console.log(myfullname.trim());

        console.log(str3);
        console.log(str3.replace("mobile", "FullStack"));
        console.log(str3.replace("Mobile", "FullStack"));
        console.log(str3.repeat(5))

        console.log(fullname);
        console.log(fullname.padStart(20));
        console.log(fullname.padEnd(20));

        // Array Operation (28 ⇒ 15 )

 // Array Operation (28 ⇒ 15 )
        const myArray = ["ganapathi", "visal", "vasan", "hari", "akash",
100, 100, 200, 10, 500.60, true, false, '#$%', null, "dog"];
        const myArrayValues1 = [100, 200, 300, 400, 500, 600];
        const myArrayValues2 = [1000, 2000, 3000, 4000, 5000];
        // console.log(myArray);
        // console.log(myArrayValues1);
        // console.log(myArrayValues2);

        // console.log(myArray);
        // console.log(myArray[0]);
        // console.log(myArray[1]);
```

```
        // Common operation
        // push / pop
        console.log(myArrayValues1);
        console.log(myArrayValues1.push(700));
        console.log(myArrayValues1);
        console.log(myArrayValues1.push(800, 900, 1));
        console.log(myArrayValues1);

        console.log(myArrayValues1.pop());
        console.log(myArrayValues1);
        console.log(myArrayValues1.pop())
        console.log(myArrayValues1);
        // unshift / shift
        console.log(myArrayValues2);
        console.log(myArrayValues2.unshift(100, 200, 400));
        console.log(myArrayValues2);

        console.log(myArrayValues2.shift());
        console.log(myArrayValues2);
        console.log(myArrayValues2.shift());
        console.log(myArrayValues2);
        // slice
        console.log(myArray);
        console.log(myArray.slice(1))
        console.log(myArray.slice(2));
        console.log(myArray.slice(2, 5));
        // concat
        console.log(myArrayValues1);
        console.log(myArrayValues2);

        console.log(myArrayValues1 + myArrayValues2);
        console.log(myArrayValues1.concat(myArrayValues2));

// Major Operation
        const myArr = [10, 20, 100, 30, 1000, 1, 2, 5, 0];
        // Map ()
        console.log(myArr);
        function mapArray(x) {
            return x + 10;
        };
```

```
        console.log(myArr.map(mapArray))
        // Sort()
        console.log(myArr);
        console.log("Asc", myArr.sort((a, b) => a - b))
        console.log("Desc", myArr.sort((a, b) => b - a))
        // filter()
        function filterArray(x) {
            return x > 9;
        }
        console.log(myArr.filter(filterArray))
        // find() ==> find the first match
        function findArray(x) {
            return x > 9;
        };
        console.log(myArr.find(findArray));
        // forEach()
        function foreachArray(i, x) {
            return console.log(`find index ${x} and add  values ${i +
100}`);
        }
        console.log(myArr.forEach(foreachArray))
        // reduce()
        function reducerArray(i, x) {
            return i + x;
        };
        const totalvalueofArray = myArr.reduce(reducerArray);
        console.log(totalvalueofArray);
        // include()
        console.log(myArr);
        console.log(myArr.includes(1));
        console.log(myArr.includes(9));
        // indexOf()
        console.log(myArr[1]);
        console.log(myArr.indexOf(1));
        // Array.isArray()
        console.log(typeof (myArr));
        const myArray = () => {
            return hello
        };
        console.log(typeof (myArray));
```

```
        console.log(Array.isArray(myArr));
        console.log(Array.isArray(myArray));
        // reverse
        console.log(myArr.reverse());
        //
        console.log(myArr.findIndex());
        console.log(myArr.length);


        // ... spread ==> coping the array
        // ... rest  ==> getherring remaing values


       // ... spread ==> coping the array
        // ... rest  ==> getherring remaing values

        let myArr1 = [10, 20, 30, 40, 50, 1000];
        let myArr2 = [60, 70, 80, 90, 100];


        console.log(myArr1 + myArr2);

        console.log(myArr1.concat(myArr2));

        let myArr3 = [...myArr1];
        console.log(myArr3.push(2000))
        console.log("My Array 3  : ", myArr3);

        let myArr4 = [...myArr1, ...myArr2]; // spread operator
        console.log(myArr4)


        let [a, b, c, d, ...e] = [1111, 2, 3, 4, 5, 6, 7, 8, 9, 10]; //
rest operator
        console.log(a);
        console.log(b);
        console.log(c);
        console.log(d);
        console.log(e);
    </script>
```

DOM Manipulating(5) & Selecting Elements (5)

Manipulation ⇒ Update
MOdification ⇒ Changes

DOm Manipulating Elements	DOm Selecting Element

Create Elements
Append Elements
Preppend
Remove
textContent
InnerHtml
innerText
Style properties
	document.getElementByTagName();
	document.getElementById();
	document.getElementByclassName();
	document.QuerySelector() //call the first match
	document.QuerySelectorAll()

```
<div>
        <h2>DOM Manipulating & Selecting Elements </h2>

        <h1>hello world</h1>
        <h1 class="hii">hii Team</h1>
        <h1 id="hii">hello team hii friends</h1>
        <h1>Friend & team</h1>
        <h1>hello</h1>
        <style>
            .msg {
                color: blueviolet;
                padding: 20px;
                margin: 20px;
                width: 500px;
                background-color: bisque;
            }
        </style>
        <div id="msg" class="msg"></div>
        <div>
            <div id="idselect"></div>
            <div id="idselect"></div>
            <div id="idselect"></div>
```

```
            <div class="classselect"></div>
            <div class="classselect"></div>
            <div class="classselect"></div>
        </div>
    </div>

    <div>
        <button onclick="nameSelect()">select Tag name </button>
        <button onclick="classSelect()">select Class Name </button>
        <button onclick="idSelector()">select Id Name </button>
        <button onclick="querySelect()">select Query select id </button>
    </div>
    <div>
        <button onclick="createTag()">Create a tag</button>
        <button onclick="myTextmsg()">Click me</button>
    </div>
    <script>
        // DOM Manipulating(5) & Selecting Elements (5)
        function nameSelect() {
            let myText = "hello batch13 guys Good Morning....!!";
            let DisplayText = document.getElementsByTagName("h1");
            DisplayText.textContent = myText
        };

        function classSelect() {
            let myText = "hello batch13 guys Good Morning....!!";
            let DisplayText = document.getElementsByClassName("hii");
            DisplayText.textContent = myText
        }


        const idSelector = () => {
            let myText = "hii friend...👍👌👌👌👌";
            let myIdtag = document.getElementById("hii");
            myIdtag.textContent = myText;
        };


        const createTag = () => {
```

```
            let myTag = document.createElement("h1");
            let myTagp = document.createElement("p");
            let myTagh2 = document.createElement("h2");
            let myTagdiv = document.createElement("div");
            let myTagtext = " Hello Ganapathi ... 😎😎😎😎👌";
            let myTagtext1 = " Hello Ganapathi hellooooo ...
😎😎😎😎👌";
            let myTextdiv = "hii team";

            myTagdiv.style.color = "red";


            myTag.textContent = myTagtext;
            myTagp.innerHTML = myTagtext
            myTagh2.innerText = myTagtext1
            myTagdiv.innerText = myTextdiv



            document.body.append(myTag);
            document.body.append(myTagp);
            document.body.append(myTagh2);
            document.body.append(myTagdiv);

        }

        function myTextmsg() {
            let myText = "hello world hii team....!!!";
            let mySpan = document.createElement("h1");
            mySpan.innerText = myText;

            mySpan.style.color = "green";
            mySpan.style.padding = "20px";
            mySpan.style.margin = "50px";
            mySpan.style.fontSize = "36px";
            mySpan.style.marginTop = "50px";
            mySpan.style.backgroundColor = "black";
            mySpan.style.width = "400px";
```

```
            document.body.append(mySpan);
        }


        let myText = "hello world hii team....!!!";
        let mySpan = document.createElement("h1");
        mySpan.innerText = myText;

        mySpan.style.color = "green";
        mySpan.style.padding = "20px";
        mySpan.style.margin = "50px";
        mySpan.style.fontSize = "36px";
        mySpan.style.marginTop = "50px";
        mySpan.style.backgroundColor = "black";
        mySpan.style.width = "400px";

        document.body.append(mySpan);


        function msg() {
            let myMsg = "good morning ";
            let myTag = document.getElementById("msg");
            myMsg.textContent = myMsg;
        }

        let myMsg = "good morning ";
        let myTag = document.getElementById("msg");
        myTag.textContent = myMsg



        function querySelect() {
            let myMsg = "Hiii visal and Surender.....!!!"
            let myIdselect = document.querySelector("#idselect");
            myIdselect.textContent = myMsg;

        }

    </script>
```

Validation of Static Web Pages

ES 6 Topics

ES 5	Es 6
Variable ⇒ var	Let ,const
Data and Data type ⇒ Primitive data ⇒ number ,string boolean,undegine ,null	BigInt,symbole
Non primitive data type ⇒ array ,object, function Array ⇒ oper Object ⇒ literal Function ⇒ FE	 … spread / rest operator `${}`, this Arrow method ⇒ =()=>{} ⇒ call back
Operator ⇒ 5	Ternary (?)
Looping ⇒ for , while	for(of), for(in)
	async/defer/ throttle
	‘useStrict’,
	Template string ` `
	Asyn / await / try catch ⇒ exceptional handling
	Scheduling ⇒
	Promise Methods ⇒
	Fetching
	Model / module ⇒ import / export
	Default js Programms

Event Handlers
Event handler
Event bubbling
Window Object

Window Object (Contd.)

Some of the commonly used window object properties is as shown below:

Method	Description
innerHeight	Return inner height of the window including scrollbars
innerWidth	Return inner width of the window including scrollbars
closed	Returns a Boolean true if a window is closed
history	Returns the history object for the window
location	Returns the location object for the window
navigator	Returns the navigator object for window
screen	Returns the screen object for window

Window Object (Contd.)

Some of the commonly used window object methods are as shown below:

Method	Description
alert()	Displays an alert box with message and ok button
close()	Closes the current window
confirm()	Displays a dialogue box with a message and ok button
open()	Opens a new browser window
stop()	Stops the window from loading
print()	Prints the current window
prompt()	Displays a dialogue box for getting input from a user

Different Event Handlers

Some of the commonly used event handlers are as shown below:

Event Handler	Description
onclick	when a user clicks the mouse button on a certain HTML element
ondblclick	when a user double clicks the mouse button on a certain HTML element
onmouseover	when a user moves the cursor over a certain element
onmouseout	when a user leaves the cursor from an element
onmousedown	when a user presses a mouse button over an element
onmouseup	when a user releases a mouse button over an element
onmousemove	when a user moves the mouse
onKeyPress	when a user press an alphabetic, numeric or punctuation key

Different Event Handlers (contd.)

Some of the commonly used event handlers are as shown below:

Event handler	Description
onkeydown and onkeyup	when a user has pressed down or released the key (even if that key does not produce a character value)
onfocus	when a user focuses on certain element
onsubmit	when a user submits the form
onchange	when a user modifies the value of a form element
onload	when the browser finished the loading of a page
onunload	when the browser unloads the page

Event Bubbling

- In JavaScript, event bubbling is a method of event propagation in the HTML DOM API.
- It works on the principle that whenever an event happens on an element, it first runs the handlers on it then on its parent, up to the containing elements in the hierarchy.
- Suppose element “X” consists of element “Y”, and element “X” is clicked.
- Then the click event will trigger element “X”, and it will bubble up and trigger it for element “Y” as well.

```
addEventListener(EventType, Action, userCapture)
```

- **`EventType`:** It is a type of event which will trigger the action.
- **`Action`:** It is a task to be performed when the event is triggered.
- **`userCapture`:** It is a Boolean value that indicates the phase of an event. By default, its value is set to false, which represents it is in the bubbling phase.

Event	Description
`onclick`	Fires when an element is clicked
`ondblclick`	Fires when an element is double-clicked
`onmousedown`	Mouse button pressed
`onmouseup`	Mouse button released
`onmousemove`	Mouse pointer moves over an element
`onmouseover`	Mouse enters an element
`onmouseout`	Mouse leaves an element
`oncontextmenu`	Right-click on an element

Event	Description
`onsubmit`	Form submitted
`onreset`	Form reset
`onfocus`	Input field gets focus
`onblur`	Input field loses focus
`onchange`	Value of input, select, or textarea changes
`oninput`	Fires when user types (live input)
`onselect`	Text is selected inside input/textarea

Event	Description
`onload`	Page or element fully loaded
`onunload`	Page is unloaded (refresh/close)
`onresize`	Browser window resized
`onscroll`	Page or element scrolled
`onbeforeunload`	Before page is unloaded (confirmation dialogs)

Menu Toggle Concept Keywords

1. **Element Selection**
    - `document.getElementById()`
    - `document.querySelector()`
2. **Event Handling**
    - `onclick`
    - `addEventListener("click", …)`
3. **Class / Style Manipulation**
    - `element.classList.toggle("className")`
    - `element.classList.add("className")`
    - `element.classList.remove("className")`
    - `element.style.display = "block / none"`
4. **Toggle Logic**
    - `if / else` conditions
    - `menu.classList.contains("show")`
5. **UI Keywords**

- **Hamburger menu (☰)** → open menu
- **Close button (✖)** → close menu
- `hidden / visible`
- `slide-in / slide-out` (for animation concepts)

6. **Responsive Design (CSS + JS combo)**
   - `@media` queries
   - `position: fixed / absolute`
   - `transition` / `transform` (animations for smooth toggle)

Advanced Methods of Js 👍 (ES6)

ES ⇒ ECMA script

⇒ . The full form of ECMA is the European Computer Manufacturers Association

⇒ Js ⇒ java core ⇒ OOPs , script ⇒ js ⇒ before version of Js ⇒ Ecma script

ES 5 ⇒ Old version of js	ES 6 ⇒ new Version of JS
Variables ⇒ Var	Variable ⇒ let,const
Data ⇒ Type ⇒ Primitive / non Primitive	Semi / Reference Data type
Primitive Data type ⇒ string,number,boolean,undefine,null	Bigint, symbole
Non Primitive ⇒ object,array, function ⇒ FE ⇒ function funName(){return }	Arrow method ⇒ call back method Const / let/ funName =()=>{return}
Object ⇒ (.) literal method	Object Destructuring ${object key} .this key word
Array ⇒ common operation (12)	(...) spread / rest Operators
Operators ⇒ 4 (ari,assign,compare,logic)	Ternary (?) ⇒ avoid ⇒ if()else{}
Looping ⇒ for ,while ,do while	for(of),for(in)

'useStrict' ⇒ mandatory use any one fun

```
'use strict' → Strict Mode
'use strict' enables Strict Mode in JavaScript.
Can be used at the beginning of a JavaScript file or inside a function.
'use strict' does not mean "mandatory to use any one function."
```

Template string ⇒ `` ⇒ string interpolation ⇒ back tick

```
It allows us to easily insert variables or expressions into a string.
This is called string interpolation.
Template strings can also create multiple lines easily.
```

async/ defer ⇒ must loading beforely or top

```
<!-- Extranal scripting -->
    <script src="script.js" async></script>
    <script src="script.js" defer></script>
```

```
async vs defer in <script>
These attributes are used when loading external JavaScript files.
How async works
```

```
The JavaScript file is downloaded while HTML is being parsed.
But once JavaScript finishes downloading, it executes immediately.
Use async when the JavaScript file is independent of the HTML or other
scripts.
defer
defer also downloads JavaScript while HTML is being parsed.
But JavaScript execution waits until the HTML parsing is completed.
```

Scheduling concepts ⇒ Timing scenarios ⇒ setTimeOut(), setTimeIntecal()

```
setTimeout() setTimeout(function, delay);
setTimeout() executes a function only once after a specified delay.
Why?
setTimeout() does not block JavaScript execution.
The browser schedules the function and continues executing the remaining
code.

setInterval() setInterval(function, interval);
setInterval() executes a function repeatedly after every specified
interval.
<script>
        // scheduling Funtion
        // setTimeOut
        // setIntravel

        alert("hii")

        setTimeout(() => { alert("hii teams ") }, 4000)

        setInterval(() => { alert("hii hello team and frineds...! ") },
8000)

        // js time seconds convert
        // 1000 millsecond
        // 5 * 1000 // seconds
        // 5 * 60 * 1000 // minu
    </script>
```

Async / await / try /catch ⇒exception handling

Promise Methods ⇒ resolve,reject,pending

```
<script>
        // let voteAge = 18;
        let voteAge = 17;
        if (voteAge >= 18) {
            console.log("Eligible to vote....!!");
        }
        else {
            console.log("NOt Eligible to Vote...!!!")
        }

        const myVote = new Promise((resolve, reject) => {
            // let voteAge = 17;
            // let voteAge = 27;
            // let voteAge = "18";
            if (voteAge >= 18) {
                resolve("Eligible to vote....!!");
            }
            else {
                reject("NOt Eligible to Vote...!!!")
            }
        });
        console.log(myVote);
    </script>
```

Fetch ⇒ Getting the data from out source

```
fetch() → Getting data from an external source
fetch() is a JavaScript function used to request data from an external
source, usually an API.
 <div id="usersdata">

    </div>

    <script>
        // Fetching
        // fetch("url")
        // .then((res)=>{format})
        // .then(((data) =>{data}))
        // .catch((err)=>{err})
```

```
        fetch("https://jsonplaceholder.typicode.com/users")
            .then((hi) => hi.json())
            .then((data) => { console.log(data) })
            .catch((err) => { console.log(err) })



        // fetch("https://jsonplaceholder.typicode.com/users")
        //     .then((res) => res.json())
        //     .then((data) => {
        //         console.log(data)

        //         let myData = ''

        //         myData += data.map((u) => {
        //             `<p ${u.id}> ${u.name} </p>`
        //             document.getElementById("usersdata").innerHTML =
data
        //         })

        //     })
        //     .catch((err) => { console.log(err) })
    </script>
```

Model / Module ⇒ import / export

```
Model / Module → import / export
A module is a JavaScript file containing code that can be shared with
other JavaScript files.
import
import allows us to use an exported value from another module.
Default Export
We can export one main value as the default export.
// es 6
import decValues from './deccount'
import addCount from './addcount'
```

```
// es 5
const addValue = require('./addcount');
const decCount = require("./deccount");


function overall() {
    let mycount = document.getElementById("count");
    let Count = 0;

    function updateCOunt() {
        mycount.textContent = Count;
    }
}

export default overall;
```

Default Programmings

```
    <div id="usersdata">

    </div>
    <div id="usersdata1">

    </div>
    <script>
        // Es 6 Concepts
        //  aritnemeticc

        // let a = 100, b = 200;
        // function add() {
        //     const c = a + b;
        //     // return c;
        //     console.log(c);
        // }
        // add()
```

```
        // function add() {
        //     const c = a + b;
        //     // return c;
        //     console.log(c);
        // }
        // add()
        // // `useStrict`
        // function sub() {
        //     const c = a - b;
        //     // return c;
        //     console.log(c);
        // }
        // sub()
        // // `useStrict`
        // function multi() {
        //     const c = a * b;
        //     // return c;
        //     console.log(c);
        // }
        // multi()

        // const myName = {
        //     name: "ganpathi",
        //     gmail: "ganapathi@gmail.com",
        //     mobile: 7502132895
        // };
        // console.log(myName);


        // console.log("My name is ", myName.name, " and my mobile number
is ", myName.mobile);
        // console.log("My name is ",
        //     myName.name, " and my mobile number is ",
        //     myName.mobile);

        // console.log(`My name is ", ${myName.name} ,
        //  " and my mobile number is ",
        //   ${myName.mobile}`) // template string  or string
interponational
```

```
        // fetching
        fetch("https://jsonplaceholder.typicode.com/users")
            // fetch("./data.json")
            .then((res) => res.json())
            .then((data) => {
                console.log(data)
                let useData = '';
                data.map((u) => {
                    useData += `
                        <p ${u.id}> ${u.name} </p>
                    `
                    document.getElementById("usersdata").innerHTML =
useData;
                })
            })
            .catch((error) => { console.log(error) })

        // fetch("https://jsonplaceholder.typicode.com/todos")
        //     .then((res) => res.json())
        //     .then((data) => {
        //         console.log(data)
        //         let useData1 = '';
        //         data.map((u) => {
        //             useData1 += `
        //                 <h1 ${u.id}> ${u.title} </h1>
        //             `
        //             document.getElementById("usersdata1").innerHTML =
useData1;
        //         })
        //     })
        //     .catch((error) => { console.log(error) })


        // scheduling concepts
        // setTimeout()
        // setIntravel()
        console.log('hii team.....!!')
        setTimeout(() => { console.log('hii team.....!!') }, 5000);
```

```
        setInterval(() => { console.log('hii team......join ur
session..!!') }, 8000);

        // Promise method
        // let myAge = 19;
        // if (myAge >= 18) {
        //     console.log("Eligible");
        // }
        // else {
        //     console.log("Not Eligible");
        // }
        // let myVoting = new Promise((resolve, reject) => {
        //     let myAge = 19;
        //     if (myAge >= 18) {
        //         // console.log("Eligible");
        //         resolve("Eligible");
        //     }
        //     else {
        //         // console.log("Not Eligible");
        //         reject("Not Eligible");
        //     }
        // });
        // console.log(myVoting);

        // await ==>
        // wait  ==>
        // sync ==>
        // async ==>

        // otp
        let otpGen = () => {
            const otpNumber = 1234;
            // const otpNumber = false;
            return new Promise((resolve, reject) => {
                if (otpNumber === 1234) {
                    resolve("Otp sended successfully ... valid for next 5
second ", otpNumber)
                }
                else {
                    reject("Otp not send...!");
```

```
        }
    });
}

// otpGen();
// console.log(otpGen());

let otpRecive = () => {
    setTimeout(() => {
        if (otpGen()) {
            console.log("otp reviced..", otpGen())
        }
        else {
            console.log("otp not recived")
        }
    }, 5000);
};

// otpRecive();
// console.log(otpRecive());

async function otpVerification() {
    try {
        // await otpRecive();
        let myOtpVerify = await otpRecive();
        console.log(myOtpVerify);
    }
    catch (err) {
        console.error(err);
    }
    // finally {
    //     console.log("All process done ...!!")
    // }
};
console.log(otpGen())
console.log(otpRecive())
console.log(otpVerification())


// localStorage.setItem();
```

```
        // localStorage.getItem();
        // localStorage.removeItem();
        // localStorage.clear

        // sessionStorage.setItem();
        // sessionStorage.getItem();
        // sessionStorage.removeItem()

        // document.cookie()

        let myDataFetcing = async () => {
            try {
                await setTimeout(() => {
                    fetch("https://jsonplaceholder.typicode.com/users")
                        // fetch("./data.json")
                        .then((res) => res.json())
                        .then((data) => {
                            console.log(data)
                            let useData = '';
                            data.map((u) => {
                                useData += `
                        <p ${u.id}> ${u.name} </p>
                    `

document.getElementById("usersdata").innerHTML = useData;
                            })
                        })
                        .catch((error) => { console.log(error) })
                }, 8000)
            }
            catch (err) {
                console.log("feching is not occur")
            }
        };

        console.log(myDataFetcing());



    </script>
```

```
    <!-- asyn / defer / throttal  ==> enhence the code or project
productivity ==> loading   -->
    <!--
    <script src="counterapp.js" async></script>
    <script src="counterapp.js" defer></script> -->
```

JavaScript Storage

- localStorage
- sessionStorage
- Cookies

localStorage

✅ Stores data in browser permanently
✅ No expiration time
✅ Data remains after browser close
✅ Storage limit ≈ 5MB
✅ Stores only **strings**
❌ Not sent to server automatically

```
localStorage.setItem("key", "value");
localStorage.getItem("key");
localStorage.removeItem("key");
localStorage.clear();

localStorage.setItem("user", JSON.stringify(obj));
JSON.parse(localStorage.getItem("user"));


// Store Data
localStorage.setItem("name", "Ganapathi");

// Get Data
let data = localStorage.getItem("name");
console.log(data);

// Remove Specific Item
localStorage.removeItem("name");
```

```
// Clear All
localStorage.clear();



let user = {
  name: "Ganapathi",
  age: 25
};

localStorage.setItem("user", JSON.stringify(user));

let storedUser = JSON.parse(localStorage.getItem("user"));
console.log(storedUser.name);
```

sessionStorage

✅ Same API as localStorage
✅ Data removed when tab/browser closes
✅ Used for temporary data
✅ Storage limit ≈ 5MB
❌ Not sent to server

```
sessionStorage.setItem("email", "test@gmail.com");

let email = sessionStorage.getItem("email");
console.log(email);

sessionStorage.removeItem("email");
```

Cookies

✅ Small data storage (~4KB)
✅ Can set expiry time
✅ Automatically sent to server with every request
✅ Used for authentication/session

```
document.cookie = "username=Ganapathi; expires=Fri, 31 Dec 2026 12:00:00 UTC; path=/";


console.log(document.cookie);
```

document.cookie = "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC;";

Feature	localStorage	sessionStorage	Cookies
Expiry	No	Tab Close	Custom
Size	~5MB	~5MB	~4KB
Sent to Server	❌	❌	✅
Use Case	Settings	Temporary Data	Login/Auth

```
// Common JavaScript Basic Programs (Default Interview Programs)
// 1) Hello World
console.log("Hello World");
// 2) Add Two Numbers
let a = 10;
let b = 20;
let sum = a + b;
console.log("Sum is:", sum);
// 3) Swap Two Numbers (Using Third Variable)
let a = 5;
let b = 10;
let temp = a;
a = b;
b = temp;
console.log("a:", a, "b:", b);
// 4) Check Even or Odd
let num = 7;
if(num % 2 === 0){
 console.log("Even");
}else{
 console.log("Odd");
}
// 5) Positive / Negative / Zero
let num = -5;
if(num > 0){
 console.log("Positive");
```

```
}else if(num < 0){
 console.log("Negative");
}else{
 console.log("Zero");
}
// 6) Largest of Two Numbers
let a = 20;
let b = 15;
if(a > b){
 console.log("A is largest");
}else{
 console.log("B is largest");
}
// 7) Largest of Three Numbers
let a = 10, b = 25, c = 15;
if(a > b && a > c){
 console.log("A is largest");
}else if(b > c){
 console.log("B is largest");
}else{
 console.log("C is largest");
}
// 8) Reverse a Number
let num = 1234;
let reverse = 0;
while(num > 0){
 let digit = num % 10;
 reverse = reverse * 10 + digit;
 num = Math.floor(num / 10);
}
console.log("Reverse:", reverse);
// 9) Reverse a String
let str = "javascript";
let reversed = str.split("").reverse().join("");
console.log(reversed);
// 10) Palindrome Number
let num = 121;
let original = num;
let reverse = 0;
while(num > 0){
```

```
 let digit = num % 10;
 reverse = reverse * 10 + digit;
 num = Math.floor(num / 10);
}
if(original === reverse){
 console.log("Palindrome");
}else{
 console.log("Not Palindrome");
}
// 11) Factorial
let num = 5;
let fact = 1;
for(let i = 1; i <= num; i++){
 fact *= i;
}
console.log("Factorial:", fact);
// 12) Fibonacci Series
let n = 10;
let a = 0, b = 1;
console.log(a);
console.log(b);
for(let i = 2; i < n; i++){
 let next = a + b;
 console.log(next);
 a = b;
 b = next
}


// 13) Prime Number Check
let num = 7;
let isPrime = true;
if(num <= 1){
 isPrime = false;
}
for(let i = 2; i < num; i++){
 if(num % i === 0){
 isPrime = false;
 break;
 }
```

```
}
if(isPrime){
 console.log("Prime");
}else{
 console.log("Not Prime");
}
// 14) Sum of Array
let arr = [10, 20, 30, 40];
let sum = 0;
for(let i = 0; i < arr.length; i++){
 sum += arr[i];
}
console.log("Sum:", sum);
// 15) Largest in Array
let arr = [5, 10, 25, 3];
let max = arr[0];
for(let i = 1; i < arr.length; i++){
 if(arr[i] > max){
 max = arr[i];
 }
}
console.log("Largest:", max);



// JavaScript Pattern, String, and Array Programs
// ==============================
// PATTERN PROGRAMS
// ==============================
// 1) Right Triangle Star Pattern
*
**
***
****
*****
Code:
let n = 5;
for(let i = 1; i <= n; i++){
 let pattern = "";
 for(let j = 1; j <= i; j++){
```

```
 pattern += "*";
 }
 console.log(pattern);
}
// 2) Reverse Right Triangle
*****
****
***
**
*
Code:
let n = 5;
for(let i = n; i >= 1; i--){
 let pattern = "";
 for(let j = 1; j <= i; j++){
 pattern += "*";
 }
 console.log(pattern);
}
// 3) Number Pattern
1
12
123
1234
12345
Code:
let n = 5;
for(let i = 1; i <= n; i++){
 let pattern = "";
 for(let j = 1; j <= i; j++){
 pattern += j;
 }
 console.log(pattern);
}
// 4) Pyramid Pattern
 *
 ***
 *****
 *******
*********
```

```
Code:
let n = 5;
for(let i = 1; i <= n; i++){
 let pattern = "";
 for(let space = 1; space <= n - i; space++){
 pattern += " ";
 }
 for(let star = 1; star <= (2*i - 1); star++){
 pattern += "*";
 }
 console.log(pattern
}

// ==============================
// STRING PROGRAMS
// ==============================
// 1) Reverse String
let str = "javascript";
let reversed = "";
for(let i = str.length - 1; i >= 0; i--){
 reversed += str[i];
}
console.log(reversed);
// 2) Palindrome Check
let str = "madam";
let reversed = str.split("").reverse().join("");
if(str === reversed){
 console.log("Palindrome");
}else{
 console.log("Not Palindrome");
}
// 3) Count Vowels
let str = "javascript";
let count = 0;
for(let i = 0; i < str.length; i++){
 let ch = str[i].toLowerCase();
 if(ch === 'a' || ch === 'e' || ch === 'i' || ch === 'o' || ch === 'u'){
 count++;
 }
}
```

```
console.log("Vowel count:", count);
// 4) Character Frequency
let str = "hello";
let obj = {};
for(let char of str){
 if(obj[char]){
 obj[char]++;
 }else{
 obj[char] = 1;
 }
}
console.log(obj);
// 5) Remove Duplicates from String
let str = "programming";
let result = "";
for(let char of str){
 if(!result.includes(char)){
 result += char;
 }
}
console.log(result);




// ==============================
// ARRAY PROGRAMS
// ==============================
// 1) Find Largest Element
let arr = [10, 45, 22, 98, 5];
let max = arr[0];
for(let i = 1; i < arr.length; i++){
 if(arr[i] > max){
 max = arr[i];
 }
}
console.log("Largest:", max);
// 2) Second Largest Element
let arr = [10, 45, 22, 98, 5];
```

```
let first = -Infinity;
let second = -Infinity;
for(let num of arr){
 if(num > first){
 second = first;
 first = num;
 } else if(num > second && num !== first){
 second = num;
 }
}
console.log("Second Largest:", second);
// 3) Remove Duplicates from Array
let arr = [1,2,3,2,4,1,5];
let unique = [];
for(let num of arr){
 if(!unique.includes(num)){
 unique.push(num);
 }
}
console.log(unique);
// 4) Reverse Array
let arr = [1,2,3,4,5];
let reversed = [];
for(let i = arr.length - 1; i >= 0; i--){
 reversed.push(arr[i]);
}
console.log(reversed);
// 5) Sum of Array
let arr = [10,20,30,40];
let sum = 0;
for(let num of arr){
 sum += num;
}
console.log("Sum:", sum)


for(let i =0 ;i<5; i++){

    }
```

```
    initi ==> declaration ==> definition ==>  starting ==> excuting value
    check ==> condition check ==> lenth of condition ==> operation
    inc/Dec ==> itration ==> count itration
```

Form validation -01

```
const myformData = document.getElementById("myform");

myformData.addEventListener("submit", (userData) => {
    userData.preventDefault();

    let userName = document.getElementById("uname").value;
    let userEmail = document.getElementById("uemail").value;
    let userMobile = document.getElementById("umobile").value;
    let userReason = document.getElementById("ureason").value;
    let userFeedback = document.getElementById("ufeedback").value;


    if (!userName || !userEmail || !userMobile || !userReason ||
!userFeedback) {
        alert("Please fill the data");
    }
    else {
        const userinfo = {
            userName,
            userEmail,
            userMobile,
            userReason,
            userFeedback
        }
        console.log(userinfo)
        alert("Data saved in console or db ...!!!")
        myformData.reset();
    }
```

```
const myformData = document.getElementById("myform");

myformData.addEventListener("submit", (userData) => {
    userData.preventDefault();

    let userName = document.getElementById("uname").value;
    let userEmail = document.getElementById("uemail").value;
    let userMobile = document.getElementById("umobile").value;
    let userReason = document.getElementById("ureason").value;
    let userFeedback = document.getElementById("ufeedback").value;

    let myErr = document.getElementById("error");



    if (!userName || !userEmail || !userMobile || !userReason ||
!userFeedback) {
        // alert("Please fill the data");
        myErr.textContent = "Please fill the data";
    }
    else {
        const userinfo = {
            userName,
            userEmail,
            userMobile,
            userReason,
            userFeedback
        }
        console.log(userinfo)
        localStorage.setItem("userdata", userinfo)
        alert("Data saved in console or db ...!!!")
        myformData.reset();
    }
```

```
const myformData = document.getElementById("myform");

myformData.addEventListener("submit", (userData) => {
    userData.preventDefault();

    let userName = document.getElementById("uname").value.trim();
    let userEmail = document.getElementById("uemail").value.trim();
    let userMobile = document.getElementById("umobile").value.trim();
    let userReason = document.getElementById("ureason").value.trim();
    let userFeedback = document.getElementById("ufeedback").value.trim();

    let myErrname = document.getElementById("errorName");
    let myErremail = document.getElementById("errorEmail");
    let myErrmobile = document.getElementById("errorMobile");
    let myErrreason = document.getElementById("errorReason");
    let myErrfeedback = document.getElementById("errorFeedback");

    let isvalid = true;

    if (!userName) {
        myErrname.textContent = "Please fill the Name"
        isvalid = false
    }
    else if (userName.length < 3) {
        myErrname.textContent = "Please fill the correct  Name data";
        isvalid = false;
    }
    else {
        myErrname.textContent = ""
        isvalid = true;
    }

    const myEmailPattern =
/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/

    if (!userEmail) {
        myErremail.textContent = "Please fill the email data";
```

```
        isvalid = false;
    }
    else if (!myEmailPattern.test(userEmail)) {
        myErremail.textContent = "Please fill the correct email data";
        isvalid = false;
    }
    else {
        myErremail.textContent = ""
        isvalid = true;
    }

    const mobilePattern = /[6-9]\d{9}$/;

    if (!userMobile) {
        myErrmobile.textContent = "please fill the mobile data";
        isvalid = false
    }
    else if (!mobilePattern.test(userMobile)) {
        myErrmobile.textContent = "plase fill the data correct data
mobile";
        isvalid = false;
    }
    else {
        myErrmobile.textContent = "";
        isvalid = true;
    }

    if (!userReason) {
        myErrreason.textContent = "please fill the reason"
        isvalid = false;
    }


    if (!userFeedback) {
        myErrfeedback.textContent = "olase fill the feedback data"
        isvalid = false;
    }
    else if (userFeedback.length < 5) {
        myErrfeedback.textContent = "please fill the below 10 charcters"
        isvalid = false;
```

```
    }
    else {
        myErrfeedback.textContent = ""
        isvalid = true
    }


    if (isvalid) {
        const userinfo = {
            userName,
            userEmail,
            userMobile,
            userReason,
            userFeedback
        }
        console.log(userinfo)
        localStorage.setItem("userdata", userinfo)
        alert("Data saved in console or db ...!!!")
        myformData.reset();
    }
```

Reactjs Notes

React Js

Nodejs ⇒

> **Run JavaScript Everywhere**
> Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

React Js 😇😇

⇒ Nodejs Basic

Run Time ENV
Platform Independent or cross-platform
Cross Browser Compartability
Open Source
Layering Concepts
Js Code Runner anywhere

⇒ ReactJS 👍

⇒ Facebook is First React application

⇒ SPA ⇒ Single Page Application
⇒ MPA ⇒ Multi page APplication

SPA ⇒ Single Page Application	**MPA ⇒ Multi page APplication**
It take part of loading ⇒ doesn't take a entire page loading	**Take entire page loading methods**
Html request will given a Json response	**Html request will given html response**
Doesn't take content changing ⇒ but content swapping action will occur	**Content changing methods**
Ex: gmail ,	**Ex:tcs**

Features or Fundamentals ⇒ Reactjs

1. **VDOM ⇒ Layering of Original layer ⇒ duplicate of real dom**
2. **Component ⇒ Part of code or reusable block code ⇒ 2 class / Function**
3. **JSX ⇒ javascript xml ⇒ combination of js and html**
4. **State ⇒ current data, component, values**
5. **Props ⇒ parameter ⇒ value carrier**
6. **Event Handlers ⇒ js event handler**
7. **One Way data Binding ⇒ one way communication**
8. **Performance ⇒ speed / memory space**
9. **Simplicity ⇒ neat and clear web app**

```
import React from "react";
import Hari from "./Demo2";
function Akash() {
    return (
        <div>
            <h1>hello Hamith
                <Hari />
            </h1>
        </div>
    )
};
export default Akash;
```

```
import React from "react";

const Hari = () => {
    return (

        <div>
            <p>hello teamby hari</p>
        </div>
    )
}
```

```
export default Hari;
import React from "react";

class Vasan extends React.Component {
    render() {
        return (
            <div>
                <h1>hello friend by vasan</h1>
            </div>
        )
    }
}

export default Vasan;
```

```
import React from "react";
function Footer() {
    return (
        <div>
            <ul>
                <li> Limited Privacy NoticeCookie PolicyAccessibility
DeclarationDisclaimerSecurity Policy   California Notice at Collection
</li>
            </ul>
        </div>
    )
}
export default Footer;
```

```
import logo from './logo.svg';
import './App.css';
import Hello from './Hello';
import Vasan from './Demo3';
import Footer from './Footer';
function App() {
  return (
    <div className="App">
      <header className="App-header">
```

```
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Edit <code>src/App.js</code> and save to reload.
        </p>  hello world ...!....
        <h1>ganapathi raja</h1>
        <a
          className="App-link"
          href="https://reactjs.org"
          target="_blank"
          rel="noopener noreferrer"
        >
          Learn React
        </a>

        <div>
          hello visal
        </div>
        <Hello />
        <Vasan />
      </header>

      <Footer />
      <Footer />
      <Footer />
    </div>
  );
}

export default App;
```

```
import React from "react";
class Sample1 extends React.Component {
    render() {
        return (
            <div>
                <div>
                    <h1>hello world</h1>
                    <h2>Class Component </h2>
                </div>
```

```
            </div>
        )
    }
}
export default Sample1;
```

```
import React from "react";

function Sample2() {
    return (
        <div>
            <h2>Hello world hii</h2>
            <h1>Function component method </h1>
            <h4>Function expression method</h4>
        </div>
    )
};
export default Sample2;
```

```
import React from "react";

const Sample3 = () => {
    return (
        <div>
            <div>
                <h2>Hello world hii team</h2>
                <h1>Function component method </h1>
                <h4>Arrow Function  method</h4>
            </div>
        </div>
    )
};
export default Sample3;
```

```
import react from "react";
import { Fragment } from 'react'
import Sample6 from "./Sample6";
function Demopage1() {
```

```
    return (
        // <div>
        //     <div>
        //         <h1>hello world</h1>
        //         <h2>Alter Method </h2>
        //         <p>hii</p>
        //     </div>
        //     <div></div>
        // </div>

        // <section>
        //     <div>
        //         <h1>hello world hii team</h1>
        //         <p>hello friends</p>
        //     </div>
        // </section>

        // <article>
        //     <div>
        //         <h1>hello world hii team</h1>
        //         <p>hello friends</p>
        //     </div>
        // </article>

        // <>
        //     <div>
        //         <h1>hello world hii team</h1>
        //         <p>hello friends</p>
        //     </div>
        // </>

        <Fragment>
            <div>
                <h1>hello world hii team</h1>
                <p>hello friends</p>
            </div>
            <Sample6 />
        </Fragment>

    )
```

```
}
export default Demopage1;
import React from "react";
import { Fragment } from 'react'
class Sample6 extends React.Component {
    render() {
        return (
            <Fragment>
                <div>
                    <h1>hello world hii team</h1>
                    <p>hello friends</p>
                </div>
            </Fragment>
        )
    }
};
export default Sample6;
```

```
import React from "react"
export default class Pavi extends React.Component {
    render() {
        return (
            <div>
                <h1>hii react</h1>
            </div>
        )
    }
};

// import Component from 'react'
// export default class Sample7 extends Component {
//     render() {
//         return (
//             <h1>hiii</h1>
//         )
//     }
// }
```

Rules

```jsx
import React from "react";
import Carimg from '../Asset/car.jpg'
import './Rule1.css'
export default function Rule1() {
    let myname =""
    var muvalue =100
    const myObj ={

    }
    const myObj1 = []

    function myFun(){
        return
    }

    const myFun1 =()=>{
        document.getElementById()
    }
    return (
        // <span>
        //     <div>
        //         <h1>This is Rules of react Js </h1>
        //         <p>Html rule </p>

        //         <h1 > hii </h1>

        //         <img></img>
        //         {/* <br></br>
        //     <hr></hr>
        //     <input> </input>
        //     <link></link>

        //     <img />
        //     <input /> */}


        //         <img src={Carimg} alt="car " />
        //     </div>
        // </span>
        <div id="rules">
```

```
            <h1>This is Rules of react Js </h1>
            <p>Html rule </p>

            <h1 > hii </h1>

            <img></img>
            {/* <br></br>
            <hr></hr>
            <input> </input>
            <link></link>

            <img />
            <input /> */}


            <img src={Carimg} alt="car " />

            <div style={{
                color: "red",
                padding: "100px 60px",
                margin: '50px',
                backgroundColor: "green",
                fontSize: "45px"
            }}>
                hello world
            </div>

            <div style={myStyle}>
                hii ganapathi raja
            </div>


            <p id="heading">
                hii akash
            </p>

            <span className="heading">
                hi visal
            </span>
```

```
            <h1>
                hii kishore
            </h1>
        </div >
    )
}
const myStyle = {
    backgroundColor: "#000",
    color: "#fff",
    fontSize: "36px",
    padding: "60px"
}
```

```
#rules h1 {
    color: red;
}

#heading {
    background-color: aqua;
}

.heading {
    background-color: blanchedalmond;
    padding: 10px;
}

#heading,
.heading {}

* {}
```

```
import React from "react";
import Demo1 from "./demo1";
import Demo2 from "./Demo2";
import Rule1 from "./Rule1";
export default class Rulespage extends React.Component {
    myVlue = 100

    myObj ={
```

```
    }

    myFunc(){

    }

    myArrowmethod =()=>{

    }
    render() {
        return (
            <div>
                <h1>Rule Page In React</h1>
                <Demo1 />
                <hr />
                <Demo2 />
                <hr />
                <Rule1 />
            </div>
        )
    }
}
```

3 keys

```
import React from "react";
import Parentpageclass from "./Classparentpage";
import Parentfunction from "./Functionparentpage";

function Mainpage() {
    return (
        <div style={{ textAlign: "center" }}>
            <h1>Main Keys In React</h1>
            <p>state // props // event handlers</p>
            <p><strong>state</strong> == current Component / current data
/ current values / passing data within Component</p>
            <p><strong>props</strong> == parameters / arguments / value or
data carrier == passing the data component to component</p>
```

```
            <p><strong>Event Handlers</strong> == default ==
preventDefault,addeventlistes,onclick,etc....</p>
            <hr />
            <Parentpageclass />
            <hr />
            <hr />
            <Parentfunction />
        </div>
    )
}
export default Mainpage;
```

```
import React from "react";
import Childpageclass from "./Classchildpage";

export default class Parentpageclass extends React.Component {
    constructor() {
        super()

        this.state = {
            count: 5,
            applicants: 28
        }
        this.raja = {
            hi: 100
        }

    }
    addCount() {
        this.setState({ count: this.state.count + 1 })
    }

    applyJob() {
        this.setState({ applicants: this.state.applicants + 1 })
    }
    hi() {
        this.setRaja({ hi: this.raja.hi + 1 })
    }
```

```
    render() {
        return (
            <>
                <section>
                    <div>
                        <h1>Class Component method </h1>
                        <h2>This is Parent Page</h2>
                        <p>state // props // event handlers</p>
                        <div>
                            <p>counter :  <strong style={{ color: "red",
fontSize: "36px" }}> {this.state.count} </strong></p>
                            <button onClick={() => { this.addCount()
}}>add count</button>
                        </div>
                        <div>
                            <p>Applicants :  <strong style={{ color:
"red", fontSize: "36px" }}> {this.state.applicants} </strong></p>
                            <button onClick={() => { this.applyJob()
}}>Apply</button>
                        </div>

                        <div>
                            <p>Applicants :  <strong style={{ color:
"red", fontSize: "36px" }}> {this.raja.hi} </strong></p>
                            <button onClick={() => { this.hi()
}}>hi</button>
                        </div>

                        <div>
                            <span>{this.state.applicants}</span>
                        </div>
                    </div>

                    <div>
                        <Childpageclass parentData={this.state.count}
hello={"hii child"} />
                    </div>
                </section>
            </>
        )
```

```
    }
}
```

```
import React from "react";

// export default class Childpageclass extends React.Component {
//     render() {
//         return (
//             <div>
//                 <h1>Class Componet methods</h1>
//                 <h2>This is class Component Child page  </h2>

//                 <div>
//                     <p>counter :  <strong style={{ color: "red",
fontSize: "36px" }}> {this.props.parentData} </strong></p>
//                 </div>

//                 <p>Counter : <strong style={{ color: "blue", fontSize:
"26px" }}> 10 </strong></p>
//             </div>
//         )
//     }
// }

export default class Childpageclass extends React.Component {
    constructor(props) {
        super(props)
    }
    render() {
        return (
            <div>
                <h1>Class Componet methods</h1>
                <h2>This is class Component Child page  </h2>

                <div>
                    <p>counter :  <strong style={{ color: "red", fontSize:
"36px" }}> {this.props.parentData} </strong></p>
                </div>
```

```
                <p>Counter : <strong style={{ color: "blue", fontSize:
"26px" }}> 10 </strong></p>


                <h2>hii parent <small style={{ color: "red" }}>
{this.props.hello} </small></h2>
            </div>
        )
    }
}
```

```
import React from "react";
import { useState } from "react";
import Childpagefunc from "./Functionchildpage";
export default function Parentfunction() {
    const [count, setCount] = useState(10)
    const [applicant, setApplicants] = useState(10)

    function addCount() {
        setCount(count + 1)
    }
    return (
        <div>
            <div>
                <h1>Function Component method </h1>
                <h2>This is Parent Page</h2>
                <p>state // props // event handlers</p>

                <div>
                    <p>counter : <strong style={{ color: "green",
fontSize: "18px" }}> {count} </strong></p>
                    <button onClick={addCount}>Add count</button>

                    <button onClick={() => { setCount(count + 5)
}}>IncCount</button>
                </div>
            </div>

            <div>
```

```
                <Childpagefunc countValue={count} />
            </div>
        </div>
    )
};
```

```
import React from "react";

const Childpagefunc = ({ countValue }) => {
    return (
        <div>
            <h1>Class Componet methods</h1>
            <h2>This is class Component Child page  </h2>
            <div>
                <p>counter : <strong style={{ color: "green", fontSize:
"18px" }}> {countValue} </strong></p>
            </div>
        </div>
    )
};
export default Childpagefunc;
```

Class Component	Function Component
Class keyword	Function key word
Oops ⇒ inheritance	Js Function ⇒ primitive one of the data type
State,props must for value handlers	useState component methods
state,setState,this,controctore,super,	We have own keywords
Fundamental concepts	Js only the main part

State	Props
Current data / value / within the component	Value carrier to one component to another one component
Immutable	Immutable

Core Features	React Life cycle	React Hooks
Fundamental core values	Class component Features	Function feature ⇒ hooks
1. **VDOM** 2. **Component** 3. **JSX** 4. **State** 5. **Props** 6. **Event Handlers** 7. **One Way data Binding** 8. **Performance** 9. **Simplicity**	Phase / stages Mounting Updateting Unmounting constructor () getDerivedStateFromProps() render() componentDidMount() shouldComponentUpdate() getSnopshotBeforeUpdate() componentDidUpdate() componentWillUnMount()	useState() useEffect() useContext() createContext() useRef() useMemo() useCallback() useReducer() useLayout() useErrorDegger()

Hooks

```
import React from "react";
import Usemethod from "./Usemethod";
import Usemethod2 from "./Usemethod2";
import Usemethod3 from "./Usemeth3";

export default function Mainpagehooks() {
    return (
        <div style={{ textAlign: "center" }}>
            <h2>This is Function Component For Hooks Concepts</h2>
            <p>useState() // useEffect() // useContext()</p>
            <hr />
            <Usemethod />
            <hr />
            <Usemethod2 />
            <hr />
            <Usemethod3 />
        </div>
    )
```

```
}
```

```
import React from 'react'
import { useState } from 'react'

export default function Usemethod() {
    const [apply, setApply] = useState(40)
    const [ProName, setProfilename] = useState("Visal")

    const NextPropfile = () => {
        setProfilename("Kishore")
    }

    console.log('hii')
    return (
        <div>
            <h1>Usemethod</h1>
            <h2>useState()== current data / values / within a Component
</h2>

            <div>
                <p>Applicants:  {apply} </p>
                <button onClick={() => { setApply(apply + 1)
}}>apply</button>
            </div>


            <div>
                <p>Hi <strong> {ProName}  </strong>, thank you for showing
interest. Kindly answer all the recruiter's questions to successfully
apply for the job.</p>

                <button onClick={NextPropfile}>next Propfile</button>
            </div>
        </div>
    )
}
```

```
import React from 'react'
```

```jsx
import { useState } from 'react'
import { useEffect } from 'react'
export default function Usemethod2() {
    const [apply, setApply] = useState(40)
    const [ProName, setProfilename] = useState("Visal")

    const NextPropfile = () => {
        setProfilename("Kishore")
    }


    // useEffect(() => {
    //     console.log("hii")
    // }, [])

    // function applyJob() {
    //     setTimeout(() => {
    //         setApply(apply + 2)
    //     }, 2000)
    // }

    // useEffect(() => {
    //     applyJob()
    // })

    // useEffect(() => {
    //     setTimeout(() => {
    //         fetch("https://jsonplaceholder.typicode.com/users")
    //             .then((res) => res.json())
    //             .then((data) => { console.log(data) })
    //             .catch((err) => { console.error(err) })
    //     }, 5000)

    //     setTimeout(() => {
    //         setProfilename("Nasika & Maha")
    //     }, 8000)
    // })

    const [userInfo, setUserinfo] = useState([])
    useEffect(() => {
```

```
        setTimeout(() => {
            fetch("https://jsonplaceholder.typicode.com/users")
                .then((res) => res.json())
                .then((data) => {
                    console.log(data);
                    setUserinfo(data)
                })
                .catch((err) => { console.error(err) })
        }, 5000)
    }, [])


    return (
        <div>
            <h1>Usemethod 2</h1>
            <h2>useEffect()== side effect of  current data / values /
within a Component </h2>

            <div>
                <p>Applicants:  {apply} </p>
                <button >apply</button>
            </div>


            <div>
                <p>Hi <strong> {ProName}  </strong>, thank you for showing
interest. Kindly answer all the recruiter's questions to successfully
apply for the job.</p>

                <button onClick={NextPropfile}>next Propfile</button>
            </div>

            <div>
                <h4>fetching datas</h4>
                {
                    userInfo.map((u) =>
                        <p key={u.id}> {u.name}  </p>
                    )
                }
            </div>
```

```
        </div >
    )
}

import React from 'react'
import Child1 from './Child1'
import Child2 from './Child2'
import Child3 from './Child3'
import Child4 from './Child4'
import Child5 from './Child5'
import myStore from './Hub'
export default function Usemethod3() {
    return (
        <div>
            <h1>Usemeth3</h1>
            <p>useContext()</p>

            <Child1 hi="hii child how are ?" />
            <Child2 hi="hii  how are ? " />
            <Child3 hi="hii child  how are ?" />


            <myStore.Provider value={"hii child how are ? hello hii
world"}>
                <Child4 />
                <Child5 />
            </myStore.Provider>
        </div>
    )
}
```

```
import React from "react";
import { createContext } from "react";
const myStore = createContext()
export default myStore;
```

```
import React from 'react'
import myStore from './Hub'
import { useContext } from 'react'
```

```
export default function Child4() {
    const myData = useContext(myStore)
    return (
        <div>Child4
            <h2>hii parent4 <span style={{ color: "green", fontSize:
"28px" }}> {myData}</span> </h2>
        </div>
    )
}
```

```
import React from 'react'
import { useContext } from 'react'
import myStore from './Hub'

export default function Child5() {
    const myValue = useContext(myStore)
    return (
        <div>Child5
            <h2>hii parent4 <span style={{ color: "green", fontSize:
"28px" }}> {myValue}</span> </h2>
        </div>
    )
}
```

React Form

```
.myform {
    background-color: aliceblue;
    padding: 70px 60px;
    text-align: center;
    display: flex;
    justify-content: center;
    align-items: center;
}

.mycontactform {
    background-color: antiquewhite;
```

```
    padding: 20px 30px;
    margin: 10px 10px;
    width: 600px;
}

.mycontactform .inputs {
    text-align: left;
    padding: 5px 5px;
    font-weight: 600;
}

.mycontactform .inputs input,
.mycontactform .inputs textarea {
    width: 100%;
    padding: 10px;
    font-size: 17px;
    color: blueviolet;
    border: 1px solid blueviolet;
    outline: none;
}

.mycontactform .inputs input::placeholder,
.mycontactform .inputs textarea::placeholder {
    color: brown;
    font-size: 16px;
    font-style: italic;
}

.inputs p {
    color: red;
    font-size: 13px;
}

.inputs button {
    padding: 15px 30px;
    margin: 5px;
    border: none;
    background-color: blueviolet;
    color: #fff;
    font-size: 18px;
```

```
    font-weight: 600;
}
```

```
import React from "react";
import './Form.css'
import { useState } from "react";
export default function Reactform() {
    const [userdata, setUSerdata] = useState({ uname: "", uemail: "",
uMobile: "", uReason: "", uMessages: "" });

    const datachange = (uinfo) => {
        setUSerdata({ ...userdata, [uinfo.target.name]: uinfo.target.value
})
    }


    function formsubmit(uinfo) {
        uinfo.preventDefault();

        if (!userdata.uname || !userdata.uemail || !userdata.uMobile ||
!userdata.uReason || !userdata.uMessages) {
            alert("please fii the data")
        }
        else {
            console.log(userdata);
            alert("Form Submitted successfully...!!")
            setUSerdata({ uname: "", uemail: "", uMobile: "", uReason: "",
uMessages: "" })
        }
    }
    return (
        <section className="myform">
            <div className="mycontactform">
                <form onSubmit={formsubmit}>
                    <h2>This React Contact Page</h2>
                    <div className="inputs">
                        <label htmlFor="cname">USer Name</label>
                        <div>
```

```
                            <input type="text" name="uname"
placeholder="Enter your name"
                                onChange={datachange}
value={userdata.uname} />
                            {/* <p>Please Fill the data</p> */}
                        </div>
                    </div>
                    <div className="inputs">
                        <label htmlFor="cname">USer email</label>
                        <div>
                            <input type="email" name="uemail"
placeholder="Enter your Email"
                                onChange={datachange}
value={userdata.uemail} />
                            {/* <p>Please Fill the data</p> */}
                        </div>
                    </div>
                    <div className="inputs">
                        <label htmlFor="cname">USer Mobile</label>
                        <div>
                            <input type="tel" name="uMobile"
placeholder="Enter your Mobile"
                                onChange={datachange}
value={userdata.uMobile} />
                            {/* <p>Please Fill the data</p> */}
                        </div>
                    </div>
                    <div className="inputs">
                        <label htmlFor="cname">USer Reason</label>
                        <div>
                            <input type="text" name="uReason"
placeholder="Enter your Reason"
                                onChange={datachange}
value={userdata.uReason} />
                            {/* <p>Please Fill the data</p> */}
                        </div>
                    </div>
                    <div className="inputs">
                        <label htmlFor="cname">USer Messages</label>
                        <div>
```

```
                            <textarea name="uMessages" placeholder="Enter
your Messages"
                                onChange={datachange}
value={userdata.uMessages}></textarea>
                            {/* <p>Please Fill the data</p> */}
                        </div>
                    </div>
                    <div className="inputs">
                        <div>
                            <button type="submit">Send
Messages...!</button>
                        </div>
                    </div>
                </form>
            </div>
        </section>
    )
}
```

```
import React from "react";
import './Form.css'
import { useState } from "react";
export default function Reactform() {
    const [userdata, setUSerdata] = useState({ uname: "", uemail: "",
uMobile: "", uReason: "", uMessages: "" });
    // const [err, setErr] = useState()
    const [err, setErr] = useState({})

    const datachange = (uinfo) => {
        setUSerdata({ ...userdata, [uinfo.target.name]: uinfo.target.value
})
        setErr({})
    }

    function checkFormdata() {
        let tempErr = {};

        if (!userdata.uname.trim()) {
            tempErr.uname = "please fill the name data"
        }
```

```
        else if (userdata.uname.length < 3) {
            tempErr.uname = "please fill the correct name data"
        }
        // else {
        //     tempErr.uname = ""
        // }

        const myEmailpattern = /[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$/
        if (!userdata.uemail.trim()) {
            tempErr.uemail = "please fill the email data"
        }
        else if (!(myEmailpattern.test(userdata.uemail))) {
            tempErr.uemail = "please fill the correct email data"
        }
        // else {
        //     tempErr.uemail = ""
        // }

        const muMobilepattern = /[6-9][0-9]{9}/
        if (!userdata.uMobile.trim()) {
            tempErr.uMobile = "please fill the  mobile daata"
        }
        else if (!(muMobilepattern.test(userdata.uMobile))) {
            tempErr.uMobile = "please fill correct mobile data"
        }
        // else {
        //     tempErr.uMobile = ""
        // }

        if (!userdata.uReason.trim()) {
            tempErr.uReason = "please fill data"
        }
        // else {
        //     tempErr.uReason = ""
        // }

        if (!userdata.uMessages.trim()) {
            tempErr.uMessages = "please fill the data messages"
        }
        // else {
```

```
        //     tempErr.uMessages = ""
        // }

        setErr(tempErr);

        return Object.keys(tempErr).length === 0;
    }


    function formsubmit(uinfo) {
        uinfo.preventDefault();

        // if (!userdata.uname || !userdata.uemail || !userdata.uMobile ||
!userdata.uReason || !userdata.uMessages) {
        //     // alert("please fii the data")
        //     setErr("please fii the data")
        // }
        // else {
        //     console.log(userdata);
        //     alert("Form Submitted successfully...!!")
        //     setUSerdata({ uname: "", uemail: "", uMobile: "", uReason:
"", uMessages: "" })
        // }

        if (checkFormdata()) {
            console.log(userdata);
            alert("Form Submitted successfully...!!")
            setUSerdata({ uname: "", uemail: "", uMobile: "", uReason: "",
uMessages: "" })
        }
    }
    return (
        <section className="myform">
            <div className="mycontactform">
                <form onSubmit={formsubmit}>
                    <h2>This React Contact Page</h2>
                    <div className="inputs">
                        <label htmlFor="cname">USer Name</label>
                        <div>
```

```
                            <input type="text" name="uname"
placeholder="Enter your name"
                                onChange={datachange}
value={userdata.uname} />
                            {err.uname && <p> {err.uname} </p>}
                        </div>
                    </div>
                    <div className="inputs">
                        <label htmlFor="cname">USer email</label>
                        <div>
                            <input type="email" name="uemail"
placeholder="Enter your Email"
                                onChange={datachange}
value={userdata.uemail} />
                            {err.uemail && <p> {err.uemail} </p>}
                        </div>
                    </div>
                    <div className="inputs">
                        <label htmlFor="cname">USer Mobile</label>
                        <div>
                            <input type="tel" name="uMobile"
placeholder="Enter your Mobile"
                                onChange={datachange}
value={userdata.uMobile} />
                            {err.uMobile && <p> {err.uMobile} </p>}
                        </div>
                    </div>
                    <div className="inputs">
                        <label htmlFor="cname">USer Reason</label>
                        <div>
                            <input type="text" name="uReason"
placeholder="Enter your Reason"
                                onChange={datachange}
value={userdata.uReason} />
                            {err.uReason && <p> {err.uReason} </p>}
                        </div>
                    </div>
                    <div className="inputs">
                        <label htmlFor="cname">USer Messages</label>
                        <div>
```

```
                            <textarea name="uMessages" placeholder="Enter
your Messages"
                                onChange={datachange}
value={userdata.uMessages}></textarea>
                            {err.uMessages && <p> {err.uMessages} </p>}
                        </div>
                    </div>
                    <div className="inputs">
                        <div>
                            <button type="submit">Send
Messages...!</button>
                        </div>
                    </div>
                </form>
            </div>
        </section>
    )
}
```

```
import React, { useState } from "react";
import "./Form.css";

export default function Reactform() {
    const [userdata, setUserdata] = useState({
        uname: "",
        uemail: "",
        uMobile: "",
        uReason: "",
        uMessages: "",
    });

    const [err, setErr] = useState({});

    // Handle input changes
    const datachange = (e) => {
        const { name, value } = e.target;

        setUserdata((prev) => ({
```

```
            ...prev,
            [name]: value,
        }));

        setErr((prev) => ({
            ...prev,
            [name]: "",
        }));
    };

    // Validate form data
    function checkFormdata() {
        const tempErr = {};

        // Name validation
        if (!userdata.uname.trim()) {
            tempErr.uname = "Please enter your name.";
        } else if (userdata.uname.trim().length < 3) {
            tempErr.uname = "Name must contain at least 3 characters.";
        }

        // Email validation
        const myEmailPattern =
            /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;

        if (!userdata.uemail.trim()) {
            tempErr.uemail = "Please enter your email.";
        } else if (!myEmailPattern.test(userdata.uemail.trim())) {
            tempErr.uemail = "Please enter a valid email address.";
        }

        // Mobile validation (Indian mobile number)
        const myMobilePattern = /^[6-9][0-9]{9}$/;

        if (!userdata.uMobile.trim()) {
            tempErr.uMobile = "Please enter your mobile number.";
        } else if (!myMobilePattern.test(userdata.uMobile.trim())) {
            tempErr.uMobile = "Enter a valid 10-digit Indian mobile
number.";
        }
```

```
        // Reason validation
        if (!userdata.uReason.trim()) {
            tempErr.uReason = "Please enter your reason.";
        }

        // Message validation
        if (!userdata.uMessages.trim()) {
            tempErr.uMessages = "Please enter your message.";
        } else if (userdata.uMessages.trim().length < 10) {
            tempErr.uMessages = "Message must contain at least 10
characters.";
        }

        setErr(tempErr);

        return Object.keys(tempErr).length === 0;
    }

    // Handle form submission
    function formsubmit(e) {
        e.preventDefault();

        if (checkFormdata()) {
            console.log("Form Data:", userdata);

            alert("Form submitted successfully!");

            setUserdata({
                uname: "",
                uemail: "",
                uMobile: "",
                uReason: "",
                uMessages: "",
            });

            setErr({});
        }
    }
```

```
    return (
        <section className="myform">
            <div className="mycontactform">
                <form onSubmit={formsubmit} noValidate>
                    <h2>React Contact Form</h2>

                    <div className="inputs">
                        <label htmlFor="uname">User Name</label>
                        <div>
                            <input
                                id="uname"
                                type="text"
                                name="uname"
                                placeholder="Enter your name"
                                onChange={datachange}
                                value={userdata.uname}
                            />
                            {err.uname && <p>{err.uname}</p>}
                        </div>
                    </div>

                    <div className="inputs">
                        <label htmlFor="uemail">User Email</label>
                        <div>
                            <input
                                id="uemail"
                                type="email"
                                name="uemail"
                                placeholder="Enter your email"
                                onChange={datachange}
                                value={userdata.uemail}
                            />
                            {err.uemail && <p>{err.uemail}</p>}
                        </div>
                    </div>

                    <div className="inputs">
                        <label htmlFor="uMobile">User Mobile</label>
                        <div>
                            <input
```

```
                    id="uMobile"
                    type="tel"
                    name="uMobile"
                    placeholder="Enter your mobile number"
                    onChange={datachange}
                    value={userdata.uMobile}
                />
                {err.uMobile && <p>{err.uMobile}</p>}
            </div>
        </div>

        <div className="inputs">
            <label htmlFor="uReason">Reason</label>
            <div>
                <input
                    id="uReason"
                    type="text"
                    name="uReason"
                    placeholder="Enter your reason"
                    onChange={datachange}
                    value={userdata.uReason}
                />
                {err.uReason && <p>{err.uReason}</p>}
            </div>
        </div>

        <div className="inputs">
            <label htmlFor="uMessages">Message</label>
            <div>
                <textarea
                    id="uMessages"
                    name="uMessages"
                    placeholder="Enter your message"
                    onChange={datachange}
                    value={userdata.uMessages}
                />
                {err.uMessages && <p>{err.uMessages}</p>}
            </div>
        </div>
```

```
                    <div className="inputs">
                        <button type="submit">Send Message</button>
                    </div>
                </form>
            </div>
        </section>
    );
}
```

```
import React from 'react'
import { useState } from 'react'
export default function Commontopic() {
    const [apply, setApply] = useState(10);
    const [theme, setTheme] = useState(false)

    return (
        <div>Commontopic



            <div style={{
                padding: "75px",
                margin: "20px",
                backgroundColor: theme ? "green" : "black",
                color: theme ? "red" : "#fff"
            }}>
                <h1>hello world</h1>
                <button onClick={() => { setTheme(!theme) }}>change
bg</button>
            </div>


            <div>
                <p> Applicants <strong>  {apply} </strong></p>
                <button style={{
                    backgroundColor: theme ? "blue" : "green",
                    color: "#fff",
                    padding: "20px"
                }}
```

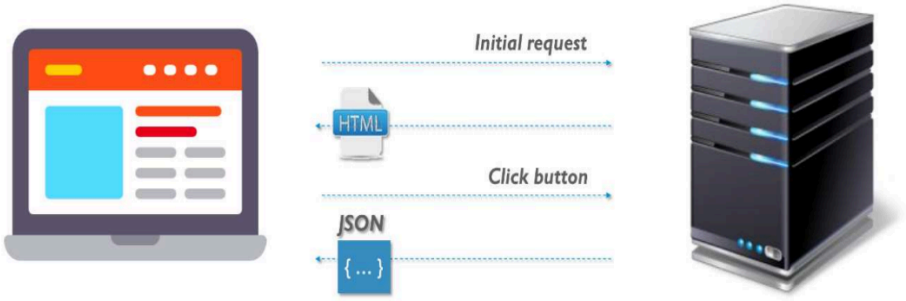
```
                    onClick={() => { setTheme(!theme) }}>

                    {theme ? "apply" : "Applied"}
                </button>
            </div>


        </div>
    )
}
```

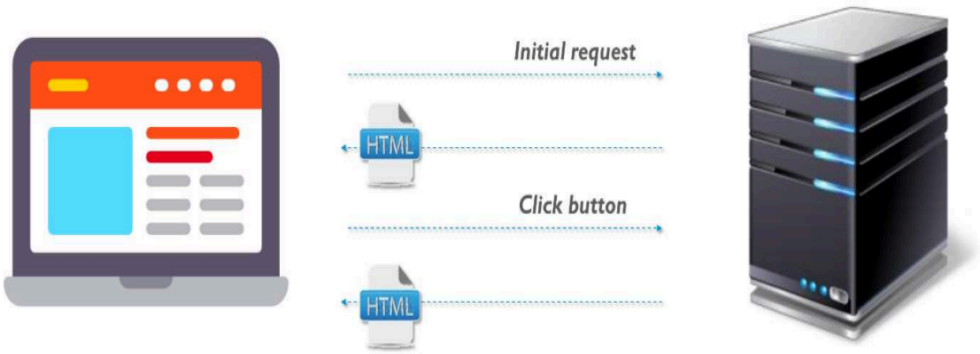
SPA (Single Page Application)

- It is an application that works inside a browser and does not require page reloading during use.
- Instead of serving a brand new page to the user, SPA swaps out the old content for new in case of any user interaction.
- It is faster, more responsive, compact, and easy to develop and deploy.
- SPA and all its content is only loaded once when the user first interacts with the web page.



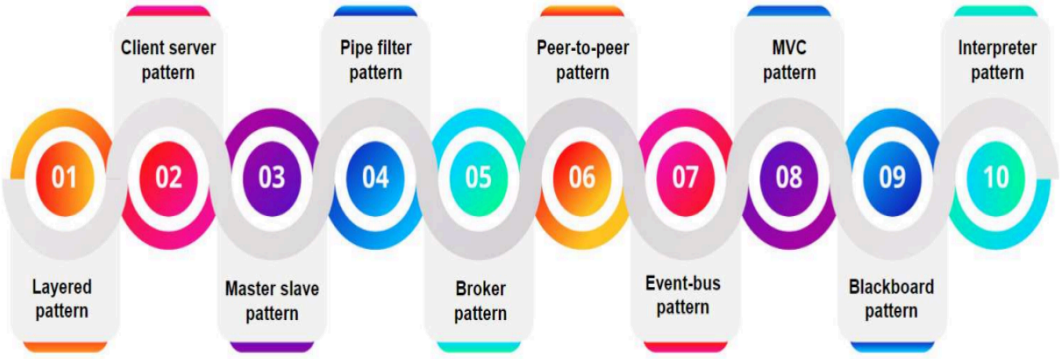
MPA (Multi Page Application)

- In an MPA, each time you click on a link or interact with the application, a new page is downloaded from the server and then rendered in the web browser.
- Here, content is organized on individual pages that are usually static.
- They do not change in response to the user's action.
- A brand new page with its own static content is served when a user clicks any button.



Web App Architecture Patterns

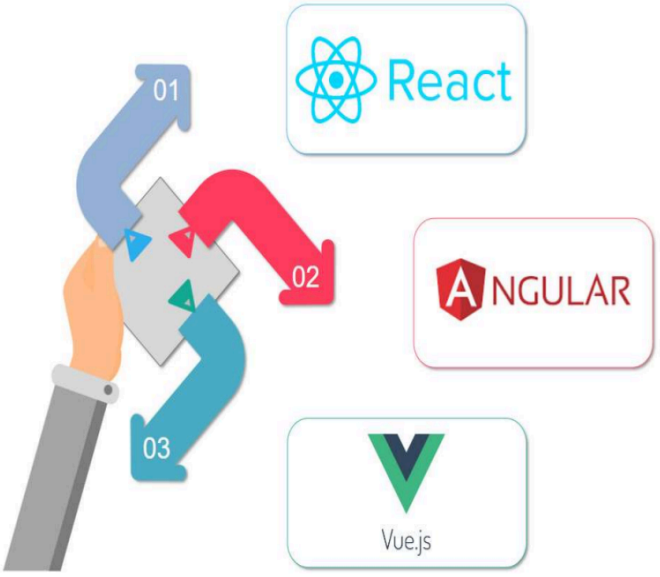
An architectural pattern is a general reusable solution to a commonly occurring problem in web development within a given context. Given below is a list of ten commonly used architectural patterns:



Usually, the MVC pattern is the opted architectural pattern by frontend technologies to develop web applications.

Different Client Side Technologies

The most commonly used front-end (client-side) technologies are:

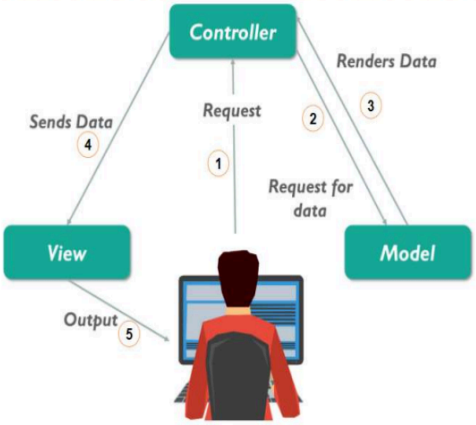


MVC Architecture

MVC is an architectural pattern commonly used for developing user interfaces that divides an application into three interconnected parts — model, view, and controller

It is a software code that controls the interactions between model and view.

A view is what you see when you visit a site. For example: blog post, contact form, etc. It sends the data to the client. Generally, views are HTML documents.



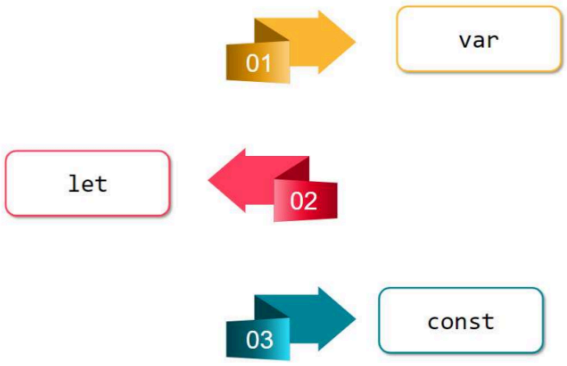
It represents application data. It retrieves, stores, and maintains the data of the application in a database.

Model View Controller

ECMAScript Variables

Variables are the defined spaces in the memory to store values in a program.

JavaScript supports dynamic typing, where variables should be necessarily defined before they are used. The majorly used ECMAScript variable types are:



ECMAScript Variables: var

var

let

const

ES5 makes use of the **var** keyword, and it was the only way to declare the variables in JavaScript; it had certain following limitations:

It has function scope, that is, if you want to restrict the access of variables within the functions it should be defined inside the function, otherwise, it will belong to the global scope.

Variables can be declared more than one time, and their values can be reassigned anytime in the same project.

ECMAScript Variables: var (Code Example)

var

let

const

Code example (input)

```
var x = 10
function test() {
var x = 100
console.log("value of x as per defined function
"+x)
}
console.log("value of x before declaration of
function "+x)
test()
```

Declaring variable globally

Declaring variable locally

Code output

```
PS C:\Users\archana\Desktop\Node.js\Node.js-Demo\firstapp> node ES.js
value of x before declaration of function 10
value of x as per defined function 100
```

ECMAScript Variables: let

var

let

const

ES6 introduced variables **let and const** which provide **block scope**.

- Block scope is the area within if, switch conditions, for, and while loops, where variable declared within curly braces { } remains available only in this scope.
- With let, you can reassign the value of the variable any time, but you cannot declare the variable more than one time.

Code example

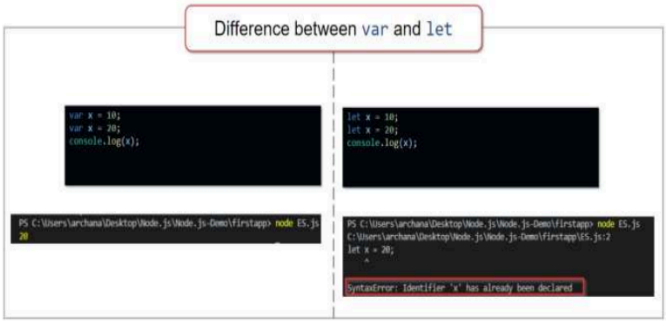
```
if(true)
{
let x = 1;
console.log(x);
}
console.log(x)
```

Difference Between var and let

var

let

const



Hence, any variable declared using the let keyword is assigned the block scope.

ECMAScript Variables: const

var

let

const

The value assigned to a **const** variable is **immutable.**

1. Value must be assigned to a const variable at the time of declaration, and it later cannot be reassigned or re-declared within the same program.

2. In case of declaring a new array using const variable, you can change the value of an element within the array.

Code example

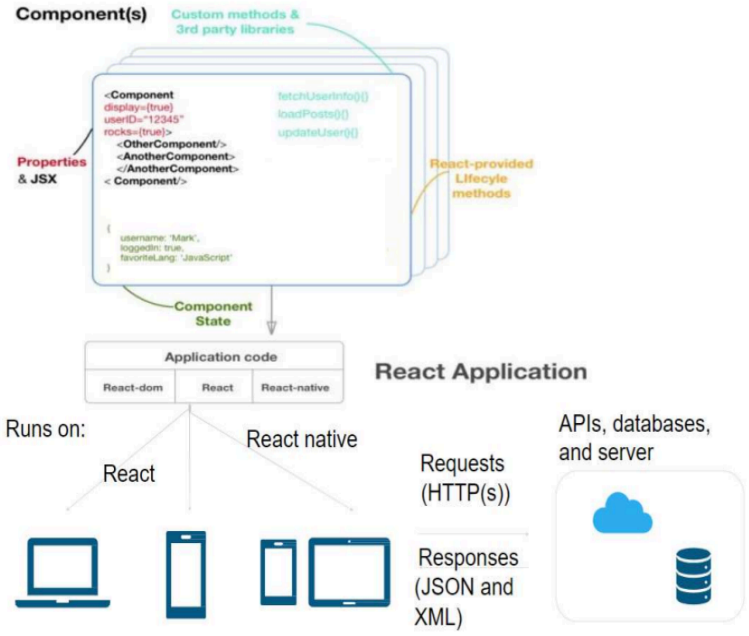
```
const a = 0;
const a = 1;
const b = [1,2];
b.push(3);
b[3] = 4;
console.log(a,b)
```

Assigns value '0' to constant *a*
TypeError: Assignment to constant variable
Sets b = [1,2,3]
Sets b = [1,2,3,4]

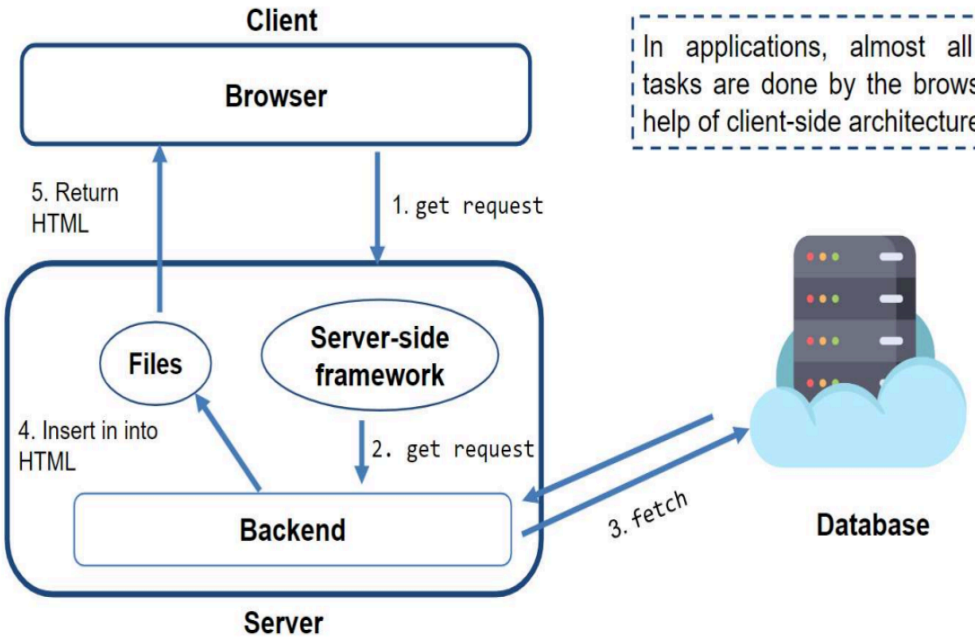
React Architecture

High level flow

- React allows the creation of an interface with smaller standalone entities called components.
- Components interact with each other and with the parent through props.
- Same code can run on browsers as a web application and on Android and iOS devices as a native application.



Client Side Architecture



In applications, almost all UI-related tasks are done by the browser with the help of client-side architecture.

Advantages of React



Building rich UI
It allows you to develop engaging and superior quality interfaces with its declarative components feature.

Short learning curve
React offers a lower learning curve; all you need to know is JavaScript and HTML.

Create custom components
JavaScript XML (JSX) allows creating unique components to suit your requirements and renders sub-components with ease.

Quick rendering
With virtual DOM, any change made in the app can be tested to check the risk level before finalizing it.

Boosts productivity
It enables developers to redistribute the same objects with the help of component reusing feature.

Advantages of React (contd.)

Community support
React developers are actively uploading text/video tutorials online and answer queries on various forums and QA sites.

Code stability
The data flow in React certifies that the parent structure is not impacted by any kind of changes made in the child structure.

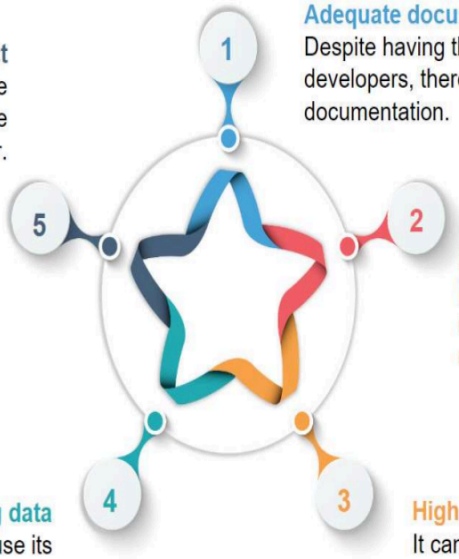


SEO-friendly
It is having fast load time and rendering which plays important for higher ranking in Search Engine Results Pages (SERP).

Useful developer toolset
Companies like Facebook have included several developer tools for React framework to help developers immensely in discovering parent and child components, observing their hierarchies, and analyze their present state, etc.

Disadvantages of React

Setting up React
Setting up React takes more time than other frameworks like Vue and Angular.

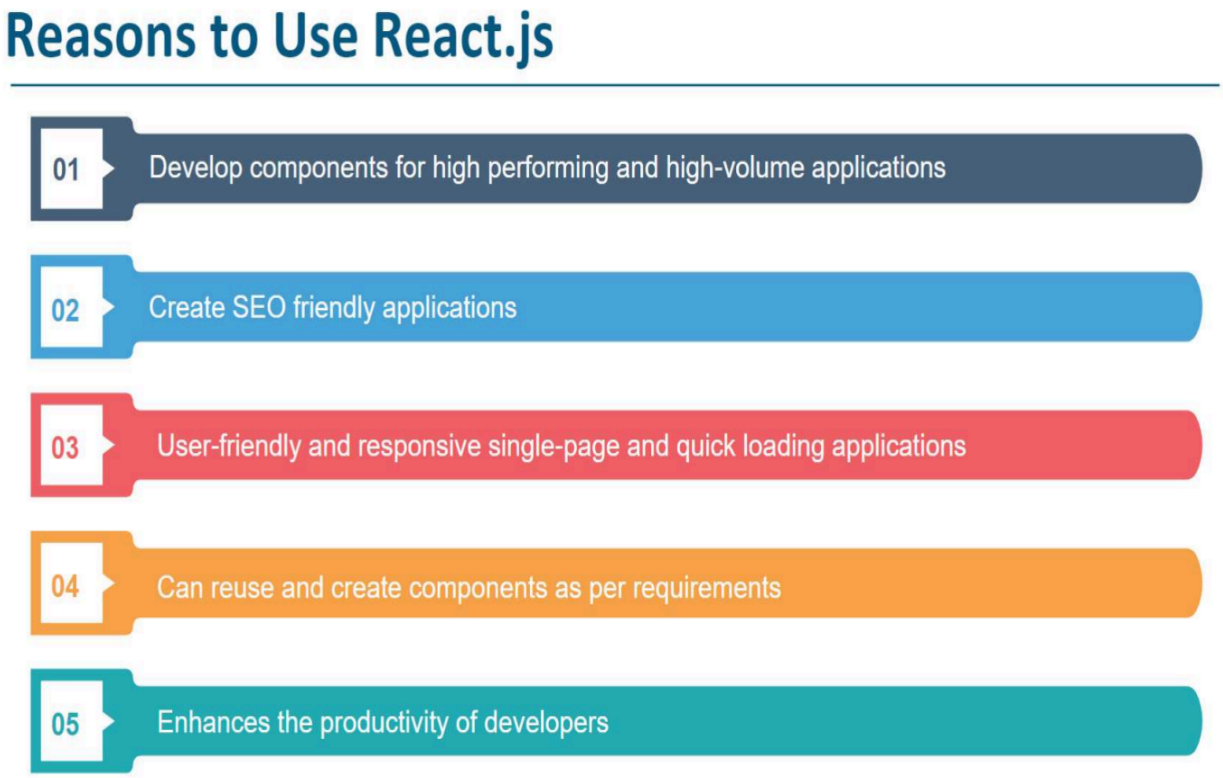


Adequate documentation
Despite having the largest community of reacting developers, there is a need of proper documentation.

Develop UI only
It is not possible to develop the complete application using React.js only.

Migrating data
Apps future must be certain because its very difficult and time-consuming to migrate from react.

High pace
It can build applications very fast, which can put a lot of pressure on the developers.



State props event handler

Class component

```
import React from "react";
import './Keys.css';
import Classparent from "./Classcomponentparent";
import Functionparent from "./Functionparent";
class Keyspage extends React.Component {
    render() {
        return (
            <div className="mainpage" style={{ textAlign: "center" }}>
                <h1>Major Three Keys </h1>
                <p>state / Props / Event Handlers</p>
                {/* <Classparent /> */}

                <Functionparent />
            </div>
        )
```

```
    }
};
export default Keyspage;
```

```
import React from "react";
import Classchild from "./Classcomponetchild";

export default class Classparent extends React.Component {
    constructor() {
        super()
        this.state = {
            applicants: 50,
            candiadte: "Vgr",
            candiadte: "Sibi",
            candiadte: "Sakthi",
        }

        this.count = {
            counts: 10
        }
    }



    applyJob() {
        this.setState({ applicants: this.state.applicants + 1 })
    }

    nextCandiadte() {
        this.setState({ candiadte: this.state.candiadte = "Bala" })
    }


    incCount() {
        this.setCount({ counts: this.count.counts + 2 })
    }
    render() {
        return (
            <section>
```

```
                <h2>This is Class Component Method </h2>
                <p>state / props / event handler </p>
                <h1>Parent page </h1>

                <p>Applicants: <strong>   {this.state.applicants}
</strong> </p>
                <button onClick={() => { this.applyJob() }}
>Apply</button>



                <div>
                    <p>Thank you for applying <strong>
{this.state.candiadte} </strong>  we will get back</p>
                    <button onClick={() => { this.nextCandiadte() }}>Next
Candidate</button>
                </div>


                <div>
                    <p>my count : {this.count.counts} </p>
                    <button onClick={() => { this.incCount() }}>inc
Count</button>
                </div>

                <br /><br /><br /><br /><br />
                <Classchild count={this.state.applicants} />
                <br /><br /><br /><br /><br />



                <h1>hello my count    {this.state.applicants} </h1>
            </section>
        )
    }
}
```

```
import React from "react";

export default class Classchild extends React.Component {
    // constructor(props) {
    //     super(props)
    // }
    render() {
        return (
            <>
                <h2>This is Class Component Method </h2>
                <h1>Child page </h1>
                <p>hii team</p>

                <h1>hello my count   <span style={{ color: "red" }}>
{this.props.count}  </span></h1>
            </>
        )
    }
}
```

```
import React from "react";
import Functionchild from "./Functionchild";
import { useState } from "react";

export default function Functionparent() {
    const [applicants, setApplicants] = useState(41);
    const [count, setCount] = useState(41);

    function applyJob() {
        setApplicants(applicants + 1)
    }
    return (
        <div style={{ textAlign: "center" }}>

            <div>
                <h2>This is Functional  Component Method </h2>
                <p>state / props / event handler </p>
                <h1>Parent page </h1>
```

```
                <div>
                    <p>Applicants: <strong> {applicants} </strong></p>
                    <button onClick={applyJob}>Apply</button>
                    <br />
                    <button onClick={() => { setApplicants(applicants + 5)
}}>Apply</button>
                </div>
            </div>

            <br /><br /><br /><br /><br />
            <Functionchild data={applicants} hi="how are you?.....!!" />
            <br /><br /><br /><br /><br />
        </div>
    )
}
```

```
import React from "react";

const Functionchild = ({ data, hii }) => {
    return (
        <>
            <h2>This is Functional Component Method </h2>
            <h1>Child page </h1>
            <p>hii team</p>
            <h1>applicant : <span style={{ color: "red" }}> {data}
</span></h1>

            <p style={{ color: "green" }}> {hii} </p>
        </>
    )
}
export default Functionchild;
```

```
import React from "react";
import { useState } from "react";
export default function Usemethod1() {
    const [apply, setApply] = useState(17);
    // const [candidate, setCondidate] = useState("Raja");
    const [candidate, setCondidate] = useState("sibi");
    const jobAply = () => {
        setApply(apply + 1)
    }

    function nextCandi() {
        setCondidate("Anu")
    }
    return (
        <section>
            <h2>Hook method #1</h2>
            <h1>useState()</h1>
            <p>current State / Data / Component</p>
            <p>Applicants: <strong>  {apply}  </strong> </p>
            <button onClick={jobAply}>Apply</button>


            <div>
                <p>Thank you!... <strong>  {candidate} </strong>  Your
Application Submitted successfully we will get back Soon...!!</p>
            </div>

            <div>
                <p>Hr Side </p>
                <p>Current Candi :  <strong>  {candidate} </strong></p>
                <button onClick={nextCandi}>next profile</button>
            </div>
        </section>
    )
}
```

```
import React from "react";
import { useState } from "react";
import { useEffect } from "react";
```

```
export default function Usemethod2() {
    const [apply, setApply] = useState(17);
    const [candidate, setCondidate] = useState("sibi");
    const jobAply = () => {
        setTimeout(() => {
            setApply(apply + 1)
        }, 2000)
    }

    // useEffect(() => {
    //     setTimeout(() => {
    //         setApply(apply + 1)
    //     }, 2000)
    // })

    function nextCandi() {
        setCondidate("Anu")
    }

    //scheduling concepts
    // useEffect(() => {
    //     setTimeout(() => { alert("hello team") }, 3000)

    //     setInterval(() => { alert("hii team") }, 4000)
    // }, [])



    //fetching method only under useEffect
    // useEffect(() => {
    //     setInterval(() => {
    //         fetch("https://jsonplaceholder.typicode.com/users")
    //         // axios("https://jsonplaceholder.typicode.com/users")
    //             .then((res) => res.json())
    //             .then((data) => { console.log(data) })
    //             .catch((err) => { console.error(err) })
    //     }, 2000)
    // })

    const [users, setUsers] = useState([]);
    useEffect(() => {
```

```
        setInterval(() => {
            fetch("https://jsonplaceholder.typicode.com/users")
                .then((res) => res.json())
                .then((data) => { console.log(data); setUsers(data) })
                .catch((err) => { console.error(err) })
        }, 2000)
    }, [])

    return (
        <section>
            <h2>Hook method #2</h2>
            <h1>useEffect()</h1>
            <p>current State effext/ Data / Component side effect</p>
            <p>Scheduling concepts / Fetcing concets / subcribtion </p>
            <p>Applicants: <strong>  {apply}  </strong> </p>
            <button onClick={jobAply}>Apply</button>


            <div>
                <p>Thank you!... <strong>  {candidate} </strong>  Your
Application Submitted successfully we will get back Soon...!!</p>
            </div>

            <div>
                <p>Hr Side </p>
                <p>Current Candi :  <strong>  {candidate} </strong></p>
                <button onClick={nextCandi}>next profile</button>
            </div>

            <div>
                {
                    users.map((infos) =>
                        <p key={infos.id}> {infos.name} </p>
                    )
                }
            </div>
        </section>
    )
}
```

```
import React from "react";
import Method3child1 from "./Method3child1";
import Method3child2 from "./Method3child2";
import Method3child3 from "./Method3child3";
import Method3child4 from "./Method3child4";
import Method3child5 from "./Method3child5";
import Method3child6 from "./Method3child6";
import Wherehouse from "./Method3hub";

export default function Method3() {
    return (
        <div>
            <h2>Hook method #3</h2>
            <h1>useContext()</h1>
            <p>Local / Globel State managemnet Component </p>
            <p>You can pass same value or differnt datas to n number child
</p>

            <div>
                <Method3child1 hi="hii child how are you ?? " />
                <Method3child2 hi="hii child how are you ?? " />
                <Method3child3 hi="hii child how are you ?? " />
            </div>

            <Wherehouse.Provider value={"hii child how are you ?? "}>
                <div>
                    <Method3child4 />
                    <Method3child5 />
                    <Method3child6 />
                </div>
            </Wherehouse.Provider>
        </div>
    )
}
```

```
import React from 'react'

export default function Method3child1({ hi }) {
    return (
```

```
        <div>method3child1
            <p>child 1</p>
            <h2> {hi}  </h2>
        </div>
    )
}
```

```
import React from 'react'

export default function Method3child2({ hi }) {
    return (
        <div>Method3child2
            <div>method3child2
                <p>child 2</p>
                <h2> {hi}  </h2>
            </div>
        </div>
    )
}
```

```
import React from 'react'

export default function Method3child3({ hi }) {
    return (
        <div>Method3child3
            <div>method3child3
                <p>child 3</p>
                <h2>  {hi} </h2>
            </div>
        </div>
    )
}
```

```
import React from 'react'
import Wherehouse from './Method3hub'
import { useContext } from 'react'
export default function Method3child4() {
```

```
    const Mydata = useContext(Wherehouse)
    return (
        <div>Method3child4

            <h1>  {Mydata} </h1>
        </div>
    )
}
```

```
import React from 'react'
import Wherehouse from './Method3hub'
import { useContext } from 'react'
export default function Method3child5() {
    const myData = useContext(Wherehouse)
    return (
        <div>Method3child5
            <h1> {myData} </h1>
        </div>
    )
}
```

```
import React from 'react'
import Wherehouse from './Method3hub'
import { useContext } from 'react'
export default function Method3child6({ hi }) {
    const Ranganathan = useContext(Wherehouse)
    return (

        <div>Method3child6
            <h1> {Ranganathan} </h1>

            <h1>{hi} </h1>
        </div>

    )

}
```

```
import React from "react";
import { createContext } from "react";
const Wherehouse = createContext();
export default Wherehouse;
```

```
import { type } from '@testing-library/user-event/dist/type';
import React from 'react'
import { useState } from 'react'
import { useReducer } from 'react';
function arithmeticOperation(state, action) {
    switch (action.type) {
        case "add":
            return { count: state.count + 1 }
            break
        case "sub":
            return { count: state.count - 1 }
            break
        case 3:
            return { count: state.count * 10 }
            break
        case 4:
            return { count: state.count / 2 }
            break
        case 5:
            return { count: state.count % 2 }
            break
        case 6:
            return { count: state.count++ }
            break


        case "reset":
            return { count: state.count = 0 }
            break
        default:
            return (state)
    }
}
export default function Method4() {
```

```
    // const [count, setCount] = useState(0);

    // function incCount() {
    //     setCount(count + 1)
    // }

    // function decCount() {
    //     setCount(count - 1)
    // }

    const [state, dispatch] = useReducer(arithmeticOperation, { count: 10
});
    return (
        <div style={{ padding: "50px 60px" }}>Method4
            <div>
                <h1>useReducer()</h1>
                <p>It uses to complex or high valoume application  </p>
                <p>else if & switch case condition  </p>

                <div>
                    {/* <p>Counter add : <strong style={{ fontSize: "36px"
}}> {count} </strong></p>

                    <div>
                        <button onClick={incCount}>Inc count</button>
                        <button onClick={decCount}>dec count</button>
                    </div> */}


                    <div>
                        <p>counter app in useReducer:  <strong style={{
fontSize: "27px " }}>  {state.count} </strong> </p>
                        <div>
                            <button onClick={() => { dispatch({ type:
"add" }) }} > Inc</button>
                            <button onClick={() => { dispatch({ type:
"sub" }) }} > dec</button>
                            <button onClick={() => { dispatch({ type: 3 })
}} > multi</button>
```

```
                            <button onClick={() => { dispatch({ type: 4 })
}} > div</button>
                            <button onClick={() => { dispatch({ type: 5 })
}} > mode</button>
                            <button onClick={() => { dispatch({ type: 6 })
}} > inc value</button>
                            <button onClick={() => { dispatch({ type:
"reset" }) }} > reset</button>
                        </div>
                    </div>
                </div>

            </div>
        </div >
    )
}
```

```
import React from 'react'
import { useState, useCallback, useRef, useEffect } from 'react';
// import { useCallback } from 'react';
// import { useRef } from 'react';
// import
export default function Method5() {
    const [count, setCount] = useState(0);
    const myOldcount = useRef()
    // function incCount() {
    //     setCount(count + 1)
    // }

    const addCount = useCallback(() => {
        setCount(count + 1)
    })

    useEffect(() => {
        myOldcount.current = count;
    })
```

```
    function decCount() {
        setCount(count - 1)
    }

    return (
        <div>Method5

            <h2>common Hooks </h2>
            <div>
                <p>Counter add : <strong style={{ fontSize: "36px" }}>
{count} </strong></p>
                <p>Prious count : {myOldcount.current}  </p>

                <div>
                    <button onClick={addCount}>Inc count</button>
                    <button onClick={decCount}>dec count</button>
                </div>
            </div>

        </div>
    )
}
```

```
import React from 'react'
import { useState } from 'react'
export default function Important() {
    const [toggle, setToggle] = useState(false)
    return (
        <div style={{ textAlign: "center" }}>
            <h1>Important Topics</h1>

            <div style={{
                padding: "100px 60px",
                margin: "20px 0",
                backgroundColor: toggle ? "black" : "green",
                color: toggle ? "#FFF" : "red"
            }}>
```

```
                <h3>hello world</h3>
                <p>hi hello world team</p>

                <button onClick={() => { setToggle(!toggle) }}>Change
Bg</button>
            </div>
        </div>
    )
}
```

```
import React, { useState } from 'react';
import './Form.css';

export default function Form() {

    const [clientinfo, setClientinfo] = useState({
        fname: "",
        femail: "",
        fpassword: "",
        cpassword: ""
    });

    const [err, setErr] = useState({});

    function formData(event) {
        setClientinfo({
            ...clientinfo,
            [event.target.name]: event.target.value
        });

        // Clear the error for the current field
        setErr({
            ...err,
            [event.target.name]: ""
        });
    }

    const formValidate = () => {
```

```
    let tempErr = {};

    if (!clientinfo.fname.trim()) {
        tempErr.fname = "Please fill name data!";
    }

    if (!clientinfo.femail.trim()) {
        tempErr.femail = "Please fill the email!";
    }

    if (!clientinfo.fpassword) {
        tempErr.fpassword = "Please fill the password!";
    }

    if (!clientinfo.cpassword) {
        tempErr.cpassword = "Please fill the confirm password!";
    }

    if (
        clientinfo.fpassword &&
        clientinfo.cpassword &&
        clientinfo.fpassword !== clientinfo.cpassword
    ) {
        tempErr.cpassword = "Passwords do not match!";
    }

    setErr(tempErr);

    return Object.keys(tempErr).length === 0;
};

const formSubmit = (event) => {
    event.preventDefault();

    if (formValidate()) {
        console.log(clientinfo);

        alert("Data Saved Successfully!");
```

```
            setClientinfo({
                fname: "",
                femail: "",
                fpassword: "",
                cpassword: ""
            });

            setErr({});
        }
    };

    return (
        <div>
            <h1>Form</h1>

            <div className="form-container">

                <form onSubmit={formSubmit}>

                    <div className="form-content">
                        <label htmlFor="fname">Name</label>
                        <br />

                        <input
                            type="text"
                            id="fname"
                            name="fname"
                            placeholder="Enter your name"
                            onChange={formData}
                            value={clientinfo.fname}
                        />

                        {err.fname && <p>{err.fname}</p>}
                    </div>


                    <div className="form-content">
                        <label htmlFor="femail">Email</label>
                        <br />
```

```
                        <input
                            type="email"
                            id="femail"
                            name="femail"
                            placeholder="Enter your Email"
                            onChange={formData}
                            value={clientinfo.femail}
                        />

                        {err.femail && <p>{err.femail}</p>}
                    </div>


                    <div className="form-content">
                        <label htmlFor="fpassword">Password</label>
                        <br />

                        <input
                            type="password"
                            id="fpassword"
                            name="fpassword"
                            placeholder="Enter your password"
                            onChange={formData}
                            value={clientinfo.fpassword}
                        />

                        {err.fpassword && <p>{err.fpassword}</p>}
                    </div>


                    <div className="form-content">
                        <label htmlFor="cpassword">Confirm
Password</label>
                        <br />

                        <input
                            type="password"
                            id="cpassword"
                            name="cpassword"
                            placeholder="Enter your confirm password"
```

```
                            onChange={formData}
                            value={clientinfo.cpassword}
                        />

                        {err.cpassword && <p>{err.cpassword}</p>}
                    </div>


                    <button type="submit">
                        Sign Up
                    </button>

                </form>

            </div>
        </div>
    );
}
```

```
.form-container {
    background-color: blueviolet;
    width: 100%;
    height: 100%;
    padding: 25px;
    text-align: left;
    display: flex;
    justify-content: center;
    align-items: center;
}

.form-container form {
    width: 600px;
    background-color: rgb(167, 88, 241);
    padding: 40px 30px;
    margin: 20px;
}

.form-content {
    color: black;
    font-size: 20px;
```

```
    text-align: left;
}

.form-content input {
    padding: 10px 15px;
    margin: 10px 0;
    width: 100%;
    outline: none;

}

.form-container button {
    padding: 15px 40px;
    margin: 5px 0;
    border: none;

}

.form-content p {
    color: red;
    font-size: 17px;
    padding: 10px 0;
}
```

```
import React from 'react'
import { useState } from 'react'
import Mainpage from './Pages/Mainpage'
import Navbar from './Header/Header'
import Menubar from './Header/Menubar'

import { BrowserRouter } from 'react-router-dom'
import Form from './Form/Form'
export default function Important() {
    const [toggle, setToggle] = useState(false)
    return (
        <div style={{ textAlign: "center" }}>
            <h1>Important Topics</h1>

            <div style={{
                padding: "100px 60px",
```

```
                margin: "20px 0",
                backgroundColor: toggle ? "black" : "green",
                color: toggle ? "#FFF" : "red"
            }}>
                <h3>hello world</h3>
                <p>hi hello world team</p>

                <button onClick={() => { setToggle(!toggle) }}>Change
Bg</button>
            </div>



            <div>
                <BrowserRouter>
                    <Navbar />
                    <Menubar />
                    <Mainpage />
                </BrowserRouter>
            </div>

            <div>
                <Form />
            </div>
        </div>
    )
}
```

```
import React from 'react'
import Home from './Home'
import About from './About'
import Service from './Service'
import Contact from './Contact'

import { Routes, Route } from 'react-router-dom'

export default function Mainpage() {
    return (
        <div>Mainpage
```

```
            <Routes>
                <Route path='/home' element={<Home />} />
                <Route path='/about' element={<About />} />
                <Route path='/service' element={<Service />} />
                <Route path='/contact' element={<Contact />} />

            </Routes>
        </div>
    )
}
```

```
import React from "react";
import './Header.css';
import { Link } from "react-router-dom";
export default function Navbar() {
    return (
        <header>
            <ul>
                <li><Link to={'/home'}>Home</Link></li>
                <li><Link to={"/about"}>About</Link></li>
                <li><Link to={"/service"}>Service</Link></li>
                <li><Link to={"/contact"}>Contact</Link></li>
            </ul>
        </header>
    )
}
```

```
import Container from 'react-bootstrap/Container';
import Nav from 'react-bootstrap/Nav';
import Navbar from 'react-bootstrap/Navbar';
import NavDropdown from 'react-bootstrap/NavDropdown';
import { Link } from 'react-router-dom';

function Menubar() {
    return (
        <Navbar expand="lg" className="bg-body-tertiary">
            <Container>
                <Navbar.Brand href="#home">React-Bootstrap</Navbar.Brand>
```

```
                <Navbar.Toggle aria-controls="basic-navbar-nav" />
                <Navbar.Collapse id="basic-navbar-nav">
                    <Nav className="me-auto">
                        <Nav.Link as={Link} to={'/home'}>Home</Nav.Link>
                        <Nav.Link as={Link} to={'/about'}>About</Nav.Link>
                        <Nav.Link as={Link}
to={'/service'}>Service</Nav.Link>
                        <Nav.Link as={Link}
to={'/contact'}>Contact</Nav.Link>
                        <NavDropdown title="Dropdown"
id="basic-nav-dropdown">
                            <NavDropdown.Item
href="#action/3.1">Action</NavDropdown.Item>
                            <NavDropdown.Item href="#action/3.2">
                                Another action
                            </NavDropdown.Item>
                            <NavDropdown.Item
href="#action/3.3">Something</NavDropdown.Item>
                            <NavDropdown.Divider />
                            <NavDropdown.Item href="#action/3.4">
                                Separated link
                            </NavDropdown.Item>
                        </NavDropdown>
                    </Nav>
                </Navbar.Collapse>
            </Container>
        </Navbar>
    );
}

export default Menubar;
```

```
import React from 'react'
import Btn from '../../Components/Button/Btn'
import { useState } from 'react'
export default function Talktous() {
    const [usersdata, setUserData] = useState({
        uname: "",
        // umobile: BigInt,
        umobile: "",
        uemail: "",
        umessages: ""
    });

    function handleChange(userInfo) {
        setUserData({ ...usersdata, [userInfo.target.name]:
userInfo.target.value });
    }


    const formSubmit = (userInfo) => {
        userInfo.preventDefault();

        if (!usersdata.uname || !usersdata.umobile || !usersdata.uemail ||
!usersdata.umessages) {
            alert("Please fill the correct Data....");
        }
        else {
            console.log("Form filled..", usersdata);
            alert("Data Printed in console ,, thank you ...!!");
            setUserData({ uname: "", umobile: "", uemail: "", umessages:
"" })
        }
    }
    return (
        <section className='Formsections'>
            <div className="workcontent">
                <h1>Talk To Us</h1>
                <p>Aenean nec tempor metus. Maecenas ligula dolor, commodo
in imperdiet interdum, vehicula ut ex. Donec ante diam.</p>
            </div>
            <div className="formpart">
```

```
                <div className="formleft">
                    <h2>Keep in touch</h2>
                    <p>110-220 Quisque diam odio, maximus ac consectetur
eu, 10260
                        auctor non lorem</p>
                    <p>You are NOT allowed to re-distribute Softy Pinko
template on any template collection websites. Thank you.</p>
                </div>
                <div className="formright">
                    <form onSubmit={formSubmit}>
                        <div className="inputs">
                            <label htmlFor="cname">Client Name</label>
                            <div className="fields">
                                <input type="text" name='uname'
placeholder='Enter your Fullname'
                                    value={usersdata.uname}
                                    onChange={handleChange} />
                                {/* <p>please fill data</p> */}
                            </div>
                        </div>

                        <div className="inputs">
                            <label htmlFor="cname">Client Mobile</label>
                            <div className="fields">
                                <input type="tel" name='umobile'
placeholder='Enter your Mobile Number'
                                    value={usersdata.umobile}
                                    onChange={handleChange} />
                                {/* <p>please fill data</p> */}
                            </div>
                        </div>

                        <div className="inputs">
                            <label htmlFor="cname">Client Email</label>
                            <div className="fields">
                                <input type="email" name='uemail'
placeholder='Enter your Email'
                                    value={usersdata.uemail}
                                    onChange={handleChange} />
                                {/* <p>please fill data</p> */}
```

```
                        </div>
                    </div>

                    <div className="inputs">
                        <label htmlFor="cname">Client Messages</label>
                        <div className="fields">
                            <textarea name="umessages"
placeholder='Enter your Messages...!!!'
                                value={usersdata.umessages}
                                onChange={handleChange} ></textarea>
                            {/* <p>please fill data</p> */}
                        </div>
                    </div>

                    <div>
                        <Btn BtnName={"Send Messages....!!"} />
                    </div>
                </form>
            </div>
        </div>
    </section>
  )
}
```

```
import React from 'react'
import Btn from '../../Components/Button/Btn'
import { useState } from 'react'
export default function Talktous() {
    const [usersdata, setUserData] = useState({
        uname: "",
        // umobile: BigInt,
        umobile: "",
        uemail: "",
        umessages: ""
    });

    const [error, setError] = useState();
```

```
    function handleChange(userInfo) {
        setUserData({ ...usersdata, [userInfo.target.name]:
userInfo.target.value });
        setError({ ...error, [userInfo.target.name]: "" });
    }


    const formSubmit = (userInfo) => {
        userInfo.preventDefault();

        if (!usersdata.uname || !usersdata.umobile || !usersdata.uemail ||
!usersdata.umessages) {
            // alert("Please fill the correct Data....");
            setError("Please fill the correct Data....");
        }
        else {
            console.log("Form filled..", usersdata);
            alert("Data Printed in console ,, thank you ...!!");
            setUserData({ uname: "", umobile: "", uemail: "", umessages:
"" })
        }
    }
    return (
        <section className='Formsections'>
            <div className="workcontent">
                <h1>Talk To Us</h1>
                <p>Aenean nec tempor metus. Maecenas ligula dolor, commodo
in imperdiet interdum, vehicula ut ex. Donec ante diam.</p>
            </div>
            <div className="formpart">
                <div className="formleft">
                    <h2>Keep in touch</h2>
                    <p>110-220 Quisque diam odio, maximus ac consectetur
eu, 10260
                        auctor non lorem</p>
                    <p>You are NOT allowed to re-distribute Softy Pinko
template on any template collection websites. Thank you.</p>
                </div>
                <div className="formright">
                    <form onSubmit={formSubmit}>
```

```
                        <div className="inputs">
                            <label htmlFor="cname">Client Name</label>
                            <div className="fields">
                                <input type="text" name='uname'
placeholder='Enter your Fullname'
                                    value={usersdata.uname}
                                    onChange={handleChange} />
                                {error && <p> {error} </p>}
                            </div>
                        </div>

                        <div className="inputs">
                            <label htmlFor="cname">Client Mobile</label>
                            <div className="fields">
                                <input type="tel" name='umobile'
placeholder='Enter your Mobile Number'
                                    value={usersdata.umobile}
                                    onChange={handleChange} />
                                {error && <p> {error} </p>}
                            </div>
                        </div>

                        <div className="inputs">
                            <label htmlFor="cname">Client Email</label>
                            <div className="fields">
                                <input type="email" name='uemail'
placeholder='Enter your Email'
                                    value={usersdata.uemail}
                                    onChange={handleChange} />
                                {error && <p> {error} </p>}
                            </div>
                        </div>

                        <div className="inputs">
                            <label htmlFor="cname">Client Messages</label>
                            <div className="fields">
                                <textarea name="umessages"
placeholder='Enter your Messages...!!!'
                                    value={usersdata.umessages}
                                    onChange={handleChange} ></textarea>
```

```
                                {error && <p> {error} </p>}
                            </div>
                        </div>

                        <div>
                            <Btn BtnName={"Send Messages....!!"} />
                        </div>
                    </form>
                </div>
            </div>
        </section>
    )
}
```

```
import React from 'react'
import Btn from '../../Components/Button/Btn'
import { useState } from 'react'
export default function Talktous() {
    const [usersdata, setUserData] = useState({
        uname: "",
        // umobile: BigInt,
        umobile: "",
        uemail: "",
        umessages: ""
    });

    const [error, setError] = useState({});

    function handleChange(userInfo) {
        setUserData({ ...usersdata, [userInfo.target.name]:
userInfo.target.value });
        setError({ ...error, [userInfo.target.name]: "" });
    }

    const validateForm = () => {
        let tempError = {};

        if (!usersdata.uname.trim()) {
```

```
            tempError.uname = "Please Fill the data";
        }
        else if ((usersdata.uname.trim().length < 3)) {
            tempError.uname = "please fill the correct format  name Data "
        }


        const mobileRule = /^[6-9][0-9]{9}$/;
        if (!usersdata.umobile.trim()) {
            tempError.umobile = "please fill the Mobile Data"
        }
        else if (!mobileRule.test(usersdata.umobile)) {
            tempError.umobile = "please fill correct mobile number"
        }

        const emailRule = /[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,4}$/
        if (!usersdata.uemail.trim()) {
            tempError.uemail = "please fill the data"
        }
        else if (!(emailRule.test(usersdata.uemail))) {
            tempError.uemail = "please fill the correct email data"
        }

        if (!usersdata.umessages.trim()) {
            tempError.umessages = "please fill the data "
        }

        setError(tempError);
        return Object.keys(tempError).length === 0;
    }


    const formSubmit = (userInfo) => {
        userInfo.preventDefault();

        if (validateForm()) {
            console.log("Form filled..", usersdata);
            alert("Data Printed in console ,, thank you ...!!");
            setUserData({ uname: "", umobile: "", uemail: "", umessages:
"" })
```

```
        }

        // if (!usersdata.uname || !usersdata.umobile || !usersdata.uemail
|| !usersdata.umessages) {
        //     // alert("Please fill the correct Data....");
        //     setError("Please fill the correct Data....");
        // }
        // else {
        //     console.log("Form filled..", usersdata);
        //     alert("Data Printed in console ,, thank you ...!!");
        //     setUserData({ uname: "", umobile: "", uemail: "",
umessages: "" })
        // }
    }
    return (
        <section className='Formsections'>
            <div className="workcontent">
                <h1>Talk To Us</h1>
                <p>Aenean nec tempor metus. Maecenas ligula dolor, commodo
in imperdiet interdum, vehicula ut ex. Donec ante diam.</p>
            </div>
            <div className="formpart">
                <div className="formleft">
                    <h2>Keep in touch</h2>
                    <p>110-220 Quisque diam odio, maximus ac consectetur
eu, 10260
                        auctor non lorem</p>
                    <p>You are NOT allowed to re-distribute Softy Pinko
template on any template collection websites. Thank you.</p>
                </div>
                <div className="formright">
                    <form onSubmit={formSubmit}>
                        <div className="inputs">
                            <label htmlFor="cname">Client Name</label>
                            <div className="fields">
                                <input type="text" name='uname'
placeholder='Enter your Fullname'
                                    value={usersdata.uname}
                                    onChange={handleChange} />
                                {error.uname && <p> {error.uname} </p>}
```

```
                            </div>
                        </div>

                        <div className="inputs">
                            <label htmlFor="cname">Client Mobile</label>
                            <div className="fields">
                                <input type="tel" name='umobile'
placeholder='Enter your Mobile Number'
                                    value={usersdata.umobile}
                                    onChange={handleChange} />
                                {error.umobile && <p> {error.umobile}
</p>}
                            </div>
                        </div>

                        <div className="inputs">
                            <label htmlFor="cname">Client Email</label>
                            <div className="fields">
                                <input type="email" name='uemail'
placeholder='Enter your Email'
                                    value={usersdata.uemail}
                                    onChange={handleChange} />
                                {error.uemail && <p> {error.uemail} </p>}
                            </div>
                        </div>

                        <div className="inputs">
                            <label htmlFor="cname">Client Messages</label>
                            <div className="fields">
                                <textarea name="umessages"
placeholder='Enter your Messages...!!!'
                                    value={usersdata.umessages}
                                    onChange={handleChange} ></textarea>
                                {error.umessages && <p> {error.umessages}
</p>}
                            </div>
                        </div>

                        <div>
                            <Btn BtnName={"Send Messages....!!"} />
```

```
                        </div>
                    </form>
                </div>
            </div>
        </section>
    )
}
```

Mysql

Data Base 👍

Data ⇒ Raw Facts ⇒ 0,1 ⇒ Collection information

Written statements

Oracle ⇒ MNC

https://www.mysql.com/

Archi ⇒ MvC

DB ⇒ Archi ⇒ RDBMS ⇒ ORDBMS

SQL Overall Concepts

1. **Data** =>Raw facts and figures that have no meaning until processed. ⇒ **{0,1}**

2. **Database (DB)** => A collection of logically organized data that can be easily accessed, managed, and updated.To store large amounts of information efficiently.

3. **Database Management (DBM)**=>The process of organizing, storing, retrieving, and securing data in a database.Planning, maintenance, data integrity, access control, and backups.It is more of a general concept or process, not a software tool.

4. **Database Management System (DBMS)** =>Software that handles the storage, retrieval, and updating of data in a database.
   Functions:
   - CRUD operations (Create, Read, Update, Delete)
   - Data security
   - Concurrency control
   - Backup & recovery

   Examples: Microsoft Access, IBM DB2, SQLite.

5. **Relational Database Management System (RDBMS)** => A type of DBMS that stores data in the form of tables (relations) and supports relationships between data.
   Based on: **E.F. Codd's relational model**.
   Key Features:

- Data stored in rows and columns (tables)
- Primary & Foreign Keys
- Normalization to avoid redundancy
- Uses SQL (Structured Query Language)
  Examples: MySQL, PostgreSQL, Oracle DB, Microsoft SQL Server.

6. **Object-Relational Database Management System (ORDBMS)**.
   Key Features:
   - A hybrid system combining features of RDBMS and Object-Oriented Databases (OODBMS)
   - Supports tables (like RDBMS) and objects (like OOP: classes, inheritance, etc.)
   - Better support for complex data types (images, audio, video, spatial data)

Examples: PostgreSQL (with object extensions), Oracle (with object features), Informix.

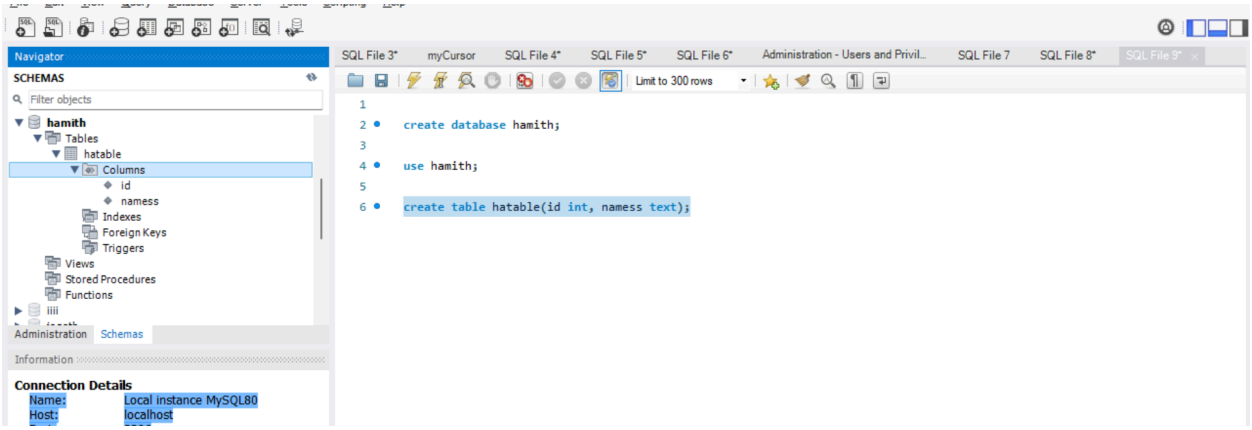
SQL Category

1. Data Query Language (DQL) - used to query data

2. Data Manipulation Language (DML) – used to create/modify(update)/destroy(delete) data

3. Data Definition Language (DDL) – used to define database schema (create/drop ⇒ database,table)

4. Data Control Language (DCL) – used for security and access control



DDL ⇒ create Database somthing;,
create table wrong(id int, names text);
drop table wrong;
drop database somthing;
Alter drop

DML ⇒ insert/update/delete
insert into trainees
value(1,"vgr",12345,'vgr@gmail.com');

update trainees set tname="Ganapathiraja" where tid=1;
delete from trainees where tid=1;

DQL⇒ Select * from wrong;

Most Important SQL Statements

1. SELECT - extracts data from a database (DQL)
2. UPDATE - updates data in a database (DML)
3. DELETE - deletes data from a database (DML)

4. INSERT - inserts new data into a database (DML)
5. CREATE DATABASE - creates a new database (DDL)
6. CREATE TABLE - creates a new table (DDL)
7. DROP TABLE - deletes a table (DDL)
8. Drop database - delete a exiting database (DDL)

Common Rules in Sql Operation

1. SQL keywords and table/column names are NOT case sensitive: ‘select’ and ‘SELECT’ are the same
2. The values stored in a table can be case-sensitive – depending on configuration
3. Usually single quotes (‘’) or double quotes (“”) don’t matter, but could be configured otherwise
4. Semicolon ‘;’ is the standard way to separate SQL statements. It can be required in some DBMS to end each statement with a ‘;’ even after a single statement
5. Comments can be used to make SQL more readable. Usually ‘--’ for single line comment, and ‘/*’ and ’*/’ for multiline comments. Add ‘--’ at the beginning to indicate a comment line
6. Standard is NOT STANDARD – none of SQL standard is fully implemented by all vendors. Pay attention to the differences that each vendor’s implementation have from the SQL ‘standard’

Common Query Keywords

1. SELECT **2. * (universal)**	**11. AND** **12. OR**	**21. FULL JOIN** **22. SELF JOIN**

3. FROM 4. TABLE 5. WHERE 6. ORDER BY 7. GROUP BY 8. ASC 9. DESC 10. LIKE	13. NOT 14. LIMIT 15. DISTINCT 16. AS 17. INNER JOIN 18. LEFT JOIN 19. RIGHT JOIN 20. CROSS JOIN	23. UNION & UNION ALL 24. ON 25. != 26. % 27. INSERT 28. INTO 29. VALUES 30. CREATE

31. UPDATE 32. DELETE 33. DROP 34. COUNT() 35. MIN() 36. MAX() 37. AVG() 38. SUM() 39. REPLACE() 40. NOT NULL	41. UNIQUE 42. PRIMARY 43. FOREIGN 44. CHECK 45. DEFAULT 46. length() 47. upper() 48. lower() 49. substring() 50. trim()

Data Definition Language (DDL)	**Data Manipulation Language (DML)**
–Create/alter/delete tables and their attributes	–Insert/delete/modify tuples in tables

Data Definition Language (DDL) commands and syntax:

CREATE DATABASE	CREATE DATABASE database_name;
DROP DATABASE	DROP DATABASE database_name;
USE / SELECT DATABASE	USE database_name;
SHOW DATABASES	SHOW DATABASES; Show tables;
COPY DATABASE	CREATE TABLE NewDB.TableName AS SELECT * FROM OldDB.TableName;
Database Export	mysqldump -u username -p database_name > filename.sql
Database Import	mysql -u username -p new_database_name < filename.sql

Data Definition Language (DDL) commands and syntax:

Create Table	CREATE TABLE table_name (column1 datatype, column2 datatype, ...);
Show Tables	SHOW TABLES;
Alter Table	ALTER TABLE table_name ADD column_name datatype; ALTER TABLE table_name DROP COLUMN column_name; ALTER TABLE table_name MODIFY column column_name new_datatype; ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
Rename Table	RENAME TABLE old_table_name TO new_table_name;
Clone Table	**a. Only Structure:** CREATE TABLE NewTable AS SELECT * FROM OldTable WHERE 1=0 or 1=1; **b. Structure + Data:** CREATE TABLE NewTable AS SELECT * FROM OldTable;
Truncate Table	TRUNCATE TABLE table_name;
Temporary Tables	CREATE TEMPORARY TABLE temp_table_name (column1 datatype, ...);
Describe Table	DESCRIBE table_name; or DESC table_name;
Show Columns	SHOW COLUMNS FROM table_name;
Drop Table	DROP TABLE table_name;

Queries (General)

A **query** is any SQL statement used to **retrieve, modify, insert, or delete** data from a database.Examples:

- Retrieve data → SELECT
- Insert data → INSERT
- Update data → UPDATE
- Delete data → DELETE

Queries

INSERT Query	**INSERT INTO table_name (column1, column2, ...)**

Adds new rows to a table.	**VALUES (value1, value2, ...);**
SELECT Query **Retrieves data from a table**	**SELECT column1, column2 FROM table_name;** -- OR to select all columns: **SELECT * FROM table_name;**
UPDATE Query **Modifies existing data in a table.**	**UPDATE table_name** **SET column1 = value1, column2 = value2** **WHERE condition;**
DELETE Query **Deletes records from a table.**	**DELETE FROM table_name WHERE condition;**

Constraints

Constraints are rules applied to table columns to enforce data integrity.

NOT NULL	Prevents null values
UNIQUE	Ensures all values are different
PRIMARY KEY	Uniquely identifies a row + not null
FOREIGN KEY	Creates a relationship between tables
CHECK	Ensures values meet a condition
DEFAULT	Sets a default value if none is provided

MYSQL Operators & Clauses

WHERE Clause	**SELECT * FROM table_name WHERE condition;**

Filters records based on a condition	**SELECT * FROM Employees WHERE Age > 30;**		
LIMIT Clause **Limits the number of rows returned.**	**SELECT * FROM table_name LIMIT number;** **SELECT * FROM Employees LIMIT 5;**		
DISTINCT Clause **Removes duplicate values.**	**SELECT DISTINCT column1 FROM table_name;** **SELECT DISTINCT Department FROM Employees;**		
ORDER BY Clause **Sorts the result set in ascending or descending order.**	**SELECT * FROM table_name ORDER BY column1 [ASC|DESC];** **ELECT * FROM Employees ORDER BY Age DESC;**		
GROUP BY Clause **Groups rows that have the same values**	**SELECT column1, COUNT(*) FROM table_name GROUP BY column1;** **SELECT Department, COUNT(*) FROM Employees GROUP BY Department;**		
HAVING Clause **Filters the result of a GROUP BY clause (like WHERE but for aggregates).**	**SELECT column, COUNT(*)** **FROM table_name** **GROUP BY column** **HAVING COUNT(*) > value;** **SELECT Department, COUNT(*)** **FROM Employees** **GROUP BY Department** **HAVING COUNT(*) > 2;**		
BETWEEN **Returns values between a range (inclusive).**	**SELECT * FROM Employees WHERE Age BETWEEN 25 AND 35;**		
UNION **Combines results of two queries (removes duplicates).**	**SELECT column1 FROM table1** **UNION** **SELECT column1 FROM table2;**		
MINUS **Not use**	**SELECT Name FROM TableA** **LEFT JOIN TableB ON TableA.Name = TableB.Name** **WHERE TableB.Name IS NULL;**		
INTERSECT **Not use**	**SELECT Name FROM TableA** **WHERE Name IN (SELECT Name FROM TableB);**		
	Operator	**Description**	**Example**
	AND	**Both conditions must**	**WHERE Age > 25 AND**

Logical Operators		**be true**	**Department = 'HR'**
	OR	**At least one condition must be true**	**WHERE Age > 25 OR Department = 'HR'**
	NOT	**Reverses the condition**	**WHERE NOT Age < 30**
	LIKE	**Pattern matching using % or _**	**WHERE Name LIKE 'A%'**
	IN	**Match any value in a list**	**WHERE Department IN ('HR', 'IT')**
	EXISTS	**Checks if subquery returns any result**	**WHERE EXISTS (SELECT * FROM ...)**
	ANY	**True if any value of subquery meets the condition**	**Age > ANY (SELECT Age FROM ...)**
	NOT EQUAL	**Checks inequality**	**Age <> 30 or Age != 30**

Aggregate Functions

These functions perform **calculations on multiple rows** and return a single value.

Functions	Description	example
COUNT()	Counts the number of rows	SELECT COUNT(*) FROM Orders;
SUM()	Adds all values in a column	SELECT SUM(Amount) FROM Sales;
AVG()	Calculates average	SELECT AVG(Price) FROM Products;
MIN()	Returns the smallest value	SELECT MIN(Salary) FROM Employees;
MAX()	Returns the largest value	SELECT MAX(Salary) FROM Employees;

Often used with=> GROUP BY.

Math Functions

These functions perform mathematical operations on numeric data.

Functions	Description	example
ABS()	Absolute value	ABS(-5) → 5
ROUND()	Rounds number to given decimals	ROUND(123.456, 2) → 123.46
FLOOR()	Rounds down	FLOOR(2.9) → 2
CEIL()	Rounds up	CEIL(2.1) → 3
POWER(x, y)	Raises x to power y	POWER(2, 3) → 8
MOD()	Returns remainder	MOD(10, 3) → 1
SQRT()	Square root	SQRT(25) → 5

String Functions

These functions help you manipulate and format string/text data.

Functions	Description	example
LENGTH()	Returns length of a string	LENGTH('Hello') → 5
CONCAT()	Joins strings together	CONCAT('SQL', ' ', 'Rocks') → SQL Rocks
UPPER()	Converts string to uppercase	UPPER('hello') → HELLO
LOWER()	Converts string to lowercase	LOWER('HELLO') → hello
SUBSTRING()	Extracts part of a string	SUBSTRING('Database', 1, 4) → Data
REPLACE()	Replaces part of a string	REPLACE('cat', 'c', 'b') → bat
TRIM()	Removes whitespace	TRIM(' SQL ') → SQL
`LEFT()` and `RIGHT()`	Returns a specified number of characters from the left or right	`SELECT LEFT('SQLQuery', 3);  -- Output: SQL` `SELECT RIGHT('SQLQuery', 5);  -- Output: Query`
`INSTR()` or `POSITION()`	Returns the position of a substring in a string.	`SELECT INSTR('database','base') ;  -- Output: 5` `SELECT POSITION('base' IN 'database');-- Output: 5`
`ASCII()` and `CHAR()`	`ASCII()`: Returns ASCII value of the first character. `CHAR()`: Converts ASCII code to character.	`SELECT ASCII('A');  -- Output: 65` `SELECT CHAR(65);  -- Output: A`

Date and Time Functions

These functions allow you to work with date and time values

Functions	Description	example
NOW()	Returns current date and time	SELECT NOW(); → 2025-04-12 12:34:56
CURDATE()	Returns current date	SELECT CURDATE(); → 2025-04-12
CURTIME()	Returns current time	SELECT CURTIME(); → 12:34:56
DATEDIFF()	Returns days between two dates	DATEDIFF('2025-04-15', '2025-04-10') → 5
DATE_ADD()	Adds days or intervals to a date	DATE_ADD('2025-04-10', INTERVAL 5 DAY)
DATE_FORMAT()	Formats a date/time to a specific pattern	DATE_FORMAT(NOW(), '%Y/%m/%d')

SQL Subquery

A Subquery is a query within another SQL query and embedded within the WHERE clause.

A **SQL Subquery** (also called an **inner query** or **nested query**) is a query inside another query. Subqueries are used to:

- Filter data based on results from another query
- Calculate intermediate result
- Replace complex joins in some scenario

Type	Description
Single-row subquery	Returns exactly one row
Multi-row subquery	Returns multiple rows
Correlated subquery	Depends on outer query row-by-row
Nested subquery	A subquery inside another subquery
Scalar subquery	Returns a single value (like a column)

Single subquery ⇒Returns one value. Use with =, <, >, etc.

SELECT * FROM users WHERE score = (SELECT MAX(score) FROM users);

Multi-Row Subquery ⇒Use with `IN`, `ANY`, `ALL`.

```
SELECT * FROM users WHERE name IN ( SELECT name FROM users WHERE
score>=90);
```

Correlated Subquery⇒Inner query refers to outer query row-by-row.

```
SELECT name, score FROM users u1 WHERE score > (
   SELECT AVG(score) FROM users u2 WHERE u2.name = u1.name
);
```

Subquery in FROM (Inline View)

```
SELECT avg_score FROM ( SELECT AVG(score) AS avg_score FROM users
) AS avg_result;
```

Subquery in SELECT (Scalar Subquery)

```
SELECT name,   (SELECT AVG(score) FROM users) AS avg_score
FROM users;
```

SQL Ranking

RANK() & DENSE_RANK()	**Used to assign rankings to rows in a result set (usually with ORDER BY).**

RANK() • **Assigns a rank to rows within a partition.** • **Ties (duplicate values) get the same rank, and the next rank is skipped.** **Same values → same rank** **Skips next rank**	**Assigns a rank with gaps for ties.** SELECT Name, Salary, RANK() OVER (ORDER BY Salary DESC) AS Rank FROM Employees;
DENSE_RANK() • **Similar to `RANK()`, but no gaps in ranking when there are ties.** **Same values → same rank** **NO skipping**	**Assigns continuous ranks (no gaps).** SELECT Name, Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRank FROM Employees;

`Percent_RANK()`	**Assigns continuous ranks (percentages vise).**
	`SELECT subject, s_name, mark, percent_rank()`
	`OVER ( partition by subject order by mark )`
	`AS 'percent_rank' FROM student_marks;`

`ROW_NUMBER()` **Assigns a unique sequential number to each row, even if the values are tied.** **`Always`**	Assigns a unique sequential number to each row, even if the values are tied. `SELECT subject, s_name, mark, ROW_NUMBER()`

unique rank (no duplicates)	`OVER ( partition by subject order by mark desc )` `AS 'rownumber' FROM student_marks;`

NTILE(n)
Divides the result set into n approximately equal parts (buckets).
	Divides the result set into n approximately equal parts (buckets)
	`SELECT subject, s_name, mark, NTILE(4)`
	`OVER ( partition by subject order by mark desc )`
	`AS 'ntitle' FROM student_marks;`

---

1. RANK()

- **Assigns a rank to rows within a partition.**
- **Ties (duplicate values) get the same rank, and the next rank is skipped.**

```
SELECT subject, s_name, mark, rank()
OVER ( partition by subject order by mark desc )
AS 'rank' FROM student_marks;
```

---

2. DENSE_RANK()

- **Similar to RANK(), but no gaps in ranking when there are ties.**

```
SELECT subject, s_name, mark, dense_rank()
OVER ( partition by subject order by mark desc )
AS 'dense_rank' FROM student_marks;
```

3. Percent_RANK()

```
SELECT subject, s_name, mark, percent_rank()
OVER ( partition by subject order by mark )
AS 'percent_rank' FROM student_marks;

SELECT subject, s_name, mark, rank()
OVER ( partition by subject order by mark )-1
AS 'rank-1', count(*) over (partition by subject)-1
AS 'total_rows-1', percent_rank()
OVER ( partition by subject order by mark ) AS 'percenr_rank'
FROM student_marks;
```

---

3. ROW_NUMBER()

- **Assigns a unique sequential number to each row, even if the values are tied.**

```
SELECT subject, s_name, mark, ROW_NUMBER()
OVER ( partition by subject order by mark desc )
AS 'rownumber' FROM student_marks;
```

---

4. NTILE(n)

- **Divides the result set into n approximately equal parts (buckets).**

```
SELECT subject, s_name, mark, NTILE(4)
OVER ( partition by subject order by mark desc )
AS 'ntitle' FROM student_marks;
```

Operator	Meaning	Example
IN	Matches any value in the list	deptid IN (...)
NOT IN	Excludes values in the list	deptid NOT IN (...)
ANY	Compares with at least one value	salary > ANY (...)
ALL	Compares with every value	salary > ALL (...)
EXISTS	Returns TRUE if the subquery returns any rows	EXISTS (...)
NOT EXISTS	Returns TRUE if the subquery returns no rows	NOT EXISTS (...)

Type	Returns	Operators Used	Example
Single-row Subquery	One Row	=, >, <, >=, <=	Compare one salary
Multi-row Subquery	Multiple Rows	IN, ANY, ALL	Multiple departments
Correlated Subquery	Executes for every outer row	EXISTS	Department average salary
Nested Subquery	Subquery inside another subquery	Any Operator	Multiple query levels
Scalar Subquery	Single Value	SELECT, WHERE	AVG(), MAX(), COUNT()

Function	Duplicate Rank	Gap in Ranking	Example
ROW_NUMBER()	❌ No	❌ No	1,2,3,4

RANK()	✅ Yes	✅ Yes	1,2,2,4
DENSE_RANK()	✅ Yes	❌ No	1,2,2,3

Concept	Easy Definition
Subquery	A query written inside another query.
Single-row Subquery	Returns one row.
Multi-row Subquery	Returns multiple rows.
Correlated Subquery	Executes once for each row of the outer query.
Nested Subquery	A subquery inside another subquery.
Scalar Subquery	Returns one value.
ROW_NUMBER()	Gives every row a unique number.
RANK()	Same values share the same rank; the next rank is skipped.
DENSE_RANK()	Same values share the same rank; no rank is skipped.

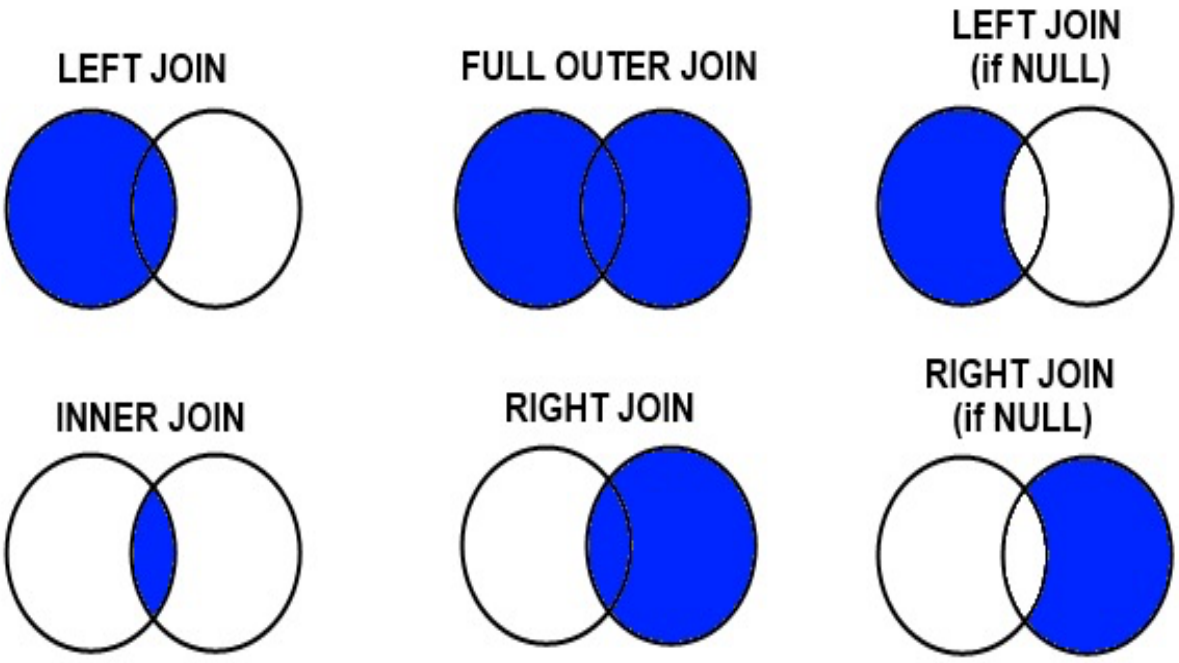
Joins

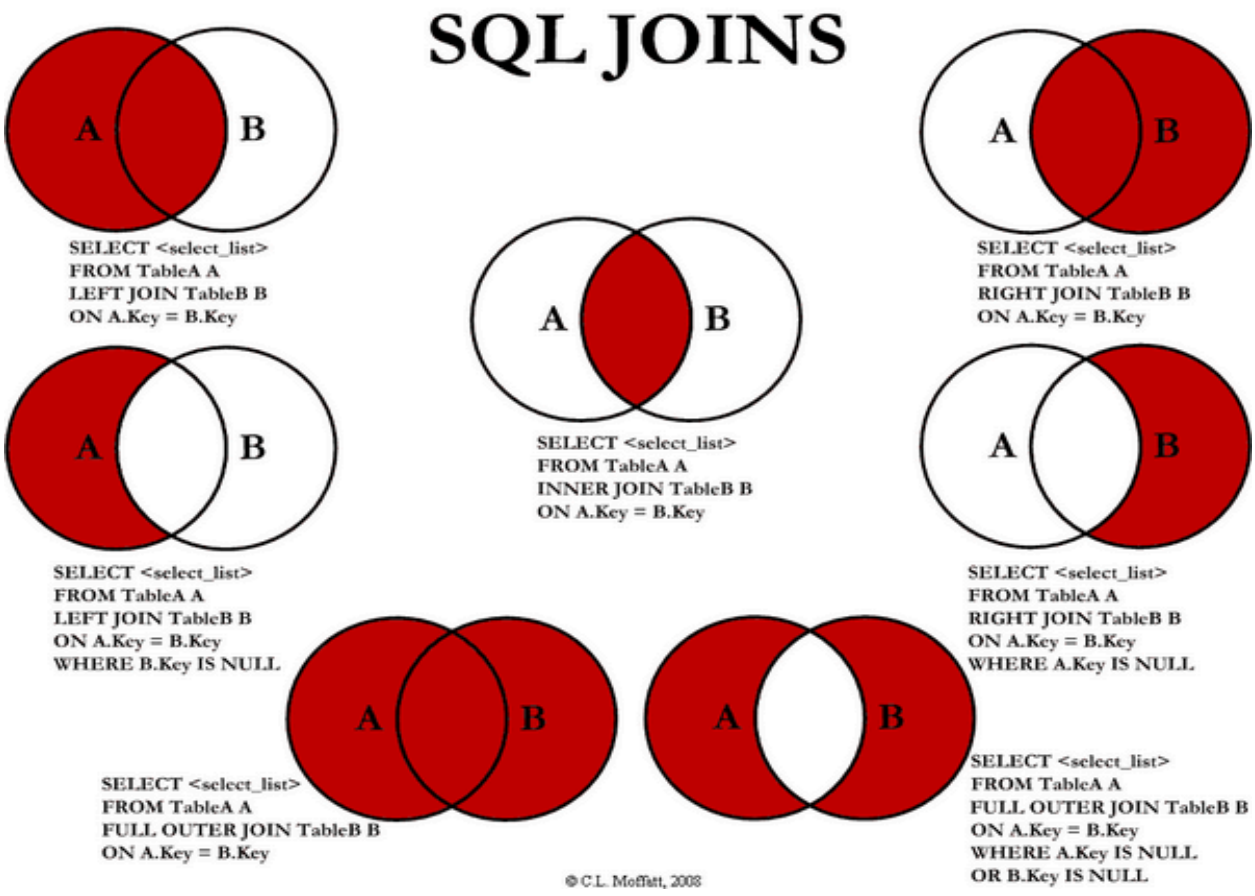
In SQL, JOIN is used to combine rows from two or more tables based on a related column between them (usually a foreign key).

Types of Joins in SQL

INNER JOIN	**Returns only matching rows from both tables**
LEFT JOIN	**Returns all rows from the left table, and matching rows from the right**
RIGHT JOIN	**Returns all rows from the right table, and matching rows from the left**
FULL OUTER JOIN	**Returns all rows from both tables, with NULLs where there's no match**
CROSS JOIN	**Returns all combinations of rows from both tables (Cartesian product)**
SELF JOIN	**A table joins with itself**

Below venn diagram shows the clear explanation about the joins concepts





Joins syntax

Join Title	syntex
INNER JOIN Keeps only the rows where both tables have a match.	You want only matching data from both tables. **SELECT A.column, B.column** **FROM TableA A** **INNER JOIN TableB B ON A.common_column = B.common_column;**
LEFT JOIN (or LEFT OUTER JOIN) Keeps all rows from the left table,	You want everything from the left table, even if there's no match in the right table. SELECT A.column, B.column FROM TableA A LEFT JOIN TableB B ON A.common_column = B.common_column;

RIGHT JOIN (or RIGHT OUTER JOIN) **Keeps all rows from the right table**	**You want everything from the right table, even if there's no match in the left.** **SELECT A.column, B.column** **FROM TableA A** **RIGHT JOIN TableB B ON A.common_column = B.common_column;**
CROSS JOIN **Returns every possible combination of rows from both tables**	**You need all combinations — for example, creating a calendar with all people and all dates.** **SELECT A.column, B.column** **FROM TableA A** **CROSS JOIN TableB B;**
FULL join	**SELECT A.column, B.column** **FROM TableA A** **RIGHT JOIN TableB B ON A.common_column = B.common_column** **UNION or UNION ALL** **SELECT A.column, B.column** **FROM TableA A** **LEFT JOIN TableB B ON A.common_column = B.common_column;**

Views (Virtual Tables)

1. Views can act as a proxy or virtual table.
2. Views reduce the complexity of SQL queries and provide secure access to underlying tables.
3. Views are a special version of tables in SQL.
4. They provide a virtual table environment for various complex operations.
5. You can select data from multiple tables, or you can select specific data based on certain criteria in views.
6. It does not hold the actual data; it holds only the definition of the view in the data dictionary.

Types of Views

1. **Simple View:** A view based on only a single table, which doesn't contain `GROUP BY` clause and any functions.

2. **Complex View:** A view based on multiple tables, which contain `GROUP BY` clause and functions.
3. **Inline View:** A view based on a subquery in `FROM` Clause, that subquery creates a temporary table and simplifies the complex query.
4. **Materialized View:** A view that stores the definition as well as data. It creates replicas of data by storing it physically.

A View is a virtual table based on the result of a query. You call (use) a view just like a table.

Syntax: **CREATE VIEW StudentView AS** **SELECT StudentID, Name, Age FROM Students WHERE Age > 18;**	**Call (Use) the View**: SELECT * FROM StudentView; **Drop a View**: DROP VIEW StudentView;

Function Concepts

MySQL control flow function, which returns a value based on the given conditions. In other words, the IF function is used for validating a function in MySQL. The IF function returns a value YES when the given condition evaluates to true and returns a NO value when the condition evaluates to false. It returns values either in a string or numeric form depending upon the context in which this function is used. Sometimes, this function is known as IF-ELSE and IF THAN ELSE function.

Keyword	Purpose
IF...THEN...ELSE	Conditional branching
CASE	Multi-branch selection
LOOP	Basic loop
WHILE	Loop with pre-condition
REPEAT	Loop with post-condition

CURSOR	Row-by-row data processing
LEAVE	Exit from loop
DECLARE HANDLER	Error/exception handling

Example Program

```
DELIMITER $$

CREATE FUNCTION addNum(a INT, b
INT)
RETURNS INT
DETERMINISTIC
BEGIN
   RETURN a + b;
END$$

DELIMITER ;
SELECT addNum(10, 20);
```

```
DELIMITER $$

CREATE FUNCTION
yearly_salary(monthly DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
   RETURN monthly * 12;
END$$

DELIMITER ;
SELECT name, yearly_salary(salary)
FROM employees;
```

```
DELIMITER $$

CREATE FUNCTION fullName(fname
VARCHAR(50), lname VARCHAR(50))
RETURNS VARCHAR(120)
DETERMINISTIC
BEGIN
   RETURN CONCAT(fname, ' ', lname);
END$$

DELIMITER ;
```

```
DELIMITER $$

CREATE FUNCTION grade(sal
DECIMAL(10,2))
RETURNS VARCHAR(20)
DETERMINISTIC
BEGIN
   IF sal >= 50000 THEN
      RETURN 'High';
   ELSEIF sal >= 30000 THEN
      RETURN 'Medium';
   ELSE
      RETURN 'Low';
   END IF;
END$$

DELIMITER ;
```

Stored Procedures

1. A Stored Procedure is a saved SQL block that can be reused. It can take parameters and include logic like conditions and loops.
2. Stored Procedure means one of the best methods in sql and it is reusable functionality.
3. A MySQL stored procedure is a group of pre-compiled SQL statements that can be reused anytime.
4. Stored procedures can be used to perform different database operations such as inserting, updating, or deleting data.

Syntax:

```
DELIMITER //
CREATE PROCEDURE GetAllStudents()
BEGIN
  SELECT * FROM Students;
END //
DELIMITER ;
```

WITH PARAMETER Common Store

```
DELIMITER $$ or //
CREATE PROCEDURE
PRODUCER_NAME([IN|OUT|INOUT]parameter_name
parameter_type)
BEGIN
      //statements of your   code……
END $$ or //
DELIMITER ;
```

Calling method:

CALL procedure_name();

CALL Procedure_name(parameter);

Deleting procedure:

DROP procedure_name(); or

DROP procedure[IF EXISTS] procedure_name; or

DROP procedure if exists procedure_name;

'@'session variable

Used for code reusability, performance, and security.

Advantages:

1. Reduce the network traffic
2. Centralized business logics
3. Secure
4. Reusable code segment or functionality

Disadvantages

1. Hard to maintains
2. Hard to troubleshoots
3. Resource maintenance is a little bit tough.

5. No need to type multi line code repeatedly	
Methods (parameter passing 3) **IN ⇒input** **OUT -==>output @symbol** **INOUT -==>in and output method @symbols**	

Cursors

A Cursor is used to loop through rows in a result set one at a time.

Cursor lets you process rows one at a time. You don't "call" a cursor like a procedure — you **declare, open, fetch, and close** it inside a procedure or block.

A **cursor** in SQL is used to **iterate over rows** from a query **one at a time**, allowing row-by-row processing in stored procedures.

When to Use a Cursor?

Use a cursor when:

- You need **procedural logic** (e.g., conditional checks per row)
- You can't solve the problem easily with set-based SQL
- You want to **update, insert, or analyze** data row-by-row

Syntax:

```
        DELIMITER //
CREATE PROCEDURE ShowStudentNames()
BEGIN
   DECLARE done INT DEFAULT 0;
   DECLARE student_name VARCHAR(100);
   DECLARE cur CURSOR FOR SELECT Name FROM Students;
   DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;
   OPEN cur;
   read_loop: LOOP
      FETCH cur INTO student_name;
      IF done THEN
         LEAVE read_loop;
      END IF;
      SELECT student_name; -- or do other processing
   END LOOP;
   CLOSE cur;
END //
DELIMITER ;
```

Call the Procedure with Cursor:

CALL ShowStudentNames();

Drop a Procedure with Cursor:
DROP PROCEDURE ShowStudentNames;

TRIGGER

A Trigger runs automatically when an event like INSERT, UPDATE, or DELETE happens on a table.

SQL Triggers

A **trigger** in SQL is a **stored procedure** that is **automatically executed** in response to certain events on a table such as `INSERT`, `UPDATE`, or `DELETE`.

Concept	Explanation
Trigger Time	`BEFORE` or `AFTER` the triggering event
Trigger Event	`INSERT`, `UPDATE`, or `DELETE`
Target Table	The table the trigger is defined on
FOR EACH ROW	Means trigger executes once for **each row** affected

CREATE TRIGGER trigger_name {BEFORE | AFTER} {INSERT | UPDATE | DELETE} ON table_name FOR EACH ROW BEGIN -- Trigger logic here END;	Drop trigger triggername

Syntax: CREATE TRIGGER LogAfterInsert	This trigger runs **automatically** when you insert into the Students table:

AFTER INSERT ON Students FOR EACH ROW BEGIN INSERT INTO LogTable(Activity) VALUES (CONCAT('New student added: ', NEW.Name)); END;	INSERT INTO Students (Name, Age) VALUES ('Alice', 22); **Drop a Trigger**: DROP TRIGGER LogAfterInsert;

DCL (Data Control Language)

Command	Description	Example
`CREATE USER`	**Creates a new database user.**	`CREATE USER 'user1'@'localhost' IDENTIFIED BY 'password';`
`GRANT`	**Grants privileges to a user.** **Insert ,update,delete**	`GRANT SELECT, INSERT ON batch13.* TO 'user1'@'localhost';`
`REVOKE`	**Removes privileges from a user.** **Insert ,update,delete**	`REVOKE INSERT ON batch13.* FROM 'user1'@'localhost';`
`FLUSH PRIVILEGES`	**Reloads privilege changes.**	`FLUSH PRIVILEGES;`

TCL (Transaction Control Language)

Command	Description	Example
`START TRANSACTION`	**Starts a new transaction.**	`START TRANSACTION;`
`SAVEPOINT`	**Creates a savepoint within the transaction.**	`SAVEPOINT sp1;`
`ROLLBACK TO SAVEPOINT`	**Rolls back to a specific savepoint.**	`ROLLBACK TO sp1;`
`ROLLBACK`	**Undoes all changes in the current transaction.**	`ROLLBACK;`
`COMMIT`	**Permanently saves all changes.**	`COMMIT;`

Keys Introduction

Keys are special columns (or sets of columns) used to uniquely identify rows in a database table and to create relationships between tables.

Primary Key	Uniquely identifies each row, no NULLs, only one per table
Foreign Key	Links two tables, refers to the Primary Key of another table
Unique Key	Ensures unique values, allows NULLs, multiple per table
Candidate Key	Any column that can be a Primary Key (only one is chosen)
Super Key	Any set of attributes that uniquely identifies a row
Composite Key	A key made up of multiple columns

Alternate Key	A Candidate Key that is not the Primary Key
Surrogate Key	An artificial key (e.g., AUTO_INCREMENT)

PRIMARY KEY

A key selected to identify each record uniquely in a table. Columns marked as primary keys aren't allowed to have null values. It keeps unique values throughout the column.
A table is expected to have only one primary key, although there are cases where a table can derive its primary key using two columns.
Primary keys can be defined using the create statement when creating a table

Foreign Key

Foreign Key is a column in a table that is used as the Primary key in another table. It can accept multiple nulls and duplicate values.

"One table primary key,when used another one table reference key that primary key was called as foreign key depends on table data sets" **or**

"The primary key of one table, when used as a reference in another table, is called a foreign key. It establishes a relationship between the two tables based on their data."

UNIQUE KEY

A **Unique Key** also ensures that all values in a column (or group of columns) are **unique**, but unlike the primary key:

- A table **can have multiple unique keys**
- It **can contain NULL values** (in some DBMS like MySQL)

Foreign key syntax:

New foreign key add **FOREIGN KEY (child_column)**	**Existing table adding foreign key** **ALTER TABLE child_table** **ADD CONSTRAINT fk_name** **FOREIGN KEY (child_column)**

REFERENCES parent_table(parent_column);	**REFERENCES parent_table(parent_column);**

PRIMARY KEY	CREATE TABLE Students (StudentID INT PRIMARY KEY, Name VARCHAR(100));
UNIQUE KEY	CREATE TABLE Users (UserID INT PRIMARY KEY, Email VARCHAR(100) UNIQUE);
FOREIGN KEY	CREATE TABLE Enrollments (EnrollID INT PRIMARY KEY, StudentID INT, FOREIGN KEY (StudentID) REFERENCES Students(StudentID));

Normalization

Normalization is the process of organizing data in databases to eliminate data redundancy and maintain data integrity.
It involves dividing large and complex tables into smaller, more manageable ones based on logical data relationships. This entire process is known as normalization.

Reduces redundancy: Redundancy is when the same information is stored multiple times, and a good way of avoiding this is by splitting data into smaller tables.
Improves query performance: You can perform faster query execution on smaller tables that have undergone normalization.
Minimizes update anomalies: With normalized tables, you can easily update data without affecting other records.
Enhances data integrity: It ensures that data remains consistent and accurate.

ACID ⇒ Atomicity Consistency Isolation Durability

Normalization is classified into six major forms:
1NF, 2NF, 3NF, BCNF (also called 3.5NF), 4NF, and 5NF.

1NF (First Normal Form):

1. **Atomic Values;**Ensures that the table has atomic (indivisible) values.
2. **No Repeating Groups**:Each column should contain only one value per row (no repeating groups or arrays).
3. **Uniqueness**: is presumably the primary key (it's unique), so uniqueness is maintenance must.

2NF (Second Normal Form):

1. Satisfies all the rules of 1NF.
2. Removes **partial dependencies** — meaning no non-prime attribute should depend on part of a candidate key (only applies to composite keys).

3NF (Third Normal Form):

1. Satisfies 1NF and 2NF.
2. Removes **transitive dependencies** — i.e., non-key attributes should not depend on other non-key attributes.

BCNF (Boyce-Codd Normal Form / 3.5NF):

1. A stricter version of 3NF.
2. Every determinant must be a super key.

4NF (Fourth Normal Form):

1. Satisfies all previous normal forms.
2. Removes multi-valued dependencies.

5NF (Fifth Normal Form):

1. Removes join dependencies and ensures the table is broken down into the smallest pieces without losing data through joins.

```
function funName() {
    return
}

if () {
    return
}
else {
    return
}


--mysql

function funName()
begin

end;


; ==> delimiter

@ ==> session variable



use Batch14;

show tables;

select * from employee;

select * from emptable;

select * from gtable;

select * from dtable;
```

```
--Join Concepts
--inner join
select eid, ename, edept, dname from emptable
INNER JOIN dtable On edept = did;

select emptable.eid, emptable.ename, emptable.edept, dtable.dname from
emptable
INNER join dtable on emptable.edept = dtable.did;

select ET.eid, ET.ename, ET.edept, DT.dname from emptable as ET
INNER JOIN dtable as DT On ET.edept = DT.did;

select eid, ename, edept, dname, egender, gname from emptable
INNER JOIN dtable On edept = did
inner join gtable on egender = gid;

--left join
select emptable.eid, emptable.ename, emptable.edept, dtable.dname from
emptable
Left join dtable on emptable.edept = dtable.did;

select emptable.eid, emptable.ename, emptable.edept, dtable.dname from
emptable
Left outer join dtable on emptable.edept = dtable.did;

--right join
select emptable.eid, emptable.ename, emptable.edept, dtable.dname from
emptable
RIght join dtable on emptable.edept = dtable.did;

select emptable.eid, emptable.ename, emptable.edept, dtable.dname from
emptable
RIght outer join dtable on emptable.edept = dtable.did;

--Full join
select emptable.eid, emptable.ename, emptable.edept, dtable.dname from
emptable
RIght join dtable on emptable.edept = dtable.did
union
```

```
select emptable.eid, emptable.ename, emptable.edept, dtable.dname from
emptable
Left join dtable on emptable.edept = dtable.did;


--Cross join
select emptable.eid, emptable.ename, emptable.edept, dtable.dname from
emptable
Cross join dtable;

--view concepts

create view rightjoinTable As select emptable.eid, emptable.ename,
emptable.edept, dtable.dname from emptable
RIght outer join dtable on emptable.edept = dtable.did;

select * from rightjoinTable;

drop view rightjoinTable;

desc emptable;

insert into emptable values(6, "maha", 345678901, 19, 102, 202);

select * from employee where empdepartment = "IT";

create view empdept As select * from employee where empdepartment = "IT";

select * from empdept;


--Function
--Function Classification
-- with parameter
        --without parameter


select 10 + 20 as number;
--without parameter
delimiter ##
```

```
    create function addTwonum()
    returns int
deterministic
begin
return 10 + 50;
    end  ##
delimiter;

select addTwonum();

-- with parameter
 delimiter %%
    create function sumTwonum(num1 int, num2 int)
    returns int
deterministic
begin
return num1 + num2;
end %%
    delimiter;

 select sumTwonum(100, 200) as sss;


--real time use

select * from employee;
select * from e1;

delimiter **
    create function lpacal(esalary decimal(10, 2))
    returns decimal(10, 2)
deterministic
begin
return esalary * 12;
end **
    delimiter;

 select empid, empname, empsalary from employee;
 select empid, empname, empsalary, lpacal(empsalary) as lpa from employee;
```

```
drop function lpacal;
drop function if exists lpacal;
```

```
select empid,empname,empsalary from employee;

select (45000 * 12) as hafillpa;
select (45000 * 12) as hafillpa;
select (45000 * 12) as hafillpa;

delimiter $$
    create function emplpas(esal decimal(5,2))
    returns decimal(10,2)
    deterministic
    begin
        return esal * 12;
    end $$
delimiter ;
drop function emplpas;

select empid,empname,empsalary,emplpas(empsalary) as lps from employee;

select max(empsalary) from employee;
select min(empsalary) from employee;

delimiter &&
    create function salRange(esalary decimal(10,2))
    returns varchar(100)
    deterministic
    begin
        if esalary >= 50000.00 then
            return "First salary person...!";
        elseif esalary >= 46000.00 and esalary <= 49000.00 then
```

```
            return "second salary person...!!";
        elseif esalary <= 45000.00 then
            return "Third salary person...!!!";
        else
            return "none of them recive low salary" ;
        end if;
    end &&
delimiter ;

select salRange(566666.00);
select salRange(5200.00) as firstsal;


select empid,empname,empsalary,emplpas(empsalary) as
lps,salRange(empsalary) as salRange from employee;


-- SP ==> Store Procedure

delimiter ??
    create procedure empTableQ()
    begin
        select empid,empname,empmobile,empsalary from employee;
        select empid,empname,empmobile,empsalary from employee where
empdepartment = "IT";
        select empid,empname,empmobile,empsalary from employee where
empsalary =
        (select max(empsalary) from employee where empsalary <
         (select max(empsalary) from employee));
        select max(empsalary) from employee;
        select min(empsalary) from employee;

    end ??
delimiter ;

call empTableQ();
drop procedure empTableQ;

-- IN,OUT,INOUT
-- IN
```

```
select empid,empname,empmobile,empsalary from employee;
select empid,empname,empmobile,empsalary from employee where empid = 10;
select empid,empname,empmobile,empsalary from employee where empid = 20;
select empid,empname,empmobile,empsalary from employee where empid = 18;

delimiter ^^
    create procedure finemp(IN eid int)
    begin
        select empid,empname,empmobile,empsalary from employee where empid
= eid;
    end ^^
delimiter ;

call finemp(2);
call finemp(10);


delimiter ^^
    create procedure findempbymobile(IN emobile bigint)
    begin
        select empid,empname,empmobile,empsalary from employee where
empmobile = emobile;
    end ^^
delimiter ;

call findempbymobile();

-- OUT

delimiter **
    create procedure justtry(OUT enumeber int)
    begin
        declare mynum int;
        set enumeber = enumeber + 10;
    end **
delimiter ;

set @myValue = 20;
call justtry(@myValue);
select @myValue;
```

```
select empid,empname,empmobile,empsalary from employee where empid = 10;

update employee set empsalary = 50000.00 where empid = 10;

-- INOUT

select * from e1;

delimiter ??
    create procedure salaryhikecom(INOUT esal decimal(10,2))
    begin
            update e1 set empsalary = empsalary + esal;
    end ??
delimiter ;

set @hike = 3000.00;
call salaryhikecom(@hike);
select @hike;

delimiter **
    create procedure hikeEMp(INOUT ehikesalary decimal(10,2))
    begin
        update e1 set empsalary = empsalary + ehikesalary;
    end **
delimiter ;

drop procedure hikeEMp;

set @hike = 2000.00;
call hikeEMp(@hike);
select @hike;


-- aternative method

delimiter ??
    create procedure salaryhike(IN esal decimal(10,2) , IN eid int)
    begin
```

```
            update employee set empsalary = empsalary + esal where empid =
eid;
    end ??
delimiter ;

call salaryhike(5000.00, 10);
call finemp(10);


-- cursor

select * from employee;
select empid,empname,empmobile,empsalary from employee;

select empid,empname,empmobile,empsalary from employee where empid =1;
call finemp(3);

delimiter ##$
    create procedure empCursor()
    begin
        declare ok int default false;
        declare eid int;
        declare ename varchar(100);
        declare emobile bigint;
        declare myCur cursor for select empid,empname,empmobile from
employee;
        declare continue handler for not found set ok = true;
        open myCur;
            read_lopp: LOOP
                fetch myCur into eid,ename,emobile;
                    if ok then
                        LEAVE read_lopp;
                    end if;
                select eid As EMPID, ename AS EMPNAME, emobile As
EMPMOBILE;
            end LOOP;
        close myCur;
    end ##$
delimiter ;
```

```
call empCursor();
drop procedure empCursor;
```

```
use batch14;
select * from employee;
select empid,empname,empsalary from employee;
select empid,empname,empsalary from employee where empid =4;
call finemp(22);

delimiter %%
    create procedure findempcur()
    begin
        declare done int default 0;
        declare eid int;
        declare ename varchar(100);
        declare esalary decimal(10,2);
        declare myfindcur cursor for select empid,empname,empsalary from
employee;
        declare continue handler for not found set done=1;
        open myfindcur;
            read_loop: LOOP
                fetch myfindcur into eid,ename,esalary;
                    if done then
                        leave read_loop;
                    end if;
                select eid as EMPID,ename as EMPNAME,esalary as EMPSALARY;
            end LOOP;
        close myfindcur;
    end %%
delimiter ;

call findempcur();

-- Trigger concepts
use batch14;
select * from emptable;
```

```
update emptable set ename = "ganapathi" where eid=1;
delete from emptable where eid = 4;
insert into emptable values (7,"visal",3456789012,20,103,203);

create table tritable(
    tid int primary key auto_increment,
    mtid int not null,
    taction varchar(50) not null,
    ttime timestamp default current_timestamp
);

select * from tritable;
insert into tritable(mtid,taction) values ();

delimiter $$
    create trigger updatetri
    after update on emptable
    for each row
    begin
        insert into tritable(mtid,taction)
        values (old.eid,"Update a exiting data");
    end $$
delimiter ;

select * from tritable;
select * from emptable;
update emptable set ename = "levon" where eid=2;

delimiter $$
    create trigger deletetri
    after delete on emptable
    for each row
    begin
        insert into tritable(mtid,taction)
        values (old.eid,"delete a exiting data in the main table " );
    end $$
delimiter ;


select * from tritable;
```

```sql
select * from emptable;
update emptable set ename = "nasika" where eid=6;
delete from emptable where eid = 1;
```

```sql


-- DCL & TCL
CREATE USER 'levon'@'localhost' IDENTIFIED BY 'love@123';
GRANT SELECT, INSERT ON batch14.emptable  TO 'levon'@'localhost';
grant update on batch14.emptable to 'levon'@'localhost';
grant delete on batch14.emptable to 'levon'@'localhost';
revoke delete on batch14.emptable from 'levon'@'localhost';

-- DCL & TCL
select * from emptable;
update emptable set edept = 204 where eid = 3;

start transaction;
update emptable set edept = 201 where eid = 3;
savepoint edeptupdate;
commit;

rollback to edeptupdate;


start transaction;
update emptable set edept = 205 where eid = 3;
commit;
savepoint ss1;

start transaction;
update emptable set edept = 207 where eid = 3;

ROLLBACK;

select * from emptable;
```

```sql
-- Key and NOrmalization  --
use batch14;
select * from emptable;
select * from gtable;
select * from dtable;

desc emptable;
desc gtable;
desc dtable;

insert into emptable values (9,"aaa",4567890123,100,104,107);

create table eee(
    eeid int primary key,
    eename varchar(50),
    eemobile bigint unique not null,
    eegender int,
    eedept int,
    foreign key(eegender) references gtable(gid),
    foreign key(eedept) references dtable(did)
);

alter table gtable modify column gid int primary key;
alter table dtable modify column did int primary key;


insert into eee values (1,"vgr",7502132895,102,204);
insert into eee values (2,"levon",8502132895,102,202);
insert into eee values (3,"pavi",6502132895,101,205);
insert into eee values (4,"rajarani",8602132895,103,202);

select * from eee;
select * from gtable;
select * from dtable;

insert into eee values (5,"ghjk",2345678901,104,205);
insert into eee values (5,"ghjk",2345678901,103,208);

-- Normatizations
```

```
select * from employee;
select * from employeestable;


create table students_course_table(
    sid int,
    sname text,
    scourse text
);
insert into students_course_table values
(1,"arun,kumar", 'python,java'),
(2,"santhosh","mern"),
(3, 'rai, kumar','python,java,c++');

select * from students_course_table;

create table snametable(
snid int ,
sfname text,
ssname text
);

insert into snametable values
(101,"arun","kumar"),
(102,"santhosh",""),
(103,"rai","kumar");

select * from students_course_table;
select * from snametable;

create table scourse(
    scid int,
    scourse text
);
insert into scourse values
(1,"java"),
(1,"python"),
(2,"mern"),
(3,"python"),
(3,"jave"),
```

```
(3,"c++");

select * from students_course_table;
select * from scourse;

create table scourstable(
    cid int,
    cname text
);

insert into scourstable values
(201, "JAva"),
(202,"python"),
(203,"mern"),
(204,"c++");

select * from students_course_table;
select * from scourse;
select * from scourstable;

create table sfinalcourse(
    scid int,
    sc1 int,
    sc2 int,
    sc3 int
);

insert into sfinalcourse values
(301,201,202,0),
(302,203,0,0),
(303,201,'202','204');

select * from students_course_table;
select * from scourse;
select * from scourstable;
select * from sfinalcourse;
select * from snametable;

create table Final_normal_studnets_table(
    sid int primary key,
```

```
    sname int,
    scourse int
);
insert into Final_normal_studnets_table values
(1,101,301),
(2,102,302),
(3,103,303);



select * from students_course_table;
select * from scourse;
select * from scourstable;
select * from sfinalcourse;
select * from snametable;
select * from Final_normal_studnets_table;


create table full_normal_table(
    fid int primary key auto_increment,
    fname varchar(100) not null,
    fmobile bigint unique not null,
    femail varchar(50) unique not null,
    fage int check(fage >= 18) not null,
    fgender varchar(20) not null,
    ffgender enum("Male","Female","others") not null,
    fsalary decimal(10,2) not null ,
    fforiegngender int not null,
    fdept int not null,
    fjoindate timestamp default current_timestamp,
    fstatus varchar(10) default("Active"),
    index findex(fmobile,femail, fage),
    foreign key  (fforiegngender) references gtable(gid),
    foreign key (fdept) references dtable(did)
);
```